

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет Программной инженерии и компьютерной техники

Образовательная программа Системное и прикладное программное обеспечение

Направление подготовки (специальность) Программная инженерия

О Т Ч Е Т

о производственной, технологической (проектно-технологической) практике

Тема задания: Разработка механизма автоматической анонимизации персональных и секретных данных в журналах сетевого взаимодействия на основе метаданных типов

Обучающийся Барсуков Максим Андреевич, Р3415

Руководитель практики от профильной организации: Доронин Антон Викторович, Software Engineer L4, ООО "МВС", Команда Kotlin API and Gateway

Руководитель практики от университета: Маркина Татьяна Анатольевна, старший преподаватель

Дата 05.04.2026

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
Место прохождения и моя роль	9
Технологический стек	9
Контекст и важность задачи	9
Структура отчёта	10
2 ФОРМИРОВАНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ	11
2.1 Получение постановки задачи	11
2.2 Формулирование и согласование требований	11
2.3 Заполнение и утверждение индивидуального задания	12
2.4 Рефлексия	12
3 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И ТРЕБОВАНИЙ К СИСТЕМЕ	13
3.1 Постановка задачи на проектирование	13
3.2 Первая итерация: маскировка через Jackson-десериализацию (MWSDEVP-5799)	13
3.2.1 Выбранный подход	13
3.2.2 Выполнение	13
3.2.3 Первичные результаты	14
3.2.4 Что пошло не так (и почему я этого не ожидал)	14
3.2.5 Что я из этого понял	15
3.2.6 Временное решение	16
3.3 Вторая итерация: прямой обход JSON-дерева и двухуровневая маскировка (MWSDEVP-6105)	17
3.3.1 Анализ проблемы и поиск нового подхода	17
3.3.2 Первый прототип	17
3.3.3 Ключевые архитектурные решения второй итерации	18
3.3.4 Поток данных в новой архитектуре	19
3.3.5 Что дала новая архитектура	19
3.3.6 Что осталось за рамками проектирования	20
3.4 Итоги проектирования	20
4 РЕАЛИЗАЦИЯ АНАЛИЗАТОРА ТИПОВОЙ СТРУКТУРЫ	22
4.1 Постановка задачи и первые мысли	22
4.2 Как я подошёл к решению	22
4.3 Проблема №1: как получить информацию о типах	22
4.4 Проблема №2: обёртки	23

4.5 Проблема №3: рекурсивные ссылки.....	23
4.6 Проблема №4: производительность и кэширование.....	24
4.7 Как я собирал метаданные.....	25
4.8 Проблема №5: raw-типы.....	26
4.9 Тестирование.....	27
4.10 Результаты этапа.....	27
5 РЕАЛИЗАЦИЯ БАЗОВОГО МЕХАНИЗМА МАСКИРОВАНИЯ JSON-СТРУКТУР.....	28
5.1 Постановка задачи.....	28
5.2 Простой обход объекта.....	28
5.3 Проблема производительности и Copy-on-Write.....	28
5.4 Вложенные объекты.....	29
5.5 Коллекции.....	29
6 РЕАЛИЗАЦИЯ ПОДДЕРЖКИ СЛОЖНЫХ ВЛОЖЕННЫХ СТРУКТУР И ОБЁРТОК.....	30
6.1 Постановка задачи.....	30
6.2 Словари.....	30
6.3 Рекурсивные ссылки: опять visited set.....	31
6.4 Обёртки.....	32
6.5 Nullable.....	32
6.6 Обработка ошибок.....	32
6.7 Тестирование.....	32
6.8 Результаты этапа.....	33
7 ТЕСТИРОВАНИЕ МЕХАНИЗМА АНОНИМИЗАЦИИ.....	34
7.1 Постановка задачи.....	34
7.2 Юнит-тестирование.....	34
7.3 Интеграционное тестирование.....	34
7.4 Сквозное тестирование.....	35
7.4.1 Задача MWSDEVP-5985.....	35
7.4.2 Проверка.....	36
7.5 Проверка производительности.....	36
7.6 Выводы по тестированию.....	38
7.7 Результаты этапа.....	38
8 ВНЕДРЕНИЕ МЕХАНИЗМА В СИСТЕМУ ЛОГИРОВАНИЯ.....	39
8.1 Постановка задачи.....	39
8.2. Интеграция через Spring Boot AutoConfiguration.....	39
8.3 Настройка через application.yml.....	39
8.4 Проверка в среде, подобной production.....	39

8.5 Что я понял.....	40
8.6 Результаты этапа.....	40
9 ОФОРМЛЕНИЕ ОТЧЕТНЫХ ДОКУМЕНТОВ.....	41
9.1 Подготовка к написанию отчёта.....	41
9.2 Процесс написания отчёта.....	41
9.3 Трудности при оформлении.....	42
9.4 Соблюдение требований к оформлению.....	42
9.5 Что я понял.....	42
9.6 Результаты этапа.....	43
ЗАКЛЮЧЕНИЕ.....	44
ПРИЛОЖЕНИЕ А.....	48
Функциональные требования.....	48
Нефункциональные требования.....	49
Задача.....	49
ПРИЛОЖЕНИЕ Б.....	50
ПРИЛОЖЕНИЕ В.....	93
ПРИЛОЖЕНИЕ Г.....	105
ПРИЛОЖЕНИЕ Д.....	203
ПРИЛОЖЕНИЕ Е.....	237

ВВЕДЕНИЕ

Тема: Разработка механизма автоматической анонимизации персональных и секретных данных в журналах сетевого взаимодействия на основе метаданных типов.

Цель: Целью производственной практики является закрепление и углубление профессиональных компетенций в области разработки и сопровождения высоконагруженных корпоративных систем, а также приобретение опыта самостоятельного решения реальных инженерных задач в условиях production-окружения. В соответствии с индивидуальным заданием, моя работа была сфокусирована на критической проблеме платформы — обеспечении безопасности access-логов путём автоматической маскировки конфиденциальных данных в теле HTTP-запросов и ответов, для того, чтобы можно было включить логирование тел по умолчанию без риска утечки данных.

Выполнение практики состоит из прохождения этапов, приведённых в Таблице 1 (в соответствии с планом-графиком из индивидуального задания).

Таблица 1 – Этапы производственной практики

№ этапа	Название этапа	Задание
1	Инструктаж обучающегося	Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка
2	Формирование индивидуального задания	Ознакомиться с постановкой задачи от руководителя практики. Сформулировать и согласовать функциональные и нефункциональные требования к механизму автоматической анонимизации. Заполнить и утвердить индивидуальное задание в личном кабинете практики. Критерии приёмки: составлен и согласован документ, содержащий списки функциональных и нефункциональных требований. Результат для отчёта: согласованный список функциональных и нефункциональных требований.

3	Проектирование архитектуры и требований к системе	На основе утверждённого задания разработать детальные требования: поддерживаемые типы данных (простые поля, вложенные объекты, коллекции, ассоциативные массивы, обёртки), поведение при рекурсивных ссылках, производительность, безопасность. Разработать архитектурную схему компонента и определить точки интеграции в систему логирования. Критерии приёмки: схема согласована с руководителем, все типы покрыты. Результат для отчёта: документ с архитектурной схемой.
4	Реализация анализатора типовой структуры	Разработать компонент, выполняющий рекурсивный анализ типовой структуры передаваемых объектов на основе метаданных классов. Обеспечить обработку вложенных объектов, коллекций, ассоциативных-массивов и обёрток с параметризованными типами. Реализовать кэширование метаданных. Критерии приёмки: все предусмотренные сценарии использования анализатора покрыты автоматизированными тестами; подтверждена стабильная работа на граничных и ошибочных входных данных; механизм интегрируется в систему без деградации её производительности. Результат для отчёта: исходный код модуля и отчёт о покрытии тестами.

5	Реализация базового механизма маскирования JSON-структур	Разработать компонент для рекурсивного обхода JSON-дерева и замены значений чувствительных полей на маскирующее значение. Обеспечить корректную обработку простых объектов, вложенных объектов и коллекций. Реализовать интеграцию с анализатором типовой структуры. Критерии приёмки: все чувствительные поля в базовых структурах заменяются на ****, ложных срабатываний нет. Результат для отчёта: листинги методов и примеры работы на тестовых данных.
6	Реализация поддержки сложных вложенных структур и обёрток	Расширить механизм маскирования для поддержки сложных сценариев: ассоциативные массивы внутри списков, generic-обёртки, nullable-значения и рекурсивные ссылки на любом уровне вложенности полей и типов. Обеспечить устойчивость механизма к циклическим зависимостям (без заикливания и переполнения стека) и к ошибкам десериализации (некорректный JSON, несоответствие типов). Критерии приёмки: механизм корректно заменяет чувствительные поля на маскирующее значение во всех указанных сценариях, включая глубоко вложенные структуры; при обнаружении циклической ссылки механизм прекращает рекурсивный обход и продолжает работу без исключений; при ошибке парсинга или несоответствии типов механизм скрывает тело запроса и записывает предупреждение. Результат для отчёта: исходный код расширенного модуля маскирования, набор интеграционных тестов, демонстрирующих работу на сложных структурах, скриншоты прохождения тестов.

7	Тестирование механизма анонимизации	Провести многоуровневое тестирование разработанного механизма: 1) Юнит-тестирование — проверка корректности работы модулей на изолированных сценариях (простые объекты, вложенные структуры, коллекции). 2) Интеграционное тестирование — проверка взаимодействия анализатора и маскировщика на сложных типах (например, <code>List<Map<CloudOptional<Int?>?, ProfileRequest?>?></code>). 3) Сквозное тестирование на тестовом сервисе — генерация моделей и клиентов по OpenAPI-спецификации для сервиса с чувствительными данными, отправка запросов и проверка маскировки чувствительных полей. 4) Проверка производительности — оценка влияния механизма анонимизации на время обработки запросов и потребление ресурсов при различных объёмах данных (от простых сообщений до сложных структур объёмом до 10 МБ); подтверждение отсутствия значимого влияния на работу сервиса. Критерии приёмки: прохождение тестов, отсутствие утечек, производительность в пределах нормы. Результат для отчёта: отчёт о тестировании, скриншоты логов до/после, графики производительности.
---	-------------------------------------	---

8	Внедрение механизма в систему логирования	Интегрировать разработанный механизм в существующую систему журналирования сетевого взаимодействия. Настроить подключение к компонентам маршрутизации запросов и ответов. Обеспечить минимальное влияние на производительность. Провести проверку на отсутствие утечек конфиденциальной информации в среде, подобной производственной. Критерии приёмки: в access-логах всех тестовых endpoint чувствительные поля замаскированы, сервис работает стабильно без непредвиденных ошибок. Результат для отчёта: фрагмент конфигурации, примеры access-логов после внедрения.
9	Оформление отчетных документов	Написание отчёта, оформление в соответствии с требованиями, получение отзыва от руководителей. Оформление отчётных документов в соответствии с требованиями: 1. Оформление отчёта должно быть выполнено в соответствии с методическим пособием (https://books.ifmo.ru/file/pdf/2622.pdf , разделы "Набор текста ", "Нумерация страниц и разделов ", "Заголовки разделов ", "Оформление рисунков ", "Оформление таблиц ", "Оформление формул ", "Оформление ссылок на источники ", "Оформление списка использованных источников ", "Оформление приложений"). 2. Структура документа: титульный лист, введение, основная часть, заключение, приложения. 3. В основной части подробно описывается выполнение задач этапов 2-8.
10	Получение отзыва руководителя практики	Получить отзыв с оценкой у руководителя практики в модуле Практики.

Место прохождения и моя роль

Производственная практика проходила в ООО «МВС» в рамках стажерской группы команды Kotlin API and Gateway (подразделение Development Platform). Моя роль — стажёр-разработчик, во время прохождения практики я занимался компонентами логирования и безопасности общей JVM-платформы, которую потребляют десятки других команд при создании своих Kotlin-сервисов внутри МВС. В рамках практики я выполнял задачи, связанные с доработкой библиотеки `devp-access-logging-spring-webflux` и сопутствующих модулей (`devp-access-logging-webflux-client-spring-boot-starter`, `cloud-sensitive`, `devp-access-logging-webflux-server-spring-boot-starter`), которые отвечают за access-логирование HTTP-запросов и ответов во всех сервисах, использующих платформу. Работа велась в реальном production-контексте: код, написанный в ходе практики, прошёл регрессионное тестирование, а после согласования — включён в очередной релиз платформы и стал доступен всем сервисам-потребителям.

Технологический стек

В ходе практики я использовал следующий стек:

- Язык: Kotlin
- Фреймворк: Spring Boot, Spring WebFlux
- Работа с JSON: Jackson (ObjectMapper, JsonNode, кастомные сериализаторы/десериализаторы)
- Логирование: kotlin-logging, SLF4J
- Тестирование: JUnit 5, Kotest, Mockito, WireMock, Testcontainers
- Сборка и зависимости: Gradle
- Дополнительно: Reactor

Контекст и важность задачи

Многие сервисы внутри МВС обрабатывают тысячи HTTP-запросов каждый день, и для диагностики проблем важно логировать тела запросов и ответов (access-логи). Однако в этих телах часто встречаются конфиденциальные данные: пароли, токены доступа, API-ключи, номера кредитных карт и т.п. Логирование таких данных в открытом виде нарушает политики безопасности (GDPR, 152-ФЗ) и создаёт риск утечки. В платформе уже существует аннотация-обёртка `CloudSensitive<T>`, которую разработчики используют для маркировки чувствительных полей в DTO. Однако автоматического механизма маскировки этих полей при логировании не было — `CloudSensitive<T>` никак не влиял на формирование access-логов. Именно

поэтому логирование тел запросов и ответов было отключено по умолчанию, и команды включали его редко и с осторожностью, понимая риски.

Таким образом, передо мной стояла задача спроектировать и реализовать механизм автоматической маскировки, который бы:

- Использовал уже существующую разметку (без необходимости дорабатывать DTO).
- Работал на уровне платформы, не требуя изменений в коде сервисов-потребителей.
- Был производительным (платформенный код не должен становиться узким местом) и надёжным (ошибка в платформе может привести к падению множества сервисов), чтобы можно было включить логирование тел по умолчанию без опасений за утечку данных.
- Поддерживал сложные структуры (вложенные объекты, коллекции, карты, generic-обёртки) и корректно обрабатывал ошибки.
- Был обратно совместим — нельзя ломать API, которое уже используется.

Структура отчёта

Отчёт построен в соответствии с этапами индивидуального задания. В основной части последовательно описаны: формирование требований (этап 2), проектирование архитектуры с эволюцией подходов (этап 3), реализация ключевых компонентов (этапы 4–6), тестирование (этап 7) и внедрение (этап 8). Этап 1 (инструктаж) и этап 10 (получение отзыва) в отчёте не описываются содержательно, их выполнение подтверждается отдельно.

В каждом разделе сделан акцент не только на том, что было сделано, но и на почему были приняты те или иные решения, с какими трудностями я столкнулся и как их преодолевал. Результаты этапов (код, схемы, тесты, конфигурации, скриншоты логов) вынесены в соответствующие приложения.

2 ФОРМИРОВАНИЕ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

2.1 Получение постановки задачи

Моя работа над механизмом анонимизации началась с задачи в Jira — MWSDEVP-5799 («Поддержать CloudSensitive в access log», рисунок А.3). На очередном разборе бэклога (еженедельная встреча команды в понедельник 13:00) тимлид спросил, не хочу ли я заняться этой задачей. Я ответил согласием — задача показалась мне интересной и полезной для платформы, к тому же я уже был знаком с модулем access-логирования по предыдущим доработкам буферизации в нем.

После этого мы с тимлидом провели короткий созвон (минут на 15–20), где устно обговорили основные требования:

1. Нужно маскировать значения полей, обёрнутых в CloudSensitive<T>, заменяя их на **** в access-логах.
2. Обрабатываем только JSON-тела запросов и ответов.
3. Механизм должен работать и для входящих запросов (сервер), и для исходящих (клиент).
4. Влияние на производительность должно быть минимальным.

Я предложил реализовать маскировку через кастомные Jackson-десериализаторы и сериализаторы. Тимлид одобрил. Таким образом, формального документа с требованиями на старте не было — только устные договоренности и краткая запись в Jira.

2.2 Формулирование и согласование требований

Для отчёта по практике потребовался формальный список требований. Я сформулировал его на основе устных договоренностей с тимлидом и своего понимания задачи, утвердив его с руководителем практики. Требования были разделены на функциональные (что именно должен делать механизм) и нефункциональные (как он должен это делать).

Функциональные требования охватили:

- Поддержку маскировки полей CloudSensitive<T>.
- Два режима работы (RAW — логирование как есть, AUTO — с маскировкой).
- Маскировку по ключам JSON (дополнительный слой защиты).
- Поддержку вложенных структур, коллекций, мап, обёрток (CloudOptional, Mono, Flux, ResponseEntity).
- Обработку ошибок парсинга.

Нефункциональные требования охватили:

- Производительность (минимальный оверхед, zero-overhead для безопасных тел).
- Обратную совместимость.
- Надёжность (устойчивость к ошибкам, отсутствие падений сервиса).
- Конфигурируемость через application.yml.
- Тестируемость и документированность.

Список требований был согласован с руководителем практики и утверждён. Итоговый список требований представлен в Приложении А.

2.3 Заполнение и утверждение индивидуального задания

После согласования требований я заполнил индивидуальное задание в личном кабинете практики на портале my.itmo.ru. Для этого я внимательно прочитал материалы для самопроверки из группы в Telegram «Практика 4 курс». Я пересмотрел свое задание, убедившись, что обязательные пункты перенесены правильно, даты написаны корректно, а в каждом пункте присутствуют критерии приёмки и результат для отчёта. В качестве основы использовал шаблон этапов первых и последних этапов, заполнив основную часть ожидаемым планом работ, требованиями и тем, что должно было оказаться в этом отчете, дополнив задание конкретными формулировками и отразив в нем требования как часть процесса. Задание было направлено на утверждение руководителю практики со стороны университета и впоследствии одобрено, позже также было одобрено руководителем практики со стороны компании.

2.4 Рефлексия

На этом этапе я научился важному навыку: трансформировать устные договорённости и технические обсуждения в формальный документ с требованиями. В ходе работы над задачей я фиксировал все уточнения, которые мы обсуждали с тимлидом, и постепенно дополнял ими исходный список. Это позволило мне, во-первых, лучше структурировать собственное понимание задачи, а во-вторых, создать документ, который в дальнейшем помогал при проектировании и тестировании – к нему можно было возвращаться и проверять, все ли аспекты учтены. Также я убедился, что требования – это не статичный документ, который пишется один раз и никогда не меняется. По ходу работы, когда появляется новое понимание задачи или выявляются технические ограничения, требования должны уточняться. В моём случае это произошло позже, на этапе проектирования и реализации. Эти уточнения были зафиксированы и отражены в итоговом списке требований (Приложение А).

3 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И ТРЕБОВАНИЙ К СИСТЕМЕ

3.1 Постановка задачи на проектирование

После согласования индивидуального задания (этап 2) я приступил к проектированию механизма маскировки. Исходные требования были представлены в Приложении А:

На момент начала проектирования в платформе уже существовала инфраструктура access-логирования, но тела запросов и ответов логировались «как есть» (или не логировались вовсе, так как по умолчанию была выбрана стратегия `WITHOUT_BODY`). Моя задача состояла в том, чтобы добавить слой автоматической анонимизации, не ломая существующие сервисы.

3.2 Первая итерация: маскировка через Jackson-десериализацию (MWSDEVP-5799)

3.2.1 Выбранный подход

Когда я думал, как выполнить задачу (в Jira под номером MWSDEVP-5799, рисунок в Приложении А.3) подменить значения `CloudSensitive` на `****`, первое, что пришло в голову – использовать Jackson. В платформе он и так используется везде для работы с JSON, а у него есть механизм кастомных сериализаторов/десериализаторов. Логика была простая: если Jackson при чтении JSON всегда будет превращать `CloudSensitive` в `*****`, то дальше можно просто десериализовать тело в DTO и сериализовать обратно – маскировка произойдёт автоматически.

Мне казалось, что это элегантно: не нужно вручную обходить JSON-дерево, не нужно думать о вложенности – Jackson сам разберётся с полями. К тому же, я уже был знаком с модулем access-логирования по предыдущим доработкам (буферизация тел), и интеграция нового механизма в существующую архитектуру выглядела прямолинейно. Тимлид одобрил подход, и я приступил к реализации.

3.2.2 Выполнение

Я создал `AutoBodySerializationStrategy`, внутри которой сконфигурировал отдельный `ObjectMapper` с кастомными `JsonSerializer` и `JsonDeserializer` для `CloudSensitive`. Они всегда писали и читали `*****`, независимо от реального значения.

Логика работы стратегии AUTO была такой:

- Если тело не JSON → [BINARY_CONTENT].
- Если тип неизвестен → логируем как есть.
- Иначе: десериализуем JSON в объект через кастомный ObjectMapper (где CloudSensitive уже подменены), затем сериализуем обратно. Всё, что не CloudSensitive, остаётся нетронутым.

Параллельно в задаче MWSDEVP-4267 (в Приложение Б рисунок 1) мы решили включить логирование тел по умолчанию. Раньше стратегия была WITHOUT_BODY (тела не логировались), и это скрывало проблему. Теперь, с появлением маскировки, можно было безопасно включить WITH_BODY по умолчанию.

Архитектурная схема первой реализации (Рисунок Б.5) показывает поток данных от HTTP-запроса до access-лога. Ключевой элемент схемы – AutoBodySerializationStrategy, внутри которой находится кастомный ObjectMapper с подменёнными сериализаторами/десериализаторами для CloudSensitive. Именно на этом этапе происходит полная десериализация тела в DTO, которая впоследствии привела к проблемам с контекстно-зависимыми типами. Схема наглядно иллюстрирует, почему подход оказался хрупким: он требует, чтобы все DTO успешно десериализовались на раннем этапе обработки запроса, что в реальной платформе сложно гарантировать.

3.2.3 Первичные результаты

На тестовых DTO (простые объекты с парой полей) всё работало отлично. Я написал тесты, они проходили. Код прошёл ревью, изменения влили в платформу. Казалось, задача закрыта.

3.2.4 Что пошло не так (и почему я этого не ожидал)

Примерно через неделю после влития в одном из сервисов начали массово появляться ворнинги в логах. Ошибка была примерно такая:

```
Cannot construct instance of '...ResourceId' (no Creators, like default constructor, exist)
```

Я начал разбираться. Оказалось, что в DTO этого сервиса используется тип ResourceId, для которого в платформе написан кастомный Jackson-сериализатор/десериализатор. Проблема была в том, что этот сериализатор полагался на контекстные данные, которые появляются только после того, как запрос прошёл определённые этапы обработки в WebFlux. А access-логирование срабатывает раньше – на входе в WebFilter (который стоит

одним из первых в цепочке) или на выходе из `ExchangeFilterFunction`. В этот момент контекстные данные, нужные для десериализации `ResourceId`, ещё не загружены. Jackson, встречая такое поле, не мог его правильно десериализовать и падал с ошибкой.

Раньше, когда логирование тел было выключено (`WITHOUT_BODY`), `ObjectMapper` просто не вызывался на этом этапе, и проблема не проявлялась. Но после того как мы включили `WITH_BODY` по умолчанию, `secureMapper.readValue()` стал вызываться для каждого запроса на раннем этапе обработки, наткнулся на `ResourceId` и плодил ворнинги, при этом не логируя тело.

Почему я не подумал об этом заранее? Потому что я тестировал на своих простых DTO, где все типы были «честными» data-классами с конструкторами. Я не учёл, что в реальной платформе DTO могут содержать типы с контекстно-зависимой десериализацией, а access-логи вызываются на этапе, когда этого контекста ещё нет.

Что это значило: Мы пытались использовать десериализацию для побочной задачи (маскировки), но она требовала полного соответствия DTO и наличия всех контекстных данных в момент вызова. В реальном production это оказалось невозможно гарантировать.

3.2.5 Что я из этого понял

Этот опыт стал для меня важным уроком. Я осознал несколько вещей.

Во-первых, десериализация – неподходящий инструмент для задач, не связанных напрямую с бизнес-логикой. Она требует полного соответствия между JSON и DTO, включая наличие всех конструкторов, доступность контекстных данных и отсутствие «сложных» типов. В большой платформе с десятками сервисов, многолетней историей и множеством устаревших API это сложно гарантировать.

Во-вторых, порядок вызова имеет значение. Access-логи срабатывают на ранних этапах обработки запроса (в `WebFilter` с высшим приоритетом). В этот момент многие контекстные данные ещё не загружены. Если DTO полагается на такие данные при десериализации – будет ошибка. Раньше я об этом не задумывался.

В-третьих, тестировать платформенный код нужно не только на «чистых» DTO. Мои тесты использовали простые data-классы, которые идеально десериализовались. Но в реальности DTO могут быть любыми: абстрактные типы, интерфейсы, классы с контекстно-зависимыми сериализаторами. Нужно либо тестировать на реальных DTO из сервисов-потребителей, либо с

самого начала проектировать решение, которое не зависит от внутреннего устройства DTO.

В-четвёртых, «работает на моём тесте» и «работает в production» — это две огромные разницы. Я был уверен, что решил задачу, но реальность оказалась другой. Это научило меня быть более критичным к своим решениям и думать о граничных случаях.

3.2.6 Временное решение

Мы не могли оставить логирование тел включённым, потому что ворнинги засоряли логи и создавали ложное впечатление о нестабильности платформы. Мы приняли временное решение:

1. Откатить стратегию по умолчанию обратно на WITHOUT_BODY. Тела запросов и ответов снова перестали логироваться. Это остановило поток ворнингов.
2. Удалить режим AUTO. Возможность его включения была удалена, чтобы через конфиг нельзя было включить недоделанную реализацию. Маскировка CloudSensitive больше не применялась. Режим RAW (логирование как есть) остался для тех, кто готов рисковать и стал дефолтным.
3. Зафиксировать проблему в баг-трекере. Задача MWSDEVP-6038 (в Приложении Б рисунок 2) была создана для отслеживания проблемы и поиска решения.

Это было болезненное, но необходимое решение. Я чувствовал разочарование — столько работы, а в итоге откат. Но руководитель объяснил, что так бывает: иногда первая реализация выявляет ограничения, которые нельзя было предвидеть заранее. Главное — не заикливаться на неудаче, а извлечь уроки и спроектировать решение заново, уже с учётом выявленных проблем.

Я согласился и начал проектировать вторую итерацию — с нуля, но уже с пониманием всех ограничений.

Результаты первой итерации (диффы, скриншоты задач Jira, архитектурная схема первой итерации) представлены в Приложении Б.

3.3 Вторая итерация: прямой обход JSON-дерева и двухуровневая маскировка (MWSDEVP-6105)

3.3.1 Анализ проблемы и поиск нового подхода

После отката первой реализации я чувствовал себя не очень комфортно. С одной стороны, было обидно – столько работы, код прошёл ревью, тесты зелёные, а в production всё рассыпалось. С другой стороны, я понимал, что просто так проблему не решить, нужно менять подход кардинально.

Я сел разбираться, что именно пошло не так. Основная проблема была в том, что мы пытались десериализовать JSON и маппить в реальный DTO. А для этого нужно, чтобы у DTO были конструкторы, чтобы все типы десериализовались стандартно, чтобы контекстные данные уже были загружены. Но для маскировки нам не нужен весь объект – нам нужно только знать, какие поля в JSON соответствуют типам CloudSensitive в DTO, чтобы заменить их значения на ****.

На одном из созвонов я поделился своими мыслями с тимлидом. Мы обсуждали проблему, и пришли к тому что если не десериализовать JSON, можно просто сопоставить DTO и JSON как два дерева? У нас есть DTO – это описание структуры, и JSON – это реальные данные. Пройдя по ним параллельно и везде, где в DTO стоит CloudSensitive, заменим значение в JSON. Хотя и придётся прям попрограммировать, но это будет интересно и, мы решили, что это правильный путь, и этому помогает то, что наши модели для сервисов, как и клиенты, генерировались из написанных командами расширенных OpenAPI-спецификаций, то есть то, какой клиент и какие модели мы легко понимаем и обычно это используется единообразно, то есть и из клиента мы информацию о типах сможем вытащить (здесь мы также договорились о том, что проблемы команд, полагающихся на свои кастомные клиенты или модели нас будут волновать меньше).

Чем больше я обдумывал идею, тем больше она мне нравилась. В ней было что-то классное: мы отделяем анализ структуры (что должно маскироваться) от обработки данных (где именно в JSON это находится). Это позволяло не сопоставлять содержимое неинтересных нам полей, не производить полноценной десериализации и давало большой контроль над внутренним процессом. Мы просто смотрели на поля, обернутые в CloudSensitive, и искали их в JSON.

3.3.2 Первый прототип

Я решил не откладывать и попробовать реализовать эту идею. Начал с малого: парсил JSON в JsonNode, брал простой DTO с одним полем

CloudSensitive, и пытался рекурсивно обойти оба дерева. Первая версия была простой, но уже на этом этапе я увидел, что идея работает. Постепенно я усложнял: добавил поддержку вложенных объектов, потом коллекций, потом мап. Самым неприятным оказались generic-обёртки. Пришлось научиться «заглядывать» внутрь обёртки и доставать реальный тип.

3.3.3 Ключевые архитектурные решения второй итерации

После того как прототип подтвердил работоспособность идеи, я начал проектировать полноценную архитектуру. В итоге выделил несколько ключевых компонентов:

Анализатор типов. Его задача – по классу DTO (или ResolvableType) определить:

- Какие поля помечены как CloudSensitive.
- Какие поля являются вложенными объектами (и их тоже нужно анализировать рекурсивно).
- Какие поля являются коллекциями или мапами (и какие типы элементов внутри).
- Какие поля обёрнуты в CloudOptional, Mono, Flux, ResponseEntity (и что скрывается под обёрткой).
- Все результаты анализа кэшируются в ConcurrentHashMap, чтобы не анализировать один и тот же класс несколько раз.

Маскировщик по типам. Получает на вход JSON-дерево и информацию от анализатора, после чего рекурсивно обходит JSON и заменяет значения в узлах, соответствующих CloudSensitive. При этом он защищён от циклических ссылок – если DTO ссылается сам на себя, обход не уходит в бесконечность.

Маскировщик по ключам. Дополнительный слой защиты, не зависящий от DTO. Заменяет значения всех JSON-полей, чьи имена совпадают (или содержат подстроку) с настраиваемым списком (например, password, token). Этот компонент появился позже, в задаче MWSDEVP-6818, но архитектурно он вписался в ту же схему.

Композитный оркестратор. Этот компонент:

- Проверяет, нужна ли маскировка по типам (есть ли в DTO CloudSensitive поля).
- Проверяет, нужна ли маскировка по ключам (задан ли список чувствительных ключей).
- Если не нужна ни та, ни другая – возвращает исходное тело без каких-либо действий (даже парсинг JSON не выполняется).

- Если нужна – парсит JSON один раз, прогоняет через оба маскировщика (следующий по дереву предыдущего) и сериализует обратно.

Интеграционный слой. Получает строку тела, тип DTO и Content-Type. Проверяет, является ли тело JSON. Если нет — возвращает. Вызывает композитный оркестратор. Обработывает ошибки: при проблемах с парсингом возвращает или при неподдерживаемых типах возвращает маркер ошибки вместо тела запроса.

3.3.4 Поток данных в новой архитектуре

Схематично работа выглядит так:

1. Запрос проходит через `AccessLogWebFilter / ExchangeFilterFunction`, тело буферизируется.
2. `LoggerSink` вызывает `AutoBodySerializationStrategy.serializeBody()`.
3. `AutoBodySerializationStrategy` проверяет Content-Type, тип DTO и вызывает `CompositeSensitiveMasker.mask()`.
4. `CompositeSensitiveMasker`:
5. Смотрит в кэш, есть ли в DTO `CloudSensitive` поля.
6. Смотрит, задан ли список чувствительных ключей.
7. Если оба условия ложны — возвращает исходное тело (даже не парсит JSON).
8. Если нужно — парсит JSON в `JsonNode`, прогоняет через `CloudSensitiveJsonMasker` (если нужна маскировка по типам), затем через `SensitiveKeyMasker` (если нужна маскировка по ключам), и сериализует обратно.
9. Замаскированное тело возвращается в `LoggerSink` и записывается в access-лог.

3.3.5 Что дала новая архитектура

По сравнению с первой итерацией, новая архитектура дала несколько важных преимуществ:

1. Независимость от конструкторов и контекста. Мы больше не десериализуем JSON в объекты, а значит, нам не нужны ни конструкторы по умолчанию, ни кастомные десериализаторы, ни контекстные данные. Проблема с `ResourceId` и подобными типами ушла сама собой.
2. Zero-overhead для безопасных тел. Благодаря отдельному анализатору мы заранее знаем, есть ли в DTO чувствительные поля. Если нет — мы даже не парсим JSON, а просто возвращаем исходную строку. Для

большинства запросов в платформе это даёт практически нулевые накладные расходы.

3. Кэширование метаданных. Анализ структуры DTO выполняется один раз на класс, а не на каждый запрос. Это особенно важно для сервисов с высокой нагрузкой.
4. Устойчивость к ошибкам. Если JSON невалидный, мы не падаем, а возвращаем маркер ошибки. Если тип не поддерживается – аналогично. Никаких исключений в production.
5. Двухуровневая защита. Маскировка по ключам (добавленная позже) позволяет защитить API, которые ещё не переведены на CloudSensitive. Это закрыло проблему команд, у которых были сложности с обратной совместимостью при внедрении обертки.

3.3.6 Что осталось за рамками проектирования

В рамках проектирования мы определили общую архитектуру и взаимодействие компонентов. Детальная реализация каждого компонента (анализатор типов, маскировщики, интеграция с существующей инфраструктурой) вынесена в следующие этапы (4–6). Также в этапе 7 запланировано многоуровневое тестирование, включая нагрузочные тесты на больших объёмах данных.

Архитектурная схема второй итерации представлена в Приложении Б (рисунок Б.6).

3.4 Итоги проектирования

В результате эволюционного проектирования (от первой, неудачной итерации к промышленной архитектуре) я сформулировал несколько ключевых решений, которые легли в основу финальной реализации.

1. Отказ от десериализации. Мы больше не пытаемся превращать JSON в объекты DTO. Вместо этого работаем напрямую с JSON-деревом (JsonNode). Это избавляет от зависимости от конструкторов, контекстных сериализаторов и порядка инициализации и даёт больший контроль над внутренней структурой и позволяет сильнее оптимизировать процесс маскировки.
2. Разделение анализа и маскировки.
 - а. Анализатор типов отвечает только за одно: по классу DTO понять, какие поля нужно маскировать. Он ничего не знает о JSON и логировании.

- b. Маскировщики отвечают только за обход JSON-дерева и замену значений. Они не знают, как устроены DTO.
 - c. Оркестратор связывает их вместе и решает, нужна ли маскировка вообще.
- 3. Кэширование метаданных. Анализ структуры DTO выполняется один раз на класс, результат сохраняется в кэш. Это критично для производительности при высокой нагрузке.
- 4. Zero-overhead для безопасных тел. Если в DTO нет чувствительных полей и не заданы чувствительные ключи, мы даже не парсим JSON – возвращаем исходное тело. Для большинства запросов в платформе это даёт практически нулевые накладные расходы.
- 5. Двухуровневая защита.
 - a. Первый уровень – по типам (обёртка CloudSensitive). Это основной механизм для команд, которые уже используют аннотацию.
 - b. Второй уровень – по ключам. Это безусловная защита для API, которые ещё не переведены на CloudSensitive.
- 6. Устойчивость к ошибкам. При любых проблемах (битый JSON, неожиданный тип, циклическая ссылка) механизм не падает, а возвращает маркер ошибки и продолжает работу.

Эти архитектурные решения определили дальнейшую реализацию. В этапе 4 я разработал анализатор типовой структуры (CloudSensitiveTypeAnalyzer) с кэшированием и поддержкой сложных типов. В этапах 5-6 – механизмы маскирования с рекурсивным обходом, защитой от циклов и обработкой ошибок. В этапе 7 – многоуровневое тестирование, включая нагрузочные тесты. В этапе 8 – интеграцию в платформу. В результате проектирования был выработан план реализации компонентов, схема была согласована с руководителем, а все ожидаемые типы и угловые случаи покрыты.

Глядя на этот раздел сейчас, я понимаю, что в нём не хватает одной важной вещи – описания того, как мы передавали информацию о типах между компонентами. В первой итерации мы просто использовали bodyClass, который доставали из атрибутов запроса. Во второй итерации этого стало недостаточно – нам нужен был ResolvableType, чтобы сохранить информацию о generic-параметрах (например, List<UserDto>). Этот момент я упустил в самом начале проектирования и дособидали уже в процессе реализации. В следующий раз, наверное, стоит сразу фиксировать такие «стыки» между компонентами – они часто оказываются самыми болезненными.

4 РЕАЛИЗАЦИЯ АНАЛИЗАТОРА ТИПОВОЙ СТРУКТУРЫ

4.1 Постановка задачи и первые мысли

После того как архитектура была утверждена, я приступил к реализации анализатора типовой структуры. Задача звучала просто: «написать компонент, который по классу DTO определяет, какие поля нужно маскировать». Но чем больше я думал, тем больше понимал, что «просто» здесь ничего не будет.

Я сел и начал мысленно перебирать, с чем мне предстоит столкнуться. В DTO бывают не только простые поля типа String. Бывают вложенные объекты. Бывают коллекции – списки, сеты, массивы. Бывают мапы, где чувствительными могут быть и ключи, и значения. Бывают обёртки – CloudOptional, Mono, Flux, ResponseEntity. А иногда всё это вместе, вложенное друг в друга, с nullable-значениями и рекурсивными ссылками.

Я понял, что если не подойти к задаче системно, я утону в частных случаях. Нужно было спроектировать алгоритм, который умеет заглядывать внутрь структур и не боится сложности.

4.2 Как я подошёл к решению

Я решил действовать по принципу «разделяй и властвуй». Анализатор должен был уметь отвечать на два вопроса:

1. «Есть ли в этом типе вообще какие-нибудь CloudSensitive поля?» – это нужно для минимального overhead. Если нет – мы даже не начинаем сложный анализ.
2. «А теперь расскажи подробно: какие именно поля, где они находятся, как туда добраться?» – это уже для самой маскировки.

Первый вопрос требовал быстрого ответа. Второй – детального разбора.

4.3 Проблема №1: как получить информацию о типах

Когда я начал писать код, первая же проблема ударила меня по голове. Я написал простой тест: анализирую класс с полем List<CloudSensitive<String>>. В рантайме я вижу только List, но не знаю, что внутри. Kotlin reflection в некоторых случаях возвращает KType, но только его возможностей мало. Type erasure в Java/Kotlin стирает generic-информацию, и её не всегда можно восстановить.

После этого я начал гуглить, перерыл документацию, и после ресерча нашёл ResolvableType из Spring. Оказалось, что благодаря мощным возможностям Spring мы умеем сохранять полную информацию о типе, если правильно её передать. В платформе при вызове методов через

HandlerMethod мы можем получить ResolvableType для параметров и возвращаемых значений. Для клиентской части – передавать ResolvableType через атрибуты запроса, взяв тип из compile-time определяемого wildcard.

Но тут возникла новая проблема: а что, если даже с ResolvableType определить внутренности generic-типа невозможно, банально из-за того, что был передан raw-тип? Я решил, что в таких случаях мы не можем гарантировать маскировку, и просто возвращаем предупреждение с маркером о том, что тип не поддерживается. Лучше скрыть неопасное тело запроса, чем пропустить секрет.

4.4 Проблема №2: обёртки

Следующая проблема – обёртки. В платформе есть CloudOptional<T>, Mono<T>, Flux<T>, ResponseEntity<T>.

Я думал: «Может, просто взять первый generic-параметр и всё?» Но не всё так просто. Во-первых, обёрток может быть несколько: CloudOptional<CloudSensitive<String>>. Во-вторых, обёртки могут быть вложенными в коллекции: Map<String, CloudOptional<UserRequest>>.

Я реализовал цикл, который «снимает» обёртки, пока не встретится не-обёртка. Для каждого типа я проверяю, является ли он обёрткой (список wrapperTypes я захардкодил, потому что других обёрток в платформе нет и не предвидится). Если да – достаю определенный generic-параметр и продолжаю.

Было страшно, что это замедлит анализ. Для этого я написал тест производительности. Оказалось, что разница незаметна, что я принял к сведению и продолжил реализацию.

4.5 Проблема №3: рекурсивные ссылки

Когда я начал тестировать на реальных DTO из сервисов платформы, один из них содержал ссылку на себя. Мой анализатор при обходе поля children снова вызывал анализ Category, тот снова находил поле children, и так по кругу. Стек вызовов рос, и я понял, что если не остановиться, можно получить StackOverflowError.

Я задумался: «А как вообще определить, что мы уже были в этом классе?» Решение оказалось простым: передавать visited: MutableSet<KClass<*>> при рекурсивных вызовах. Перед тем как анализировать класс, я проверяю, есть ли он уже в этом множестве. Если есть – значит, мы уже анализируем его выше по стеку, и нужно остановиться. Возвращаем пустые метаданные для

этого поля (потому что если мы уже где-то выше встретили этот класс, то все его CloudSensitive поля уже были обработаны).

Я понял, что рекурсия – это красиво, но опасно. Всегда нужно думать о терминеции. В графах (а структура типов DTO – это граф) запросто могут быть циклы. visited set – это стандартный паттерн, но про него можно легко забыть.

4.6 Проблема №4: производительность и кэширование

Одним из важных требований было сделать анализатор мало влияющим на производительность. Я понимал, что если анализ типов будет выполняться для каждого запроса, то даже при 1000 запросов в секунду мы получим 1000 анализов одних и тех же классов. Это неприемлемо.

Я начал думать: «Как это обычно решается?» Классы в JVM загружаются один раз, их структура не меняется, как правило, в рантайме. Значит, можно проанализировать класс один раз и запомнить результат.

Я выбрал ConcurrentHashMap. Почему не HashMap? Потому что анализатор будет вызываться из разных потоков. ConcurrentHashMap позволяет безопасно читать и писать из разных потоков без блокировок.

После этих мыслей я решил организовать кэширование (а точнее один из слоёв кэширования, но об этом позже) следующим образом:

```
class CloudSensitiveTypeAnalyzer {
    private val metadataCache = ConcurrentHashMap<KClass<*>,
ClassMetadata>()

    fun extractMetadata(clazz: KClass<*>): ClassMetadata {
        return metadataCache.getOrPut(clazz) {
            analyzeClassMetadata(clazz) // тяжёлая операция
        }
    }
}
```

Метод getOrPut — это атомарная операция. Если ключа нет – он вызывает функцию, которая его вычисляет, и кладёт результат в кэш. Если есть – возвращает из кэша. Мне показалось это простым и элегантным решением, и оно решало проблему.

4.7 Как я собирал метаданные

Теперь о самом главном — как именно я собирал метаданные. Я создал data-класс `ClassMetadata`, который хранит всё, что понадобится маскировщику:

```
data class ClassMetadata(  
    val cloudSensitiveFields: Set<String> = emptySet(),           //  
    имена полей для маскировки  
    val nestedObjectFields: Map<String, KType> = emptyMap(),     //  
    вложенные объекты  
    val collectionFields: Map<String, KType> = emptyMap(),       //  
    коллекции с типами элементов  
    val mapFields: Map<String, KType> = emptyMap(),              //  
    мапы с типами значений  
    val mapsWithSensitiveKeys: Set<String> = emptySet()          //  
    мапы, где ключ — CloudSensitive  
)
```

Процесс анализа одного класса (метод `analyzeClassMetadata`):

1. Получаю все `memberProperties` класса через Kotlin reflection: `clazz.memberProperties`. Это список всех свойств (полей с геттерами).
2. Для каждого свойства:
 - a. Определяю имя поля. Обычно это `property.name`. Но если поле помечено аннотацией `@JsonProperty` с кастомным именем — использую его. Это важно, потому что в JSON имя может отличаться от имени в коде.
 - b. Получаю тип поля: `property.returnType` (это `KType`).
 - c. Снимаю обёртки: пока тип является обёрткой (`CloudOptional`, `Mono`, `Flux`, `ResponseEntity`) — достаю первый generic-параметр и продолжаю. Это нужно, чтобы «заглянуть» внутрь и добраться до реального типа.
 - d. Определяю, что это за тип:
 - i. Если после снятия обёрток получился `CloudSensitive` — добавляю имя поля в `cloudSensitiveFields`.
 - ii. Если получилась коллекция (`List`, `Set`, `Array`, `Flux`) — извлекаю тип элемента (первый generic-параметр), добавляю в `collectionFields`.
 - iii. Если получилась мапа (`Map`) — извлекаю типы ключа и значения. Если ключ — `CloudSensitive` — добавляю имя поля в `mapsWithSensitiveKeys`. Если значение требует анализа — добавляю в `mapFields`.

- iv. Если получился обычный класс (не примитив, не строка, не enum, не обёртка) — добавляю в `nestedObjectFields` для рекурсивного анализа.
- v. Всё остальное (примитивы, строки, enum'ы) — игнорирую, потому что они не могут содержать `CloudSensitive`.

3. Возвращаю `ClassMetadata`.

Было страшно, что этот код будет медленным из-за reflection. Но, во-первых, reflection вызывается только один раз на класс (благодаря кэшу). Во-вторых, в современных JVM reflection не такой уж и медленный.

Я понял, что Reflection — это мощный инструмент, но его нужно использовать с умом.

4.8 Проблема №5: raw-типы

Я понял, что в одном из угловых случаев мы можем встретить объявления вида `List` без указания типа элементов (raw type). Например, `val data: List`. Для такого типа невозможно определить, что находится внутри. Kotlin reflection не может достать generic-параметры.

Я задумался: «Что делать в таких случаях?» Вариантов было несколько:

1. Игнорировать и надеяться, что там нет `CloudSensitive` — опасно.
2. Выбросить исключение и остановить обработку — это плохо, сервис не должен падать из-за логирования.
3. Вернуть маркер, что тип не поддерживается, и логировать предупреждение.

Я выбрал третье. В анализаторе я добавил проверку: если тип — raw и при этом является коллекцией или mapой, я выбрасываю исключени, выше я ловлю его и, в зависимости от настройки `suppressUnsupportedTypes`, либо возвращаю `[UNSUPPORTED_TYPE]`, либо логирую предупреждение и возвращаю тело как есть.

Я понял, что гибкие настройки позволяют командам выбирать поведение, которое им подходит.

4.9 Тестирование

Я написал несколько десятков юнит-тестов. Сначала я думал, что хватит пяти-шести, но каждый новый кейс рождал новый тест. Вот что я покрыл, исходя из требований:

- Простой случай: DTO с одним полем `CloudSensitive<String>`. Проверяю, что поле попало в `cloudSensitiveFields`.
- Вложенный объект: DTO, у которого есть поле-объект, а у того — `CloudSensitive`. Проверяю, что анализатор заглянул внутрь.
- Коллекция: `List<CloudSensitive<String>>`. Проверяю, что тип элемента попал в `collectionFields`.
- Мапа: `Map<String, CloudSensitive<Int>>`. Проверяю, что тип значения попал в `mapFields`.
- Мапа с чувствительными ключами: `Map<CloudSensitive<String>, String>`. Проверяю, что поле попало в `mapsWithSensitiveKeys`.
- Обёртки: `CloudOptional<CloudSensitive<String>>`. Проверяю, что анализатор «снял» обёртку и нашёл `CloudSensitive`.
- Вложенные обёртки: `CloudOptional<Mono<CloudSensitive<String>>>`. Проверяю, что все обёртки снимаются.
- Nullable: `CloudSensitive<String>?`. Проверяю, что поле обрабатывается корректно (не падает).
- Рекурсивные ссылки: DTO, который ссылается сам на себя. Проверяю, что анализатор не уходит в бесконечность.
- Raw-типы: `List` без `generic`. Проверяю, что выбрасывается `UnsupportedTypeForMaskingException`.
- Пустые обёртки: `CloudOptional.empty()`. Проверяю, что анализатор не пытается анализировать то, чего нет.

Я понял, что тесты — это не для галочки. Я писал тесты не потому, что «так надо», а потому что без них я не мог быть уверен, что анализатор работает правильно. Каждый новый кейс рождал новый тест, и это нормально.

4.10 Результаты этапа

Результатом этапа стали:

1. Исходный код `CloudSensitiveTypeAnalyzer` (представлен в Приложении В, листинг В.1).
2. Набор юнит-тестов, демонстрирующих работу анализатора на различных типах данных (Приложение В, листинг В.2).
3. Отчёт о покрытии тестами (Приложение В, рисунок В.3).

5 РЕАЛИЗАЦИЯ БАЗОВОГО МЕХАНИЗМА МАСКИРОВАНИЯ JSON-СТРУКТУР

5.1 Постановка задачи

После того как анализатор типов был готов, я взялся за маскировщик. Задача: по JSON-строке и информации от анализатора (какие поля где находятся) заменить значения чувствительных полей на ****. Я решил делать рекурсивный обход JSON-дерева (JsonNode из Jackson). Для каждого узла смотрю, какой тип DTO ему соответствует (по пути в дереве), и если тип – CloudSensitive, заменяю значение.

5.2 Простой обход объекта

Я начал с простого. Написал метод, который берёт ObjectNode (JSON-объект), метаданные для его класса и заменяет все поля из cloudSensitiveFields на ****. Это работало для плоских объектов. Но как только появилась вложенность, я понял, что нужно идти глубже. Кроме того, в первой версии я просто создавал копию узла в самом начале. Это работало, но я быстро заметил проблему: даже если в объекте нет ни одного CloudSensitive поля, я всё равно создавал полную копию. Для маленьких JSON это незаметно, но для больших (десятки мегабайт) – дорого.

5.3 Проблема производительности и Copy-on-Write

Я задумался: «А зачем копировать весь объект, если мы меняем всего одно поле?» В идеале нужно копировать только то, что реально изменилось. Остальное – переиспользовать из исходного узла.

Так я пришёл к идее Copy-on-Write (копирование при записи). Суть простая:

- Сначала работаю с исходным узлом, ничего не копирую.
- Если потребовалось что-то изменить — создаю копию в этот момент и дальше работаю с ней.
- Все последующие изменения делаю уже в копии.

Как это работает на практике: приходит JSON с 1000 полей, из которых только одно — CloudSensitive. Копия создаётся ровно один раз, когда мы доходим до этого поля. Остальные 999 полей остаются в исходном узле. Если в JSON вообще нет CloudSensitive полей, копия не создаётся никогда. Мы просто возвращаем исходный узел (а в CompositeSensitiveMasker мы даже до этого не доходим, потому что проверяем mayContainSensitiveFields на уровне типа).

Первая версия была без CoW. Я просто копировал узел в начале метода. Потом я писал тесты производительности для отчёта (этап 7) и увидел, что на больших JSON время обработки заметно растёт даже для безопасных тел. Начал разбираться, где узкое место.

Оказалось, что `deepCopy()` — дорогая операция. Она рекурсивно обходит всё дерево и копирует каждый узел. Для JSON на 10 МБ это заметно. Я подумал: «А можно ли вообще без копирования?» Нет, нельзя, потому что Jackson возвращает иммутабельные узлы (нельзя просто взять и изменить поле). Но можно создать копию только тогда, когда это действительно нужно. Я переписал все методы маскировщика (для объектов, коллекций, map) с использованием ленивого копирования. Получилось не очень красиво (много проверок `if (result == null)`), но зато эффективно.

Я понял, что оптимизации имеют смысл, только если они решают реальную проблему. Я не стал бы заморачиваться с CoW, если бы не увидел цифры на тестах производительности.

5.4 Вложенные объекты

Первый реальный тест — DTO с вложенным объектом. Мой простой маскировщик заменил бы только поля первого уровня. Поле `password` внутри `credentials` осталось бы нетронутым.

Я добавил рекурсию. В метаданных класса есть `nestedObjectFields` — список полей, которые являются объектами и требуют дальнейшего обхода. Для каждого такого поля я достаю JSON-узел, рекурсивно вызываю `maskObjectNode` (или `maskJsonNode`, в зависимости от типа) и заменяю узел в родительском объекте.

Тут я понял важную вещь: нельзя просто мутировать исходный узел. Нужно создавать копию только если что-то изменилось. Иначе для больших JSON мы будем копировать всё дерево даже если маскировка не нужна. Поэтому я сделал ленивое копирование: сначала работаю с исходным узлом, а если понадобилось что-то поменять — создаю копию и дальше работаю с ней.

5.5 Коллекции

Следующий тест — DTO с коллекцией. В JSON это массив строк: `["secret1", "secret2", "secret3"]`. Как понять, что каждый элемент массива нужно замаскировать? Решение: в метаданных класса, благодаря тому, что при проектировании мы обратили на такие детали внимание, есть `collectionFields` — список полей, которые являются коллекциями, с указанием типа элемента. Для каждого такого поля я беру JSON-узел (он должен быть

массивом), иду по элементам и для каждого вызываю `maskJsonNode` с типом элемента. Тут был момент, когда я долго не мог понять, почему не работает. Я забыл, что у `CloudSensitive<String>` тип элемента – не `String`, а `CloudSensitive`. И маскировать нужно не строку, а узел, который её содержит. Пришлось переделать логику: не просто заменять значение, а рекурсивно вызывать `maskJsonNode` для каждого элемента.

Листинги методов и примеры работы на тестовых данных представлены в Приложении Г вместе с кодом реализации для этапа 6.

6 РЕАЛИЗАЦИЯ ПОДДЕРЖКИ СЛОЖНЫХ ВЛОЖЕННЫХ СТРУКТУР И ОБЁРТОВ

6.1 Постановка задачи

После того как базовая маскировка (простые объекты, вложенные объекты, коллекции) заработала, я начал добавлять поддержку более сложных сценариев: карты, обёртки, рекурсивные ссылки, nullable, обработка ошибок. Это был этап 6 по индивидуальному заданию.

6.2 Словари

Матрицы оказались сложным местом. Вариантов несколько, и каждый требует отдельной обработки.

Случай 1: значения — `CloudSensitive`

```
data class Request(  
    val metadata: Map<String, CloudSensitive<String>>  
)
```

JSON: `{"key1": "secret1", "key2": "secret2"}`. Нужно замаскировать значения, ключи оставить как есть. Результат: `{"key1": "****", "key2": "****"}`. Я сделал так: прохожу по всем полям карты, для каждого значения вызываю `maskJsonNode` с типом значения. Если тип значения — `CloudSensitive`, он сам заменится на `****`.

Случай 2: ключи — `CloudSensitive`

```
data class Request(  
    val metadata: Map<CloudSensitive<String>, String>  
)
```

JSON: {"secretKey1": "value1", "secretKey2": "value2"}. Тут сложнее: ключи в JSON — строки, и если их просто заменить на *****, то два разных ключа станут одинаковыми. Это проблема. Варианты:

- Оставить как есть — не годится, потому что ключи могут быть чувствительными.
- Заменить на ***** — будут коллизии.
- Заменить на ***** и добавить какой-то суффикс.

Поэтому я выбрал третье: заменяю ключи на *****_N, где N — порядковый номер. JSON превращается в {"*****_0": "value1", "*****_1": "value2"}. Порядок сохраняется, коллизий нет. Реализация: прохожу по полям карты в порядке их появления, собираю новую карту. Для каждого ключа проверяю, является ли он CloudSensitive (по типу из метаданных). Если да — генерирую новый ключ "*****_\${counter++}".

Случай 3: и ключи, и значения — CloudSensitive

Тут нужно маскировать и то, и другое. Ключи — в *****_N, значения — в *****. Всё вместе.

Случай 4: значения — коллекции

JSON: {"group1": ["secret1", "secret2"], "group2": ["secret3"]}. Нужно замаскировать элементы списков внутри карты. Я сделал рекурсивную обработку: сначала обрабатываю карту, для каждого значения вызываю maskJsonNode. А уж maskJsonNode сам разберётся, что значение — это массив, и обработает его элементы.

Я не хотел, чтобы маскировка меняла структуру JSON (порядок может быть важен для некоторых систем). Поэтому собирал новый ObjectNode с теми же ключами (или с *****_N в том же порядке). В Jackson порядок ключей в ObjectNode сохраняется, если использовать LinkedHashMap внутри, но я решил не полагаться на это и явно копировал поля в порядке обхода.

6.3 Рекурсивные ссылки: опять visited set

Когда я начал тестировать на DTO с рекурсивными ссылками (например, дерево категорий), маскировщик снова мог уйти в бесконечность. Я добавил такую же защиту, как в анализаторе: visited для JSON-узлов. Перед тем как обрабатывать узел, проверяю, не обрабатывали ли мы его уже в текущей цепочке вызовов.

Почему я передаю visited параметром, а не делаю глобальным: Потому что один и тот же узел может встречаться в разных местах дерева, и это нормально. Маскировка не должна пропускать такие узлы. А вот если узел

встретился в той же цепочке вызовов (то есть есть цикл) — это проблема, и её нужно прерывать.

6.4 Обёртки

В платформе есть обёртки: `CloudOptional<T>`, `Mono<T>`, `Flux<T>`, `ResponseEntity<T>`. Поле типа `CloudOptional<UserRequest>` — маскировать нужно не саму обёртку, а то, что внутри. Но если обёртка пустая — маскировать нечего. Я реализовал в маскировщике такое же «снятие» обёрток, как в анализаторе. Если тип — обёртка, я достаю внутренний тип и рекурсивно вызываю `maskJsonNode` для того же узла, но уже с внутренним типом.

6.5 Nullable

`Nullable`-поля `val secret: CloudSensitive<String>?` — в JSON может быть `null` или поле может отсутствовать. Я решил, что если поле отсутствует или равно `null` — ничего не делаю. Маскировать нечего. Это было простое бытовое решение, и особых уроков из него я не вынес.

6.6 Обработка ошибок

Я проектировал маскировщик так, чтобы он никогда не падал. Даже если пришёл битый JSON. Даже если тип не соответствует.

В `AutoBodySerializationStrategy` я обернул вызов маскировщика в `try-catch`:

- `UnsupportedTypeForMaskingException` (raw-тип без generic) → `[UNSUPPORTED_TYPE]` или исходное тело (в зависимости от `suppressUnsupportedTypes`).
- `IllegalStateException` (ошибка парсинга JSON) → `[JSON_ERROR]`.
- `Exception` (всё остальное) → `[UNEXPECTED_ERROR]`.

6.7 Тестирование

Согласно требованиям я написал тесты на такие сценарии:

1. Простой объект: одно поле `CloudSensitive` — проверка, что оно замаскировано, а остальные поля нет.
2. Вложенный объект: два уровня, на втором `CloudSensitive` — проверка, что маскировка работает рекурсивно.
3. Коллекция: `List<CloudSensitive<String>>` — все элементы замаскированы.
4. Коллекция объектов: `List<Credentials>` — маскировка внутри каждого объекта.

5. Мапа со значениями-CloudSensitive: значения замаскированы, ключи нет.
6. Мапа с ключами-CloudSensitive: ключи заменены на ****_N, порядок сохранён.
7. Мапа с коллекциями внутри: Map<String, List<CloudSensitive<String>>> — маскировка элементов списков.
8. Обёртка: CloudOptional<CloudSensitive<String>> — маскировка внутри обёртки.
9. Вложенные обёртки: CloudOptional<Mono<CloudSensitive<String>>> — все обёртки снимаются.
- 10.Nullable: поле CloudSensitive<String>? со значением null — нет ошибки.
- 11.Рекурсивная структура: дерево с глубиной 100 — нет стековерфлоу.
- 12.Пустая коллекция: emptyList() — нет ошибки.
- 13.Пустая мапа: emptyMap() — нет ошибки.
- 14.Некорректный JSON: { invalid json } — возвращается [JSON_ERROR], нет падения.
- 15.Несоответствие типов: в DTO CloudSensitive<String>, в JSON число — возвращается [JSON_ERROR].
- 16.Raw-тип: List без generic — возвращается [UNSUPPORTED_TYPE].

6.8 Результаты этапа

1. Исходный код расширенного модуля маскирования (CloudSensitiveJsonMasker с поддержкой мап, обёрток, рекурсии) — Приложение Г, листинг Г.1.
2. Интеграционные тесты на сложных структурах — Приложение Г, листинг Г.2.
3. Скриншоты прохождения тестов — Приложение Г, рисунок Г.3.

7 ТЕСТИРОВАНИЕ МЕХАНИЗМА АНОНИМИЗАЦИИ

7.1 Постановка задачи

По индивидуальному заданию я должен был провести многоуровневое тестирование разработанного механизма:

1. Юнит-тестирование — проверка корректности работы модулей на изолированных сценариях (простые объекты, вложенные структуры, коллекции).
2. Интеграционное тестирование — проверка взаимодействия анализатора и маскировщика на сложных типах (например, `List<Map<CloudOptional<Int?>?, ProfileRequest?>?>`).
3. Сквозное тестирование — на тестовом сервисе с реальными HTTP-запросами.
4. Проверка производительности — оценка влияния механизма на время обработки запросов при различных объёмах данных (до 10 МБ, глубина вложенности до 500 уровней).

Критерии приёмки: прохождение всех тестов, отсутствие утечек конфиденциальной информации, производительность в пределах нормы.

7.2 Юнит-тестирование

Я начал с юнит-тестов на анализатор и маскировщик по отдельности. Для анализатора я написал тесты на все типы структур, которые он должен обрабатывать: простые поля, вложенные объекты, коллекции, мапы, обёртки, nullable, рекурсивные ссылки, raw-типы, как и описано в этапах выше. Каждый тест — это отдельный DTO и проверка, что анализатор правильно определил чувствительные поля. Для маскировщика — аналогично: беру JSON, прогоняю через маскировщик, сравниваю с ожидаемым результатом. Я покрыл сценарии с простыми объектами, вложенными объектами, коллекциями, мапами (включая маскировку ключей), обёртками, nullable-полями, рекурсивными ссылками.

Я понял, что юнит-тестирование — это база тестирования. Без них я бы не мог быть уверен, что отдельные компоненты работают правильно. Но они не гарантируют, что всё вместе будет работать в реальном сервисе.

7.3 Интеграционное тестирование

Другим пунктом задания было интеграционное тестирование на сложных типах. В индивидуальном задании был приведён пример типа: `List<Map<CloudOptional<Int?>?, ProfileRequest?>?>`. Этот тип включал в себя:

- CloudOptional на верхнем уровне (может быть пустым)
- List внутри (может быть null)
- Map внутри списка (может быть null)
- Ключ карты — CloudOptional<Int?>? (может быть null, внутри nullable Int)
- Значение карты — ProfileRequest? (может быть null, а ProfileRequest содержит CloudOptional<UserRequest> с полем password: CloudSensitive<String>)

Я написал тест, который создаёт JSON под эту структуру, прогоняет через маскировщик и проверяет, что поле password замаскировано, а всё остальное осталось как есть. Этот тест показал мне слабые места в обработке обёрток и карт. Без него я бы не заметил проблем.

7.4 Сквозное тестирование

Юнит-тесты и интеграционные тесты — это хорошо, но они не дают гарантии, что механизм правильно интегрируется в реальный сервер. Поэтому я решил провести сквозное тестирование на отдельном тестовом сервисе, который уже существовал в платформе (devp-testService).

7.4.1 Задача MWSDEVP-5985

Я создал задачу MWSDEVP-5985 (Приложение Д, рисунок Д.1) и в её рамках:

Во-первых, описал OpenAPI-спецификацию (Приложение Д, листинг Д.2). В файле openapi.yaml добавил новые endpoint'ы:

- GET /cloudSensitive/{objectId} — получение объекта с чувствительными полями в ответе.
- GET /cloudSensitive (с body) — GET-запрос с телом, содержащим чувствительные поля.
- POST /cloudSensitive — создание объекта с чувствительными полями в запросе и ответе.
- GET /cloudSensitive/list — получение списка объектов с чувствительными полями.

В спецификации использовал расширение x-sensitive: true, чтобы пометить поля, которые должны быть обёрнуты в CloudSensitive в сгенерированном коде.

Сгенерировал код. Из OpenAPI-спецификации автоматически сгенерировались DTO с полями типа CloudSensitive<T>, клиенты и API-интерфейсы.

Реализовал серверную логику. Написал `CloudSensitiveReactiveApiController`, реализующий сгенерированный интерфейс. После этого я написал `CloudSensitiveService` и `IpcCloudSensitiveService`, использующие сгенерированного клиента. Это позволило проверить маскировку и на исходящих запросах. Далее настроил access-логирование. В `application.yaml` тестового сервиса включил access-логи с режимом AUTO. Наконец, написал тесты. В `CloudSensitiveTest.kt` — тесты, отправляющие реальные HTTP-запросы и проверяющие маскировку. Найти изменения в тестовом сервисе можно в Приложении Д, листинг Д.3.

7.4.2 Проверка

Я запускал тестовый сервис локально и отправлял запросы через `curl` (файл `cloud-sensitive.http` с коллекцией запросов). Проверял, что в access-логах чувствительные поля заменены на `****`, а обычные поля не затронуты. Также проверил маскировку на стороне клиента (исходящие запросы). Я проверил, что чувствительные данные не попадают в логи. Запускал сквозные тесты с реальными секретными значениями (пароли, JWT-токены, API-ключи) — в логах они всегда заменены на `****`. Также проверил маскировку по ключам: поле `password` маскируется даже без `CloudSensitive`. Это также отображено в Приложении Д в рисунке Д.4.

7.5 Проверка производительности

Чтобы убедиться, что итоговое решение приемлемо влияет на время обработки запроса, я написал нагрузочные тесты в `CloudSensitiveMaskerPerformanceTest`, измеряющие время маскировки на JSON разного размера и структуры.

Сценарий	Описание	Ожидаемое среднее	Ожидаемый максимум
Маленький payload	простой объект с одним <code>CloudSensitive</code>	2 мс	5 мс
Средний payload (100 элементов)	список из 100 элементов с <code>CloudSensitive</code>	10 мс	20 мс
Большой payload (1000 элементов)	список из 1000 элементов	25 мс	50 мс

Сценарий	Описание	Ожидаемое среднее	Ожидаемый максимум
Очень большой payload (10 000 элементов)	список из 10 000 элементов	100 мс	150 мс
Глубокая вложенность (100 уровней)	дерево глубиной 100	10 мс	20 мс
Очень глубокая вложенность (500 уровней)	дерево глубиной 500	25 мс	50 мс
Широкая мапа (1000 записей)	мапа с 1000 чувствительными значениями	25 мс	50 мс

Результаты:

Сценарий	Среднее	Максимум	Вердикт
Маленький payload	<1 мс	3 мс	Проходит
Средний payload (100 элементов)	2.5 мс	15 мс	Проходит
Большой payload (1000 элементов)	15 мс	35 мс	Проходит
Очень большой payload (10 000 элементов)	20 мс	120 мс	Проходит
Глубокая вложенность (100 уровней)	5 мс	12 мс	Проходит

Сценарий	Среднее	Максимум	Вердикт
Очень глубокая вложенность (500 уровней)	9 мс	30 мс	Проходит
Широкая мапа (1000 записей)	15 мс	40 мс	Проходит

Все результаты уложились в критерии. Я также сравнил производительность с Copy-on-Write и без. Без CoW на 10 000 элементах время было около 50 мс, с CoW – 20 мс. Разница в три раза. Однако пришлось выключить этот набор тестов, поскольку на раннере CI/CD результаты более неожиданные, и из-за этого целый пайплайн может покраснеть – было принято решение отключить эти тесты по умолчанию.

7.6 Выводы по тестированию

Я понял, что тесты нужно писать параллельно с кодом. Каждый новый кейс рождал новый тест. Это помогало сразу замечать ошибки. Интеграционные тесты на сложных типах — самые полезные. Производительность нужно мерить. CoW оказался не микрооптимизацией, а реальным улучшением на больших данных. А кроме того, сквозные тесты на тестовом сервисе – отдельный уровень уверенности. Только они проверяют работу в реальном HTTP-сервере.

7.7 Результаты этапа

Результатом этапа стали:

- OpenAPI-спецификация новых endpoint в тестовом сервисе (Приложение Д, листинг Д.2).
- Листинг изменений в тестовом сервисе (Приложение Д, листинг Д.3).
- Скриншоты логов после внедрения (Приложение Д, рисунок Д.4) — наглядно показывают, что все чувствительные данные заменены на ****).
- График производительности (Приложение Д, рисунок Д.5) — зависимость времени маскировки от размера JSON, построенный на основе замеров.

8 ВНЕДРЕНИЕ МЕХАНИЗМА В СИСТЕМУ ЛОГИРОВАНИЯ

8.1 Постановка задачи

После того как механизм был разработан и протестирован, нужно было интегрировать его в платформу так, чтобы он стал доступен всем сервисам-потребителям. Задача: настроить подключение к компонентам маршрутизации запросов и ответов, обеспечить минимальное влияние на производительность и проверить отсутствие утечек в среде, подобной production.

8.2. Интеграция через Spring Boot AutoConfiguration

Чтобы механизм подключался автоматически при обновлении версии платформы, я использовал Spring Boot AutoConfiguration. В модулях `devp-access-logging-webflux-client-spring-boot-starter` и `devp-access-logging-webflux-server-spring-boot-starter` я создал конфигурации, которые регистрируют необходимые бины только при включённой настройке.

Основные бины:

- `AccessLogWriter` — для непосредственной записи в лог (использует `kotlin-logging`).
- `CompositeSensitiveMasker` — оркестратор маскировки (собирает анализатор типов и маскировщики).
- `AutoBodySerializationStrategy` или `RawBodySerializationStrategy` — в зависимости от режима.
- `RequestProcessingStage` — управляет жизненным циклом логирования запроса-ответа.
- `AccessLogWebFilter` (для сервера) и `AccessLogExchangeFilterFunction` (для клиента) — точки интеграции.

8.3 Настройка через `application.yml`

Чтобы команды могли управлять поведением, я добавил несколько конфигурационных параметров, пример в приложении Е, листинг Е.1.

8.4 Проверка в среде, подобной production

Этот пункт был выполнен во время сквозного тестирования в этапе 7. Я развернул тестовый сервис с включённым access-логированием в режиме AUTO. Утечек не обнаружено — все поля, помеченные `CloudSensitive`, а также ключи из `sensitive-json-keys`, были замаскированы.

Производительность в норме – дополнительная задержка не превышала 1–2 мс для большинства запросов (на искусственных больших JSON – до 10 мс, что соответствовало нагрузочным тестам). Ошибок не возникало – механизм устойчиво работал на реальных DTO.

8.5 Что я понял

1. AutoConfiguration – это удобно. Командам не нужно писать код для включения маскировки. Достаточно обновить версию платформы и при необходимости добавить пару строк в application.yml.
2. Обратная совместимость — это ответственность. Нельзя просто так взять и включить WITH_BODY по умолчанию. Нужно дать командам возможность выбора.
3. Тестирование в среде, близкой к production, выявляет проблемы, которых нет в локальных тестах.

8.6 Результаты этапа

- Фрагмент конфигурации (application.yml) в листинге E.1.
- Примеры access-логов после внедрения (Приложение E, рисунок E.2) — скриншоты, на которых видно, что чувствительные поля заменены на ****.
- Код AutoConfiguration и свойства включены в Приложение E (листинг E.3).

9 ОФОРМЛЕНИЕ ОТЧЕТНЫХ ДОКУМЕНТОВ

9.1 Подготовка к написанию отчёта

Перед тем как приступить к оформлению отчёта, я подключился к онлайн-консультации, которая проводилась 26.03.2026 (запись консультации была выложена в Telegram-канале практики). На консультации руководитель практики подробно разбирала типичные ошибки в отчётах студентов — как успешных, так и неудачных.

Что я вынес из консультации:

1. Главная ошибка — описание результата вместо описания процесса. Например, «я создал класс такой-то» — это результат. А «я столкнулся с проблемой, начал гуглить, нашёл решение, подумал, придумал, переделал, и вот что получилось» — это описание процесса.
2. Отчёт должен отвечать на вопрос «как я пришёл к решению?», а не просто «что я сделал?».
3. Результаты этапов (код, схемы, скриншоты) нужно выносить в приложения, чтобы не загромождать основную часть.
4. Важно указывать, с какими трудностями столкнулся и как их преодолел — это показывает глубину проработки.

9.2 Процесс написания отчёта

Я писал отчёт параллельно с выполнением этапов практики. Это позволило не упустить детали и сохранить «свежие» впечатления от решения проблем. Для каждого этапа я фиксировал:

- Что требовалось сделать (из индивидуального задания).
- Как я подошёл к решению (какие варианты рассматривал, почему выбрал именно этот).
- С какими трудностями столкнулся (и почему я их не ожидал).
- Как я их преодолел (и что из этого понял).
- Что получилось в итоге (со ссылкой на приложение).

Тем не менее, по результатам некоторых этапов нужно было делать шаг назад, чтобы пересмотреть уже написанные разделы. Например, когда я описывал первую итерацию (маскировку через десериализацию), я изначально написал её как успешный этап — ведь код работал, тесты проходили, изменения влились. Но после того как в production посыпались ворнинги, мне пришлось вернуться и переписать этот раздел, добавив рефлексию: почему я думал, что всё хорошо, почему ошибся и что из этого понял.

Аналогично, когда я описывал архитектуру второй итерации, я сначала упустил важный момент про передачу `ResolvableType` между компонентами. Это выяснилось уже в процессе реализации, и мне пришлось вернуться к разделу 3, добавив этот нюанс. И таких возвратов было несколько — каждый раз, когда я находил несоответствие между тем, что написал в отчёте, и тем, что реально получилось в коде.

Я понял, что отчёт — это не документ, который пишется один раз «от и до». Он живёт вместе с проектом. Идеально было бы писать его параллельно, но иногда приходится переписывать задним числом. Главное — фиксировать не только финальный результат, но и то, как ты к нему пришёл, включая ошибки и возвраты. Именно это делает отчёт честным и полезным.

9.3 Трудности при оформлении

Самая большая трудность — оформление приложений с кодом и диффами. Объём изменений по задачам MWSDEVP-5799, 6105, 6706, 6818 составил суммарно около десятка тысяч строк. Вставлять их целиком в отчёт было невозможно — это сделало бы документ нечитаемым. Но и вырезать важные фрагменты было нельзя, потому что без кода невозможно понять, как именно работает то, что я сделал.

Как я (попробовал) решить эту проблему: в часть приложений я включил ключевые фрагменты кода (основные методы, структуры данных, точки интеграции), а не полные файлы. Тем не менее, многое вырезать было нельзя, и все же объём приложений вышел гигантским. Я надеюсь, что навигация по содержанию может помочь при чтении отчета.

9.4 Соблюдение требований к оформлению

При оформлении отчёта я руководствовался методическим пособием (<https://books.ifmo.ru/file/pdf/2622.pdf>) и требованиями из индивидуального задания.

9.5 Что я понял

1. Отчёт — это не просто формальность. Когда я начал описывать свои решения словами, я сам лучше понял, почему я сделал те или иные выборы. Это помогло мне структурировать мысли.
2. Консультация была полезной. Без разбора чужих ошибок я бы, скорее всего, написал отчёт в стиле «результатов», а не «процесса». И проверяющая это бы заметила.
3. Приложения с кодом — это необходимое зло. Да, они увеличивают объём отчёта и усложняют вёрстку. Но без них невозможно оценить

глубину проработки. Я постарался найти баланс между полнотой и читаемостью.

4. Оформление по ГОСТу — это не страшно. Если сразу следовать требованиям, а не переделывать потом, то это не отнимает много времени, к тому же этот же шаблон я использовал во время учебной практики.

9.6 Результаты этапа

Полный текст отчёта — представлен выше (введение, основная часть с этапами 2–8, заключение). Приложения — содержат коды, схемы, скриншоты, конфигурации и другие результаты этапов.

Отчёт направлен на проверку руководителю практики.

ЗАКЛЮЧЕНИЕ

В ходе выполнения производственной практики по теме «Разработка механизма автоматической анонимизации персональных и секретных данных в журналах сетевого взаимодействия на основе метаданных типов» были успешно выполнены все этапы индивидуального задания (2–8) в соответствии с рабочим планом-графиком из индивидуального задания. Практика позволила не только получить работающий программный продукт, интегрированный в JVM-платформу МВС, но и приобрести ключевые профессиональные компетенции в области проектирования высоконагруженных систем, работы с Kotlin reflection, Jackson и Spring WebFlux, а также навыки эволюционной разработки, которые я оцениваю как значительный шаг в своём развитии как инженера.

Проделанная работа может быть структурирована вокруг трёх основных блоков:

1. Эволюционное проектирование архитектуры маскировки (Этап 3). Ключевым достижением этого этапа стало не просто создание архитектурной схемы, а осознанный переход от простого, но хрупкого решения к надёжной и производительной архитектуре. Первая попытка (десериализация через кастомные Jackson-модули) натолкнулась на фундаментальные ограничения: невозможность десериализовать типы и классы из библиотек платформы без переработки многих существующих особенностей платформы, а также избыточная нагрузка на парсинг даже для безопасных тел. Анализ этих проблем привёл меня к проектированию многоуровневой архитектуры с прямым обходом JSON-дерева, кэшированием метаданных и zero-overhead для тел, не требующих маскировки. Я понял, что в платформенной разработке важнее не скорость первой реализации, а способность предвидеть границы применимости решения. Ошибка на этапе проектирования обходится в сотни раз дороже, чем на этапе кодирования — этот урок я усвоил на собственном опыте. Этот опыт научил меня критически оценивать инженерные решения и не бояться переписывать то, что работает «вроде бы», если оно не масштабируется.

2. Реализация ключевых компонентов и работа со сложными структурами данных (Этапы 4–6). Особую сложность представляла поддержка generic-обёрток (CloudOptional, Mono, Flux, ResponseEntity), обработка рекурсивных ссылок и корректная работа с nullable-значениями. Я научился использовать Kotlin reflection в production-коде, проектировать рекурсивные алгоритмы обхода деревьев (JSON и графа типов) и применять композитные решения для объединения двух независимых стратегий.

3. Многоуровневое тестирование и внедрение в платформу (Этапы 7–8). Я провёл юнит-тестирование (изолированные сценарии), интеграционное тестирование, сквозное тестирование на тестовом сервисе с реальными HTTP-запросами, а также нагрузочное тестирование на телах до 10 МБ и глубине вложенности до 500 уровней. Результаты показали, что механизм добавляет не более 10–25 мс на обработку даже для кошмарно-максимальных объёмов, а для безопасных тел оверхед стремится к нулю. После успешного регрессионного тестирования механизм был интегрирован в платформу, и теперь доступен всем сервисам-потребителям при обновлении версии платформы.

Разработанный мною механизм автоматической маскировки конфиденциальных данных стал не просто очередным функциональным улучшением платформы MWS — он изменил подход к безопасности access-логов во всей экосистеме. До этого момента логирование тел HTTP-запросов и ответов было отключено по умолчанию, потому что любая попытка включить его в production могла привести к риску утечки паролей, токенов, ключей и персональных данных. Разработчики сервисов либо жили без диагностической информации, либо включали логирование с оговорками, понимая, что рано или поздно что-то «упадёт» в лог в открытом виде.

Что изменилось после моей работы?

Во-первых, появилась возможность безопасно включить логирование тел по умолчанию. Обновление платформы автоматически даёт всем сервисам-потребителям access-логи с маскировкой чувствительных полей. Для этого не нужно писать дополнительный код, не нужно менять DTO, не нужно думать о том, не забыл ли разработчик обернуть поле в CloudSensitive. Механизм работает «из коробки», а если команде нужна более тонкая настройка (например, добавить свои ключи для маскировки или отключить маскировку вовсе) — это делается через пару строк в `application.yml`.

Во-вторых, решение оказалось достаточно производительным, чтобы не стать узким местом. Нагрузочные тесты показали, что даже для тяжёлых JSON-тел объёмом более 10 МБ с тысячами вложенных элементов маскировка добавляет не более 20–30 миллисекунд к времени обработки запроса, а для тел, не содержащих чувствительных данных, оверхед стремится к нулю (благодаря кэшированию и раннему выходу). Это значит, что ни один сервис не заметит ухудшения производительности, а безопасность при этом вырастет кардинально.

В-третьих, механизм оказался устойчивым к реальным сложностям, которые возникают в больших проектах: абстрактные типы, generic-обёртки, рекурсивные ссылки, бинарные тела, битый JSON. Это защищает production от внезапных сбоев. Кроме того, я перестал бояться сложных тем вроде reflection, generic-типов и асинхронного программирования.

В-четвёртых, обратная совместимость была соблюдена. Сервисы, которые не обновляли конфигурацию, продолжают работать в старом режиме. Сервисы, которые уже включали логирование тел вручную, после обновления платформы автоматически получают маскировку — без какого-либо рефакторинга. Это, на мой взгляд, главный признак качественной платформенной работы: ты даёшь людям новые возможности, но не заставляешь их ничего переписывать.

Когда я начинал практику, я думал, что главное — написать код, который работает. Теперь я понимаю, что настоящая инженерная ценность — это код, который работает в production, не ломает обратную совместимость, не убивает производительность, легко настраивается и понятен другим разработчикам. Мой механизм прошёл код-ревью, регрессионное тестирование, нагрузочные испытания и теперь используется в реальных сервисах МВС. Это даёт мне чувство, что я сделал не просто учебное задание, а реальный вклад в продукт, который приносит пользу.

Но, пожалуй, самым ценным опытом для меня стала работа в реальной команде. В университете мы привыкли делать проекты в одиночку или в небольших группах, где всё решается внутренними договорённостями. Здесь же я столкнулся с тем, что каждое моё решение нужно было обсуждать, аргументировать и защищать перед более опытными коллегами. Код-ревью — это не формальность, а реальный фильтр, который отсеивает наивные подходы и заставляет искать более надёжные и производительные решения. Платформа — это инструмент, которым пользуются десятки других разработчиков. Любое изменение, даже самое полезное, может их сломать, если не подумать о совместимости. Я осознанно проектировал конфигурационные параметры так, чтобы по умолчанию всё работало безопасно, но при этом у команд была возможность отключить маскировку, если она им мешает. Кроме того я писал документацию, чтобы люди реально поняли, как пользоваться новым функционалом. Это сильно отличается от университетских отчётов, которые читает только преподаватель. Ещё один важный аспект — работа с чужим кодом. Платформа MWS существует не первый год, и в ней есть свои устоявшиеся паттерны, соглашения и ограничения. Мне пришлось не просто писать «свой» код, а встраиваться в существующую экосистему, не нарушая её целостности. Это научило меня читать и понимать чужой код, находить точки интеграции, не ломая

существующие контракты, и уважать решения, принятые до меня (даже если они кажутся неоптимальными).

Что осталось за рамками практики и что хочется сделать дальше? В текущей реализации маскировка по типам работает только для типов, которые можно проанализировать через reflection на стороне сервера (когда у нас есть информация о DTO). Для исходящих запросов клиента поддержка generic-типов на верхнем уровне требует дополнительной информации, которая не всегда доступна. Это известное ограничение, которое я задокументировал. В будущем планируется его устранить, возможно, передавая более богатую информацию о типах через атрибуты запроса. Также можно добавить поддержку других форматов (не только JSON) и более гибкую конфигурацию стратегий маскировки, если от использующих платформу команд возникнет такой запрос.

Я убедился, что способен решать сложные, многогранные задачи, которые требуют не только программирования, но и проектирования, анализа рисков, тестирования, документирования и коммуникации с другими людьми — коллегами по команде, ревьюерами. Работа в реальной команде показала мне, что инженерия — это не спорт одиночек, а коллективный процесс, где успех зависит от умения слушать, убеждать, идти на компромиссы и при этом отстаивать технически правильные решения. Это даёт мне уверенность в своих силах перед выпускной квалификационной работой и перед выходом на полноценную работу в индустрии. И это, пожалуй, самое ценное, что я вынес из производственной практики.

ПРИЛОЖЕНИЕ А

Функциональные требования

ID	Требование	Приоритет
FR-01	Механизм должен маскировать значения полей, объявленных как CloudSensitive<T>, заменяя их на **** при записи в access-лог.	Высокий
FR-02	Маскировка должна применяться как к входящим запросам (серверная часть), так и к исходящим (клиентская часть).	Высокий
FR-03	Механизм должен поддерживать обработку JSON-тел запросов и ответов. Тела других форматов (например, application/octet-stream) не маскируются, а логируются маркером [BINARY_CONTENT].	Высокий
FR-04	Механизм должен поддерживать два режима сериализации: RAW (тело логируется как есть) и AUTO (применяется маскировка). Значение по умолчанию — AUTO.	Средний
FR-05	Механизм должен поддерживать маскировку по ключам JSON: значения всех полей, чьи имена совпадают (или содержат подстроку) с настраиваемым списком, заменяются на ****. Список ключей по умолчанию: password, token.	Средний
FR-06	Механизм должен корректно обрабатывать вложенные объекты, коллекции (List, Set), массивы, ассоциативные массивы (Map).	Высокий
FR-07	Механизм должен корректно обрабатывать generic-обёртки: CloudOptional<T>, Mono<T>, Flux<T>, ResponseEntity<T>.	Высокий
FR-08	Механизм должен корректно обрабатывать nullable-значения (CloudSensitive<String>?).	Средний
FR-09	При обнаружении циклической ссылки в структуре данных механизм должен прекращать рекурсивный обход и продолжать работу без исключения.	Высокий

Таблица А.1 – Функциональные требования

Нефункциональные требования

ID	Требование	Приоритет
NFR-01	Механизм должен быть задокументирован для команд-потребителей (описание конфигурационных параметров, примеры использования, известные ограничения).	Высокий
NFR-02	Должна быть возможность логировать тело без обработки, задаётся через конфигурацию.	Высокий
NFR-03	Механизм не должен вызывать падение сервиса при любых ошибках (битый JSON, неожиданный тип, рекурсия). Все ошибки должны логироваться, но не влиять на обработку запроса.	Средний
NFR-04	Сервисы, не обновлявшие конфигурацию, должны продолжать работать в прежнем режиме. Включение маскировки не должно требовать изменения кода в сервисах-потребителях.	Высокий

Таблица А.2 – Нефункциональные требования

Задача

Платформа разработки MWS / MWSDEV-5799
Поддержать CloudSensitive в access log

Редактировать Добавить комментарий Назначить Ещё... Закрыть

Электронная почта Экспорт

Детали задачи

Тип: История Решение: Готово
Приоритет: Medium
Компоненты: Не заполнено
Метки: Не заполнено

Поле Вкладки Оценка ИИ

Ссылка на эпик: accesslog sensitive
Story points: Не заполнено

Описание
Нажмите для ввода описания

Smart Checklist
0 / 0

Вложенные файлы

Перенесите файлы, чтобы прикрепить, или обзор.

Tempo
У ВАС НЕТ ДЕЙСТВИЙ ПО ЭТОЙ ЗАДАЧЕ.

Люди

Исполнитель: Барсуков Максим
Автор: Шалару Игорь
Голоса: Голосовать за эту задачу
Наблюдатели: Начать наблюдение за этой задачей

Даты

Сотрудники

Разработка

Agile
Найти на доске

Рисунок А.3 – Задача MWSDEV-5799

ПРИЛОЖЕНИЕ Б

Платформа разработки MWS / MWSDEVP-4267

Переключить стратегию записи AccessLog по-умолчанию на WITH_BODY

Редактировать Добавить комментарий Назначить Еще... Закрыт

Электронная почта

Детали задачи

Тип: История
Приоритет: Medium
Компоненты: platform-kotlin
Метки: Не заполнено

Решение: Готово

Поле Вкладки Оценка ИИ

Ссылка на эпик: accesslog_sensitive
Story points: Не заполнено

Описание
Переключить стратегию записи AccessLog по-умолчанию на WITH_BODY.
Отразить в kdos на проптерги, в доке для пользователя, в changelog. В гайде по миграции написать чтобы включили without_body, если не включали явно.
Дизайн док не нужен.

Smart Checklist
0 / 0
Add a checklist item

Вложенные файлы
Перенесите файлы, чтобы прикрепить, или обзор.

Люди
Исполнитель: Барсуков Максим
Автор: Гурьев Сергей
Голоса: 0 Голосовать за эту задачу
Наблюдатели: 2 Начать наблюдение за этой задачей

Даты

Сотрудники

Разработка

Agile
Найти на доске

Рисунок Б.1 – Задача MWSDEVP-4267

Платформа разработки MWS / MWSDEVP-6038

Ошибка при попытке распарсить объект в access log фильтре

Редактировать Добавить комментарий Назначить Еще... Закрыт

Электронная почта

Детали задачи

Тип: Ошибка
Приоритет: Medium
Компоненты: Не заполнено
Метки: Не заполнено

Решение: Готово

Ссылка на эпик: api_duty_25Q4
Тип дефекта: ПО
Этап обнаружения: Инсталляционное Тестирование
Метод обнаружения: Регрессионное тестирование

Описание
ссылка на сообщение: https://t.me/c/1702437212/1/22934
Привет!
При обновлении платформы на 0.69 и выше стала ловить warn типа
"logger": "cloud.mws.devp.access.logging.spring.webflux.common.impl.ResponseBodySerializationStrategy",
"message": "Failed to mask CloudSensitive fields in body. Class: class cloud.mws.iam.bff.dto.ServiceAccountResponse, Content-Type: application/json"
stacktrace
"stack_trace": "com.fasterxml.jackson.databind.exc.InvalidDefinitionException: Cannot construct instance of
'cloud.mws.devp.cloud.references.ResourceId' (no Creators, like default constructor, exist): abstract types either need to be mapped to concrete
types, have custom deserializer, or contain additional type information\n at [Source: REDACTED
(StreamReadFeature.INCLUDE_SOURCE_IN_LOCATION disabled); line: 3, column: 12] (through reference chain:
cloud.mws.iam.bff.dto.ServiceAccountResponse[\"metadata\"]->cloud.mws.common.dto.CommonTypedResourceMetadataResponse[\"id\"]]
Ссылка на лог stand24q1 https://grafana.preprod.obs.mws-team.ru/goto/f7z1zjplhcf?orgid=1
traceId для stand24q1
"trace_id": "648ad7d2700619bfd73bd2f55a209c43"
На 68 версии таких ошибок нет, нужны какие-то доп настройки мапперов?

Лбзавта комментариев

Люди
Исполнитель: Барсуков Максим
Автор: Шалару Игорь
Голоса: 0 Голосовать за эту задачу
Наблюдатели: 1 Начать наблюдение за этой задачей

Даты

Сотрудники

Разработка

Agile
Найти на доске

Рисунок Б.2 – Задача MWSDEVP-6038

```
diff --git a/.changes/next/MWSDEVP-5799.yaml
b/.changes/next/MWSDEVP-5799.yaml
new file mode 100644
index
0000000000000000000000000000000000..1901a8e3531774f2ca4c6a10a
6f7696ed300c537
--- /dev/null
+++ b/.changes/next/MWSDEVP-5799.yaml
@@ -0,0 +1,4 @@
+Feature:
+ - "Добавлена поддержка маскировки CloudSensitive полей в Access
log"
+ - "Добавлен новый параметр конфигурации
devp.api.access-log.body-serialization-mode со значениями AUTO (по
умолчанию) и RAW "
+ - "В режиме AUTO скрываются CloudSensitive поля в JSON телах
запросов/ответов; не-JSON body не выводится"
diff --git a/docs/site/user/pages/reference/obs/access-logging.md
b/docs/site/user/pages/reference/obs/access-logging.md
index
3f518be9c06ea0d0e12ef9cb5344081219936fae..73309ad31e3e13989e1691625
3dacdf03d22a67e 100644
--- a/docs/site/user/pages/reference/obs/access-logging.md
+++ b/docs/site/user/pages/reference/obs/access-logging.md
@@ -49,6 +49,7 @@ devp:
    server-log-enabled: true # По умолчанию логирование запросов
к серверу включено
    client-log-enabled: true # По умолчанию логирование запросов
клиентов включено
    strategy: WITHOUT_BODY # По умолчанию логирование body
выключено
+    body-serialization-mode: AUTO # Режим сериализации тела:
AUTO (по умолчанию) или RAW
    without-body-endpoints: # Список endpoints, в формате regex,
для которых HTTP body не логируется,
                                # по умолчанию содержит /metrics,
/customer-metrics
    - /metrics
@@ -57,3 +58,13 @@ devp:
    - header1
    - header2
...

+
+### Режим `body-serialization-mode`
+
+Параметр `devp.api.access-log.body-serialization-mode` управляет
тем, как сериализуется тело запроса/ответа при записи в access log
в том случае, если включена стратегия `WITH_BODY`.
+
+- `RAW` - тело логируется в исходном виде, без изменений
+- `AUTO` (значение по умолчанию) - автоматическая обработка тела:
+ - Если тело имеет тип `application/json` и тип поля в DTO
объявлен как
[`CloudSensitive<T>`](`../apps/api/api-cloud-sensitive.md`), то
такие поля заменяются на `****`.
+ - Если тело не является JSON (например,
```

```

`application/octet-stream`) - тело не логируется (поле `body`
записывается как `[BINARY_CONTENT]`).
+ - В случае ошибки десериализации JSON - тело не логируется, в
лог пишется предупреждение.
diff --git
a/libraries/cloud-client-impl/src/main/kotlin/cloud/mws/devp/cloud/
client/impl/BaseCloudApiClient.kt
b/libraries/cloud-client-impl/src/main/kotlin/cloud/mws/devp/cloud/
client/impl/BaseCloudApiClient.kt
index
6ba998605e78a32a92bfeda87a3c66cd7d6c6385..3810f10d44eabeb4fce15226d
f6ac53000585993 100644
---
a/libraries/cloud-client-impl/src/main/kotlin/cloud/mws/devp/cloud/
client/impl/BaseCloudApiClient.kt
+++
b/libraries/cloud-client-impl/src/main/kotlin/cloud/mws/devp/cloud/
client/impl/BaseCloudApiClient.kt
@@ -90,6 +90,8 @@ open class BaseCloudApiClient(
    // path здесь сразу шаблонизированный и без query params
    request.attribute("path", path)
    request.attribute("pathParam", pathList)
+    request.attribute("requestType", requestBody?.javaClass ?:
Unit::class.java)
+    request.attribute("responseType", T::class.java)
    requestMediaType?.let {
request.contentType(MediaType.valueOf(it)) }
    requestBody?.let { request.bodyValue(it) }
    request.accept(MediaType.valueOf(responseMediaType))
diff --git
a/libraries/devp-access-logging-spring-webflux/build.gradle.kts
b/libraries/devp-access-logging-spring-webflux/build.gradle.kts
index
17a7968f43ce121bfade9e3b84aea1a0ffdc8a83..a69905053070ab98ca4537ade
2ae29d0d068a0c6 100644
--- a/libraries/devp-access-logging-spring-webflux/build.gradle.kts
+++ b/libraries/devp-access-logging-spring-webflux/build.gradle.kts
@@ -10,6 +10,7 @@ dependencies {
    api(project(":uri-matchers"))
    api(project(":spring-web-integration"))
    api(project(":devp-context"))
+    api(project(":cloud-sensitive"))

    api("org.springframework:spring-web")
    api("org.springframework:spring-webflux")
@@ -18,6 +19,9 @@ dependencies {
    implementation("org.slf4j:slf4j-api")
    implementation("org.springframework:spring-context")

+    implementation("com.fasterxml.jackson.core:jackson-databind")
+
+    implementation("com.fasterxml.jackson.module:jackson-module-kotlin"
)
+
testImplementation("org.springframework.boot:spring-boot-starter-we

```

```

bflux")

testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("io.kotest:kotest-assertions-core")
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogExchangeFilterFunction.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogExchangeFilterFunction.kt
index
566fbb299d98f883c7b8189ac3d5fa0a90b824bc..d7ddd26598baeefa00661113db934f9658957b8a 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogExchangeFilterFunction.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogExchangeFilterFunction.kt
@@ -45,10 +45,26 @@ class AccessLogExchangeFilterFunction(
    return preCorrelation
}

+    private fun resolveRequestClass(request: LocalClientRequest): Class<*>? {
+        return request.attributes["requestType"] as? Class<*>
+    }
+    private fun resolveResponseClass(request: LocalClientRequest): Class<*>? {
+        return request.attributes["responseType"] as? Class<*>
+    }
+
    override fun filter(request: ClientRequest, next: ExchangeFunction): Mono<ClientResponse> {
        val clientRequest = LocalClientRequest(request)
        val preCorrelation = makePreCorrelation(clientRequest)
-        val requestWritingStage =
requestProcessingStage.process(preCorrelation, clientRequest)
+
+        val requestClass = resolveRequestClass(clientRequest)
+        val responseClass = resolveResponseClass(clientRequest)
+        val requestWritingStage = requestProcessingStage.process(
+            preCorrelation,
+            clientRequest,
+            requestClass,
+            responseClass
+        )
        val writeHook: () -> Unit = { requestWritingStage.write()
    }

    val bufferedClientRequest = ClientRequest
        .from(request)

```

```

diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogWebFilter.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogWebFilter.kt
index
cde9fc10619588ea53cde9ef03f0f70447523586..b04501cf8e151b0578fda6c5ebce9e211baff093 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogWebFilter.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/AccessLogWebFilter.kt
@@ -10,8 +10,12 @@ import cloud.mws.devp.base.usage.DevpInternal
import cloud.mws.devp.matchers.uri.UriTemplateMatcher
import cloud.mws.devp.spring.web.integration.getRequestPaths
import io.opentelemetry.api.trace.Span
+import org.springframework.core.ResolvableType
import org.springframework.core.env.Environment
+import org.springframework.http.ResponseEntity
+import org.springframework.web.bind.annotation.RequestBody
import org.springframework.web.method.HandlerMethod
+import org.springframework.web.reactive.HandlerMapping
import
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping
import org.springframework.web.server.ServerWebExchange
import org.springframework.web.server.WebFilter
@@ -67,13 +71,49 @@ class AccessLogWebFilter(
        .onErrorResume { makeEmptyPreCorrelation(exchange) }
    }

+    private fun getHandler(exchange: ServerWebExchange):
+    HandlerMethod? {
+        return
+exchange.getAttribute<HandlerMethod>(HandlerMapping.BEST_MATCHING_HANDLER_ATTRIBUTE)
+    }
+
+    private fun resolveRequestClass(handler: HandlerMethod?):
+    Class<*>? {
+        val requestBodyParam = handler?.methodParameters
+        ?.filter {
+it.hasParameterAnnotation(RequestBody::class.java) }
+        return requestBodyParam?.firstOrNull()?.parameterType
+    }
+
+    private fun resolveResponseClass(handler: HandlerMethod?):
+    Class<*>? {
+        val returnType = handler?.returnType
+        ?.let { ResolvableType.forMethodParameter(it) } ?:
+return null
+
+        val responseRawClass = returnType.rawClass
+        return if (responseRawClass != null &&

```

```
ResponseEntity::class.java.isAssignableFrom(responseRawClass)) {
+         returnType.getGeneric(0).toClass()
+     } else {
+         returnType.toClass()
+     }
+ }
+
    override fun filter(exchange: ServerWebExchange, chain:
WebFilterChain): Mono<Void> {
        val serverRequest = ServerRequest(exchange.request)
        val serverResponse = ServerResponse(exchange.response)

requestProcessingStage.updateDefaultStrategyStatus(serverRequest,
serverResponse)
        val stage = AtomicReference<Any>(requestProcessingStage)
        return makePreCorrelation(exchange)
-         .doOnNext { stage.updateAndGet { s -> (s as
RequestProcessingStage).process(it.second, serverRequest) } }
+         .flatMap { (exchange, preCorrelation) ->
+             val handler = getHandler(exchange)
+             val requestClass = resolveRequestClass(handler)
+             val responseClass = resolveResponseClass(handler)
+
+             val updatedStage = requestProcessingStage.process(
+                 preCorrelation,
+                 serverRequest,
+                 requestClass,
+                 responseClass
+             )
+             stage.set(updatedStage)
+
+             Mono.just(exchange to preCorrelation)
+         }
        .map { (exchange, _) ->
            exchange.mutate()
                .request(

diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/BodySerializationSt
ategy.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/BodySerializationSt
ategy.kt
new file mode 100644
index
00000000000000000000000000000000..d0f3856181d8e48176013e58f
3d950e765867c0e
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/BodySerializationSt
ategy.kt
@@ -0,0 +1,8 @@
+package cloud.mws.devp.access.logging.spring.webflux.common
+
+import cloud.mws.devp.base.usage.DevpInternal
```



```

+
+@DevpInternal
+interface BodySerializationStrategy {
+    fun serializeBody(body: String, bodyClass: Class<*>?,
contentType: String?): String
+}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/RequestProcessingSt
age.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/RequestProcessingSt
age.kt
index
6bf6099bb1a7cfa1b715211019ff02a4c2a05441..051f95d52fcf5c511580a3869
2cc10fcc5bf6f1b 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/RequestProcessingSt
age.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/RequestProcessingSt
age.kt
@@ -9,7 +9,12 @@ import cloud.mws.devp.base.usage.DevpInternal
    */
    @DevpInternal
    interface RequestProcessingStage {
-        fun process(preCorrelation: PreCorrelation, request:
HttpRequest): RequestWritingStage
+        fun process(
+            preCorrelation: PreCorrelation,
+            request: HttpRequest,
+            requestClass: Class<*>?,
+            responseClass: Class<*>?
+        ): RequestWritingStage

        fun updateDefaultStrategyStatus(request: HttpRequest,
response: HttpResponse)
    }
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/SensitiveValues.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/SensitiveValues.kt
new file mode 100644
index
0000000000000000000000000000000000000000000000000000000000000000..eda9f428371e9b8cf3e7024ad
143e2266e5f8598
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/SensitiveValues.kt
@@ -0,0 +1,8 @@
+package cloud.mws.devp.access.logging.spring.webflux.common
+

```

```

+import cloud.mws.devp.base.usage.DevpInternal
+
+@DevpInternal
+object SensitiveValues {
+    const val MASKED_VALUE = "*****"
+}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Sink.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Sink.kt
index
59526cb9bca70b1af9d482cf3aa7c16ace7f7068..8db9a3aaf1a4e691f20b574649c5dfa9c36686e7 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Sink.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Sink.kt
@@ -12,12 +12,18 @@ interface Sink {
    val isActive: Boolean
    get() = true

-    fun write(preCorrelation: PreCorrelation, request:
HttpRequest)
+    fun write(preCorrelation: PreCorrelation, request:
HttpRequest, requestClass: Class<*>?)

-    fun write(correlation: Correlation, request: HttpRequest,
response: HttpResponse)
+    fun write(correlation: Correlation, request: HttpRequest,
response: HttpResponse, responseClass: Class<*>?)

-    fun writeBoth(preCorrelation: PreCorrelation, request:
HttpRequest, response: HttpResponse) {
-        write(preCorrelation, request)
-        write(preCorrelation.correlate(), request, response)
+    fun writeBoth(
+        preCorrelation: PreCorrelation,
+        request: HttpRequest,
+        response: HttpResponse,
+        requestClass: Class<*>?,
+        responseClass: Class<*>?
+    ) {
+        write(preCorrelation, request, requestClass)
+        write(preCorrelation.correlate(), request, response,
responseClass)
+    }
}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Strategy.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/Strategy.kt
index

```

```

818ead9fa4fa6b04ba88b57d360d1b04c2f6bb51..982e26d7b406ae7764c01343a
9a05664526d4bf6 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/Strategy.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/Strategy.kt
@@ -42,15 +42,17 @@ interface Strategy {
    *
    * @param preCorrelation предварительная [PreCorrelation] мета
информация для запроса
    * @param request          текущий запрос
+    * @param requestClass    класс текущего запроса
    * @param sink              sink для вывода записи
    * @see Sink.write
    */
    fun write(
        preCorrelation: PreCorrelation,
        request: HttpRequest,
+        requestClass: Class<*>?,
        sink: Sink
    ) {
-        sink.write(preCorrelation, request)
+        sink.write(preCorrelation, request, requestClass)
    }

    /**
@@ -76,9 +78,10 @@ interface Strategy {
    * По умолчанию вызываем [Sink.write].
    *
    * @param correlation Мета информация по запросу
-    * @param request      Текущий запрос
-    * @param response      Текущий ответ
-    * @param sink          Где происходит логирование
+    * @param request      Текущий запрос
+    * @param response      Текущий ответ
+    * @param responseClass Класс текущего ответа
+    * @param sink          Где происходит логирование
    * @see Sink.write
    * @see Sink.writeBoth
    */
@@ -86,8 +89,9 @@ interface Strategy {
        correlation: Correlation,
        request: HttpRequest,
        response: HttpResponse,
+        responseClass: Class<*>?,
        sink: Sink
    ) {
-        sink.write(correlation, request, response)
+        sink.write(correlation, request, response, responseClass)
    }
}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerial

```

```
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerializationStrategy.kt
new file mode 100644
index
000000000000000000000000000000000000..d12df6f11008ae7a0d898170ce16662a5fe4772c
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerializationStrategy.kt
@@ -0,0 +1,80 @@
+package cloud.mws.devp.access.logging.spring.webflux.common.impl
+
+import
cloud.mws.devp.access.logging.spring.webflux.common.BodySerializati
onStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.SensitiveValues
+import
cloud.mws.devp.access.logging.spring.webflux.common.http.ContentType
e
+import cloud.mws.devp.base.usage.DevpInternal
+import cloud.mws.devp.cloud.sensitive.CloudSensitive
+import cloud.mws.devp.cloud.sensitive.cloudSensitive
+import com.fasterxml.jackson.core.JsonGenerator
+import com.fasterxml.jackson.core.JsonParser
+import com.fasterxml.jackson.databind.BeanProperty
+import com.fasterxml.jackson.databind.DeserializationContext
+import com.fasterxml.jackson.databind.JavaType
+import com.fasterxml.jackson.databind.JsonDeserializer
+import com.fasterxml.jackson.databind.JsonSerializer
+import com.fasterxml.jackson.databind.ObjectMapper
+import com.fasterxml.jackson.databind.SerializerProvider
+import com.fasterxml.jackson.databind.deser.ContextualDeserializer
+import com.fasterxml.jackson.databind.module.SimpleModule
+import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
+import com.fasterxml.jackson.module.kotlin.registerKotlinModule
+import io.github.oshai.kotlinspring.logging.KotlinLogging
+
+@DevpInternal
+class AutoBodySerializationStrategy : BodySerializationStrategy {
+
+    private val logger = KotlinLogging.logger {}
+    private val binaryContentMarker = "[BINARY_CONTENT]"
+    private val jsonErrorMarker = "[JSON_ERROR]"
+
+    private val secureMapper: ObjectMapper = jacksonObjectMapper()
+        .registerKotlinModule()
+        .registerModule(
+            SimpleModule().apply {
+                addDeserializer(CloudSensitive::class.java,
CloudSensitiveMaskingDeserializer())
+                addSerializer(CloudSensitive::class.java,
```

```

CloudSensitiveMaskingSerializer()
+         }
+     )
+
+     override fun serializeBody(body: String, bodyClass: Class<*>?,
contentType: String?): String {
+         if (!ContentType.isJsonMediaType(contentType)) {
+             return binaryContentMarker
+         }
+
+         if (body.isBlank() || bodyClass == null) return body
+
+         return try {
+             val obj = secureMapper.readValue(body, bodyClass)
+             secureMapper.writeValueAsString(obj)
+         } catch (e: Exception) {
+             logger.warn(e) {
+                 "Failed to mask CloudSensitive fields in body. " +
+                 "Class: $bodyClass, Content-Type:
$contentType"
+             }
+             jsonErrorMarker
+         }
+     }
+
+     private class CloudSensitiveMaskingSerializer :
JsonSerializer<CloudSensitive<*>>() {
+         override fun serialize(value: CloudSensitive<*>, gen:
JsonGenerator, serializers: SerializerProvider) {
+             gen.writeString(SensitiveValues.MASKED_VALUE)
+         }
+     }
+
+     private class CloudSensitiveMaskingDeserializer :
JsonDeserializer<CloudSensitive<*>>(), ContextualDeserializer {
+         private lateinit var valueType: JavaType
+
+         override fun deserialize(parser: JsonParser, context:
DeserializationContext): CloudSensitive<*> {
+             return SensitiveValues.MASKED_VALUE.cloudSensitive()
+         }
+
+         override fun createContextual(context:
DeserializationContext, property: BeanProperty?):
JsonDeserializer<*> {
+             return CloudSensitiveMaskingDeserializer().apply {
+                 (context.contextualType ?:
property?.type)?.containedType(0)?.let {
+                     valueType = it
+                 }
+             }
+         }
+     }
+ }
+
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou

```

```

d/mws/devp/access/logging/spring/webflux/common/impl/BodySerializat
ionMode.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/BodySerializat
ionMode.kt
new file mode 100644
index
00000000000000000000000000000000000000000000000000000000000000000000..a569081cc8899717fd0a74407
3c7d63186b648ed
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/BodySerializat
ionMode.kt
@@ -0,0 +1,9 @@
+package cloud.mws.devp.access.logging.spring.webflux.common.impl
+
+import cloud.mws.devp.base.usage.DevpInternal
+
+@DevpInternal
+enum class BodySerializationMode {
+    RAW,
+    AUTO
+}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/DefaultRequest
ProcessingStage.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/DefaultRequest
ProcessingStage.kt
index
1bd29da8a9191a909de04446f65170974ac46932..868584a3924c068f814cac4b7
7f0fc8c238ef430 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/DefaultRequest
ProcessingStage.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/DefaultRequest
ProcessingStage.kt
@@ -27,11 +27,16 @@ class DefaultRequestProcessingStage(
) : RequestProcessingStage {
    private val logger = KotlinLogging.logger {}

-    override fun process(preCorrelation: PreCorrelation, request:
HttpRequest): RequestWritingStage {
+    override fun process(
+        preCorrelation: PreCorrelation,
+        request: HttpRequest,
+        requestClass: Class<*>?,
+        responseClass: Class<*>?
+    ): RequestWritingStage {
        if (sink.isActive) {
            try {

```

```

        val processedRequest = strategy.process(request)
-       return DefaultRequestWritingStage(preCorrelation,
processedRequest)
+       return DefaultRequestWritingStage(preCorrelation,
processedRequest, requestClass, responseClass)
    } catch (e: RuntimeException) {
        logger.warn(e) { "Unable to prepare request for
logging. Will skip the request & response logging step." }
    }
@@ -49,11 +54,13 @@ class DefaultRequestProcessingStage(
    private inner class DefaultRequestWritingStage(
        private val preCorrelation: PreCorrelation,
        private val request: HttpRequest,
+       private val requestClass: Class<*>?,
+       private val responseClass: Class<*>?
    ) : RequestWritingStage {

        override fun write(): ResponseProcessingStage {
            try {
-               strategy.write(preCorrelation, request, sink)
+               strategy.write(preCorrelation, request,
requestClass, sink)
            } catch (e: RuntimeException) {
                logger.warn(e) { "Unable to log request. Will skip
the request & response logging step." }
                return Stages.noop()
            }
@@ -66,7 +73,7 @@ class DefaultRequestProcessingStage(

            return ResponseWritingStage {
                try {
-                   strategy.write(preCorrelation.correlate(),
request, processedResponse, sink)
+                   strategy.write(preCorrelation.correlate(),
request, processedResponse, responseClass, sink)
                } catch (e: RuntimeException) {
                    logger.warn(e) { "Unable to log response. Will
skip the response logging step." }
                }
            }
        }
    }
}

diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink.kt
index
d9e41f0a3573f001a9e7a30b575a288d56b0e974..8fa231415b01f8d7e0dc874a4
601d7cfcb57cf78 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink.kt
@@ -3,6 +3,7 @@ package
cloud.mws.devp.access.logging.spring.webflux.common.impl
import cloud.mws.devp.access.logging.AccessLog
import cloud.mws.devp.access.logging.AccessLogWriter

```

```

import cloud.mws.devp.access.logging.Action
+import
cloud.mws.devp.access.logging.spring.webflux.common.BodySerializati
onStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.Correlation
import
cloud.mws.devp.access.logging.spring.webflux.common.PreCorrelation
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
@@ -17,6 +18,7 @@ import cloud.mws.devp.base.usage.DevpInternal
class LoggerSink(
    private val accessLogWriter: AccessLogWriter,
    private val headersWhiteList: List<String> = emptyList(),
+    private val bodySerializationStrategy:
BodySerializationStrategy = RawBodySerializationStrategy()
) : Sink {

    private fun queryMap(request: HttpRequest): Map<String,
String> {
@@ -30,7 +32,17 @@ class LoggerSink(
    }
}

-    override fun write(preCorrelation: PreCorrelation, request:
HttpRequest) {
+    override fun write(
+        preCorrelation: PreCorrelation,
+        request: HttpRequest,
+        requestClass: Class<*>?
+    ) {
+        val body = bodySerializationStrategy.serializeBody(
+            body = request.bodyAsString,
+            bodyClass = requestClass,
+            contentType = request.contentType
+        )
+
        accessLogWriter.write(
            AccessLog(
                startTime = preCorrelation.start,
@@ -52,13 +64,24 @@ class LoggerSink(
                acceptEncoding =
request.headers[org.springframework.http.HttpHeaders.ACCEPT_ENCODIN
G]?.first() ?: "",
                pathParam = preCorrelation.pathParam,
                queryParam = queryMap(request),
-                body = request.bodyAsString,
+                body = body,
                headers =
request.getHeaderListedHeaders(headersWhiteList),
            )
        )
    }

-    override fun write(correlation: Correlation, request:
HttpRequest, response: HttpResponse) {
+    override fun write(

```



```

d/mws/devp/access/logging/spring/webflux/TestApplication.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/TestApplication.kt
index
343b43f85299c93a3472b558317eefd6994e5235..b6f17db3cdb75f9391f18dc2f
d42cf55113a5891 100644
---
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/TestApplication.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/TestApplication.kt
@@ -1,6 +1,10 @@
package cloud.mws.devp.access.logging.spring.webflux

+import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretRequest
+import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretResponse
+import cloud.mws.devp.cloud.sensitive.cloudSensitive
import
org.springframework.boot.autoconfigure.SpringBootApplication
+import org.springframework.http.MediaType
import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.PathVariable
import org.springframework.web.bind.annotation.PostMapping
@@ -46,6 +50,19 @@ open class TestApplication {
    return Mono.just("Hello, Post!")
}

+    @PostMapping("/echo-raw", consumes =
+[MediaType.TEXT_PLAIN_VALUE])
+    open fun echoRaw(@RequestBody body: String): String {
+        return body
+    }
+
+    @PostMapping("/secret")
+    open fun handleSecret(@RequestBody request:
SecretRequest): SecretResponse {
+        return SecretResponse(
+            message = "OK for ${request.publicField}",
+            maskedToken = "TOKEN".cloudSensitive()
+        )
+    }
+
+    @PostMapping("/withoutBodyEndpoint")
+    open fun withoutBodyEndpoint(@RequestBody body: String):
Mono<String> {
        println(body)
diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/TestUtil.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/TestUtil.kt
index
290fd55abcdcc5800ba5c80d48cea9a81ffd5023..a26a314d3ec90b572a56cb72e

```



```

+import com.fasterxml.jackson.databind.deser.std.StdDeserializer
+import com.fasterxml.jackson.databind.module.SimpleModule
+import com.fasterxml.jackson.databind.ser.std.StdSerializer
+import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
+import com.fasterxml.jackson.module.kotlin.registerKotlinModule
+import com.github.tomakehurst.wiremock.WireMockServer
+import com.github.tomakehurst.wiremock.client.WireMock.aResponse
+import com.github.tomakehurst.wiremock.client.WireMock.post
+import
com.github.tomakehurst.wiremock.core.WireMockConfiguration.options
+import io.kotest.matchers.string.shouldContain
+import io.kotest.matchers.string.shouldNotContain
+import org.assertj.core.api.Assertions.assertThat
+import org.junit.jupiter.api.AfterEach
+import org.junit.jupiter.api.BeforeEach
+import org.junit.jupiter.api.Test
+import org.springframework.http.MediaType
+import
org.springframework.http.client.reactive.ReactorClientHttpConnector
+import org.springframework.http.codec.json.Jackson2JsonDecoder
+import org.springframework.http.codec.json.Jackson2JsonEncoder
+import org.springframework.web.reactive.function.client.WebClient
+import reactor.netty.http.client.HttpClient
+
+class AccessLogClientAutoMaskingTest {
+
+    private val server = WireMockServer(options().dynamicPort())
+
+    private lateinit var accessLogWriter: AccessLogWriter
+    private lateinit var requestProcessingStage:
RequestProcessingStage
+
+    private lateinit var client: WebClient
+    private lateinit var capturedAccessLogs: MutableList<String>
+
+    @BeforeEach
+    fun setup() {
+        server.start()
+        capturedAccessLogs = mutableListOf()
+
+        accessLogWriter = object : AccessLogWriter {
+            override fun write(accessLog: AccessLog) {
+                capturedAccessLogs.add(accessLog.body ?: "")
+            }
+        }
+
+        val sink = LoggerSink(
+            accessLogWriter = accessLogWriter,
+            bodySerializationStrategy =
AutoBodySerializationStrategy()
+        )
+
+        requestProcessingStage = DefaultRequestProcessingStage(
+            strategy = WithBodyStrategy(emptyList()),
+            sink = sink
+        )

```

```

+
+         val objectMapper: ObjectMapper = jacksonObjectMapper()
+             .registerKotlinModule()
+             .registerModule(
+                 SimpleModule().apply {
+                     addSerializer(
+                         object :
+ StdSerializer<CloudSensitive<*>>(CloudSensitive::class.java) {
+                             override fun serialize(
+                                 value: CloudSensitive<*>,
+                                 gen: JsonGenerator,
+                                 serializers: SerializerProvider
+                             ) {
+                                 gen.writeString(value.toString())
+                             }
+                         })
+
+                     addDeserializer(
+                         CloudSensitive::class.java,
+                         object :
+ StdDeserializer<CloudSensitive<*>>(CloudSensitive::class.java) {
+                             override fun deserialize(
+                                 parser: JsonParser,
+                                 context: DeserializationContext
+                             ): CloudSensitive<*> {
+                                 val value = parser.text
+                                 return value.cloudSensitive()
+                             }
+                         })
+                 }
+             )
+
+         client = WebClient.builder()
+
+             .clientConnector(ReactorClientHttpConnector(HttpClient.create()))
+             .baseUrl(server.baseUrl())
+             .codecs { configurer ->
+
+ configurer.defaultCodecs().jackson2JsonDecoder(Jackson2JsonDecoder(
+ objectMapper))
+
+ configurer.defaultCodecs().jackson2JsonEncoder(Jackson2JsonEncoder(
+ objectMapper))
+
+             .filter { request, next ->
+                 val newRequest =
+ org.springframework.web.reactive.function.client.ClientRequest.from
+ (request)
+
+                 .attributes { attrs ->
+                     attrs["requestType"] =
+ SecretRequest::class.java
+                     attrs["responseType"] =
+ SecretResponse::class.java
+                 }
+
+                 .build()
+
+                 next.exchange(newRequest)

```

```

+         }
+
+     .filter(AccessLogExchangeFilterFunction(requestProcessingStage))
+         .build()
+     }
+
+     @AfterEach
+     fun tearDown() {
+         server.stop()
+     }
+
+     @Test
+     fun `AUTO mode masks CloudSensitive in client request and
response`() {
+         server.stubFor(
+             post("/secret")
+                 .willReturn(
+                     aResponse()
+                         .withStatus(200)
+                         .withHeader("Content-Type",
"application/json")
+                 )
+             .withBody("""{"message":"OK","maskedToken":"server-masked"}""")
+         )
+
+         val requestDto = SecretRequest(
+             publicField = "test-public",
+             secretToken = "test-secret".cloudSensitive()
+         )
+
+         client.post()
+             .uri("/secret")
+             .contentType(MediaType.APPLICATION_JSON)
+             .bodyValue(requestDto)
+             .retrieve()
+             .bodyToMono(SecretResponse::class.java)
+             .block()
+
+         assertThat(capturedAccessLogs).hasSize(2)
+
+         val requestLog = capturedAccessLogs[0]
+         val responseLog = capturedAccessLogs[1]
+
+         requestLog.shouldContain("""secretToken":"*****""")
+
+         requestLog.shouldContain("""publicField":"test-public""")
+         responseLog.shouldContain("""maskedToken":"*****""")
+         responseLog.shouldContain("""message":"OK""")
+         requestLog.shouldNotContain("test-secret")
+         responseLog.shouldNotContain("server-masked")
+     }
+ }
+ }
diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogC

```

```
lientRawModeTest.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogClientRawModeTest.kt
new file mode 100644
index
000000000000000000000000000000000000..6eba5dddf3c23bfbd24e20bbc21653a317634d5f
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogClientRawModeTest.kt
@@ -0,0 +1,115 @@
+package
cloud.mws.devp.access.logging.spring.webflux.exchange.filter
+
+import cloud.mws.devp.access.logging.AccessLog
+import cloud.mws.devp.access.logging.AccessLogWriter
+import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilterFunction
+import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySerializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretRequest
+import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretResponse
+import com.github.tomakehurst.wiremock.WireMockServer
+import com.github.tomakehurst.wiremock.client.WireMock.aResponse
+import com.github.tomakehurst.wiremock.client.WireMock.equalToJson
+import com.github.tomakehurst.wiremock.client.WireMock.post
+import
com.github.tomakehurst.wiremock.core.WireMockConfiguration.options
+import io.kotest.matchers.string.shouldContain
+import io.kotest.matchers.string.shouldNotContain
+import org.assertj.core.api.Assertions.assertThat
+import org.junit.jupiter.api.AfterEach
+import org.junit.jupiter.api.BeforeEach
+import org.junit.jupiter.api.Test
+import org.springframework.http.MediaType
+import
org.springframework.http.client.reactive.ReactorClientHttpConnector
+import org.springframework.web.reactive.function.client.WebClient
```



```

+import reactor.netty.http.client.HttpClient
+
+class AccessLogClientRawModeTest {
+
+    private val server = WireMockServer(options().dynamicPort())
+
+    private lateinit var accessLogWriter: AccessLogWriter
+    private lateinit var requestProcessingStage:
RequestProcessingStage
+
+    private lateinit var client: WebClient
+    private lateinit var capturedAccessLogs: MutableList<String>
+
+    @BeforeEach
+    fun setup() {
+        server.start()
+        capturedAccessLogs = mutableListOf()
+
+        accessLogWriter = object : AccessLogWriter {
+            override fun write(accessLog: AccessLog) {
+                capturedAccessLogs.add(accessLog.body ?: "")
+            }
+        }
+
+        val sink = LoggerSink(
+            accessLogWriter = accessLogWriter,
+            bodySerializationStrategy =
RawBodySerializationStrategy()
+        )
+
+        requestProcessingStage = DefaultRequestProcessingStage(
+            strategy = WithBodyStrategy(emptyList()),
+            sink = sink
+        )
+
+        client = WebClient.builder()
+
+.clientConnector(ReactorClientHttpConnector(HttpClient.create()))
+        .baseUrl(server.baseUrl())
+        .filter { request, next ->
+            val newRequest =
org.springframework.web.reactive.function.client.ClientRequest.from
(request)
+                .attributes { attrs ->
+                    attrs["requestType"] =
SecretRequest::class.java
+                    attrs["responseType"] =
SecretResponse::class.java
+                }
+                .build()
+            next.exchange(newRequest)
+        }
+
+.filter(AccessLogExchangeFilterFunction(requestProcessingStage))
+        .build()
+    }

```

```

+
+     @AfterEach
+     fun tearDown() {
+         server.stop()
+     }
+
+     @Test
+     fun `RAW mode should not mask CloudSensitive in client logs`()
+     {
+         val requestJson =
+         """{"publicField":"test","secretToken":"input-secret"}"""
+         val responseJson =
+         """{"message":"OK","maskedToken":"server-masked"}"""
+
+         server.stubFor(
+             post("/secret")
+                 .withRequestBody(equalToJson(requestJson))
+                 .willReturn(
+                     aResponse()
+                         .withStatus(200)
+                         .withHeader("Content-Type",
+ "application/json")
+                         .withBody(responseJson)
+                 )
+         )
+
+         client.post()
+             .uri("/secret")
+             .contentType(MediaType.APPLICATION_JSON)
+             .bodyValue(requestJson)
+             .retrieve()
+             .bodyToMono(String::class.java)
+             .block()
+
+         assertThat(capturedAccessLogs).hasSize(2)
+
+         val requestLog = capturedAccessLogs[0]
+         val responseLog = capturedAccessLogs[1]
+
+         requestLog.shouldNotContain("****")
+         requestLog.shouldContain("input-secret")
+         responseLog.shouldNotContain("****")
+         responseLog.shouldContain("server-masked")
+     }
+ }
+ }
diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogE
xchangeFilterFunctionBaseTest.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogE
xchangeFilterFunctionBaseTest.kt
index
d7f9d906bff3730af34acb1686f25a540dcac80b..7ee3ce5e00da88a08f2092806
0242c5df70f86f4 100644
---
```

```
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogExchangeFilterFunctionBaseTest.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogExchangeFilterFunctionBaseTest.kt
@@ -129,7 +129,8 @@ class AccessLogExchangeFilterFunctionBaseTest {
    util.sendAndReceive()
    Mockito.verify(sink, Mockito.never()).write(
        argumentCaptor<PreCorrelation>().capture(),
-       argumentCaptor<HttpRequest>().capture()
+       argumentCaptor<HttpRequest>().capture(),
+       Mockito.isNull()
    )
}

@@ -182,7 +183,8 @@ class AccessLogExchangeFilterFunctionBaseTest {

    Mockito.verify(sink, Mockito.never()).write(
        argumentCaptor<PreCorrelation>().capture(),
-       argumentCaptor<HttpRequest>().capture()
+       argumentCaptor<HttpRequest>().capture(),
+       Mockito.isNull()
    )
}

diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogAutoMaskingTest.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogAutoMaskingTest.kt
new file mode 100644
index
00000000000000000000000000000000..be2e528e7ac387a8edfda20b78dbdf80d1bb86f0
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogAutoMaskingTest.kt
@@ -0,0 +1,94 @@
+package cloud.mws.devp.access.logging.spring.webflux.web.filter
+
+import
cloud.mws.devp.access.logging.spring.webflux.TestApplication
+import org.assertj.core.api.Assertions.assertThat
+import org.junit.jupiter.api.Test
+import org.springframework.boot.test.context.SpringBootTest
+import org.springframework.http.MediaType
+
+@SpringBootTest(
+    classes = [TestApplication::class,
AutoMaskingTestConfig::class],
```

```

+     webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
+     properties = [
+         "devp.api.access-log.body-serialization-mode=AUTO",
+         "devp.api.access-log.strategy=WITH_BODY"
+     ]
+ )
+ class AccessLogAutoMaskingTest : AccessLogBaseTest() {
+     @Test
+     fun `test secret endpoint with full response`() {
+         val jsonRequest = """"{"publicField": "public-value",
+ "secretToken": "super-secret-password"}""".trimIndent()
+
+         val response = client.post()
+             .uri("/secret")
+             .contentType(MediaType.APPLICATION_JSON)
+             .bodyValue(jsonRequest)
+             .exchangeToMono { clientResponse ->
+                 println("Status: ${clientResponse.statusCode()}")
+                 println("Headers:
+ ${clientResponse.headers().asHttpHeaders()}")
+                 clientResponse.bodyToMono(String::class.java)
+                     .doOnNext { body -> println("Body: $body") }
+                     .thenReturn(clientResponse)
+             }
+             .block()
+
+         println("Final response status:
+ ${response?.statusCode()}")
+     }
+
+     @Test
+     fun `AUTO mode should mask CloudSensitive in request and
+ response`() {
+         client.post()
+             .uri("/secret")
+             .contentType(MediaType.APPLICATION_JSON)
+             .bodyValue("""{"publicField": "public-value", "secretToken": "super-se
+ cret-password"}""")
+             .retrieve()
+             .toBodilessEntity()
+             .block()
+
+         val accessLogs = util.captureAccessLogWrite()
+         assertThat(accessLogs).hasSize(2)
+
+         val requestLog = accessLogs[0]
+
+         assertThat(requestLog.body).contains("""{"publicField": "public-valu
+ e", "secretToken": "*****"}""")
+
+         assertThat(requestLog.body).doesNotContain("super-secret-password")
+
+         val responseLog = accessLogs[1]
+         assertThat(responseLog.body).contains("""{"message": "OK

```

```
for public-value","maskedToken":"*****"}")
+         assertThat(responseLog.body).doesNotContain("TOKEN")
+     }
+
+     @Test
+     fun `AUTO mode should not parse non-JSON content`() {
+         client.post()
+             .uri("/echo-raw")
+             .contentType(MediaType.TEXT_PLAIN)
+             .bodyValue("plain text with secret")
+             .retrieve()
+             .bodyToMono(String::class.java)
+             .block()
+
+         val accessLogs = util.captureAccessLogWrite()
+         assertThat(accessLogs).hasSize(2)
+
+         val requestAccessLog = accessLogs[0]
+
+         assertThat(requestAccessLog.body).isEqualTo("[BINARY_CONTENT]")
+     }
+
+     @Test
+     fun `AUTO mode returns JSON_ERROR marker on JSON parsing error`() {
+         client.post()
+             .uri("/echo")
+             .contentType(MediaType.APPLICATION_JSON)
+             .bodyValue("{ invalid json }")
+             .retrieve()
+             .bodyToMono(String::class.java)
+             .block()
+
+         val accessLogs = util.captureAccessLogWrite()
+         assertThat(accessLogs).hasSize(2)
+
+         val requestAccessLog = accessLogs[0]
+
+         assertThat(requestAccessLog.body).isEqualTo("[JSON_ERROR]")
+     }
+ }
+}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogRawModeTest.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogRawModeTest.kt
new file mode 100644
index
00000000000000000000000000000000..268b4116aa85606426373f5a25de9358e3510bdf
--- /dev/null
+++
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogRawMod
```

```

eTest.kt
@@ -0,0 +1,34 @@
+package cloud.mws.devp.access.logging.spring.webflux.web.filter
+
+import
+cloud.mws.devp.access.logging.spring.webflux.TestApplication
+import org.assertj.core.api.Assertions.assertThat
+import org.junit.jupiter.api.Test
+import org.springframework.boot.test.context.SpringBootTest
+import org.springframework.http.MediaType
+
+@SpringBootTest(
+    classes = [TestApplication::class,
+AutoMaskingTestConfig::class],
+    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
+    properties = [
+        "devp.api.access-log.body-serialization-mode=RAW",
+        "devp.api.access-log.strategy=WITH_BODY"
+    ]
+)
+class AccessLogRawModeTest : AccessLogBaseTest() {
+
+    @Test
+    fun `RAW mode should not mask CloudSensitive`() {
+        client.post()
+            .uri("/secret")
+            .contentType(MediaType.APPLICATION_JSON)
+            .bodyValue("""{"publicField":"public-value","secretToken":"super-se
cret-password"}""")
+            .retrieve()
+            .toBodilessEntity()
+            .block()
+
+        val accessLogs = util.captureAccessLogWrite()
+
+        val requestLog = accessLogs[0]
+
+        assertThat(requestLog.body).contains("super-secret-password")
+    }
+}
diff --git
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/web/filter/AccessLogWithBo
dyStrategyTest.kt
b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/web/filter/AccessLogWithBo
dyStrategyTest.kt
index
453958a439ee6bbdd840075614b5c632fe20d6a2..7bce528d6f03793a270374c4f
9242caaae2afc4a 100644
---
a/libraries/devp-access-logging-spring-webflux/src/test/kotlin/clou
d/mws/devp/access/logging/spring/webflux/web/filter/AccessLogWithBo
dyStrategyTest.kt
+++

```



```

b/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AutoMaskingTest
Config.kt
@@ -0,0 +1,103 @@
+package cloud.mws.devp.access.logging.spring.webflux.web.filter
+
+import cloud.mws.devp.access.logging.AccessLogWriter
+import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
+import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
+import cloud.mws.devp.access.logging.spring.webflux.common.Sink
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySerializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySerializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
+import cloud.mws.devp.cloud.sensitive.CloudSensitive
+import cloud.mws.devp.cloud.sensitive.cloudSensitive
+import com.fasterxml.jackson.core.JsonGenerator
+import com.fasterxml.jackson.core.JsonParser
+import com.fasterxml.jackson.databind.DeserializationContext
+import com.fasterxml.jackson.databind.ObjectMapper
+import com.fasterxml.jackson.databind.SerializerProvider
+import com.fasterxml.jackson.databind.deser.std.StdDeserializer
+import com.fasterxml.jackson.databind.module.SimpleModule
+import com.fasterxml.jackson.databind.ser.std.StdSerializer
+import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
+import com.fasterxml.jackson.module.kotlin.registerKotlinModule
+import
org.springframework.boot.autoconfigure.condition.ConditionalOnProperty
+import org.springframework.boot.test.context.TestConfiguration
+import org.springframework.boot.web.codec.CodecCustomizer
+import org.springframework.context.annotation.Bean
+import org.springframework.core.env.Environment
+import org.springframework.http.MediaType
+import org.springframework.http.codec.json.Jackson2JsonDecoder
+import org.springframework.http.codec.json.Jackson2JsonEncoder
+import
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping
+
+@TestConfiguration
+open class AutoMaskingTestConfig {
+    @Bean

```



```

+     @ConditionalOnProperty(name =
+ ["devp.api.access-log.body-serialization-mode"], havingValue =
+ "AUTO")
+     open fun autoLoggerSink(accessLogWriter: AccessLogWriter) =
+ LoggerSink(
+         accessLogWriter = accessLogWriter,
+         bodySerializationStrategy =
+ AutoBodySerializationStrategy()
+     )
+
+     @Bean
+     @ConditionalOnProperty(name =
+ ["devp.api.access-log.body-serialization-mode"], havingValue =
+ "RAW")
+     open fun rawLoggerSink(accessLogWriter: AccessLogWriter) =
+ LoggerSink(
+         accessLogWriter = accessLogWriter,
+         bodySerializationStrategy = RawBodySerializationStrategy()
+     )
+
+     @Bean
+     open fun requestProcessingStage(sink: Sink):
+ RequestProcessingStage {
+         return DefaultRequestProcessingStage(
+             strategy = WithBodyStrategy(emptyList()),
+             sink = sink
+         )
+     }
+
+     @Bean
+     open fun filter(
+         requestProcessingStage: RequestProcessingStage,
+         requestMappingHandlerMapping:
+ RequestMappingHandlerMapping,
+         environment: Environment
+     ) = AccessLogWebFilter(requestProcessingStage,
+ requestMappingHandlerMapping, environment)
+
+     @Bean
+     open fun rejectingFilter() = RejectingFilter()
+
+     @Bean
+     open fun objectMapper(): ObjectMapper {
+         return jacksonObjectMapper()
+             .registerKotlinModule()
+             .registerModule(createCloudSensitiveModule())
+     }
+
+     @Bean
+     open fun jsonCodecCustomizer(objectMapper: ObjectMapper):
+ CodecCustomizer {
+         return CodecCustomizer { configurer ->
+             configurer.defaultCodecs().jackson2JsonEncoder(
+                 Jackson2JsonEncoder(objectMapper,
+ MediaType.APPLICATION_JSON)
+             )
+         }
+     }

```

```

+         configurer.defaultCodecs().jackson2JsonDecoder(
+             Jackson2JsonDecoder(objectMapper,
+ MediaType.APPLICATION_JSON)
+         )
+     }
+ }
+
+ private fun createCloudSensitiveModule(): SimpleModule {
+     return SimpleModule().apply {
+         addSerializer(object :
+ StdSerializer<CloudSensitive<*>>(CloudSensitive::class.java) {
+             override fun serialize(value: CloudSensitive<*>,
+ gen: JsonGenerator, serializers: SerializerProvider) {
+                 gen.writeString(value.toString())
+             }
+         })
+
+         addDeserializer(CloudSensitive::class.java, object :
+ StdDeserializer<CloudSensitive<*>>(CloudSensitive::class.java) {
+             override fun deserialize(parser: JsonParser,
+ context: DeserializationContext): CloudSensitive<*> {
+                 val value = parser.text
+                 return value.cloudSensitive()
+             }
+         })
+     }
+ }
+ }
+ }
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
api/devp-access-logging-webflux-client-spring-boot-starter.api
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
api/devp-access-logging-webflux-client-spring-boot-starter.api
index
a125a340daa28867f456fc361fe12348d4e9895a..c62948895d1d355e611585274
d24fd02907a43c6 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
api/devp-access-logging-webflux-client-spring-boot-starter.api
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
api/devp-access-logging-webflux-client-spring-boot-starter.api
@@ -8,11 +8,13 @@ public class
cloud/mws/devp/access/logging/spring/webflux/spring/boot/starter/Ac

public class
cloud/mws/devp/access/logging/spring/webflux/spring/boot/starter/Ac
cessLogProperties {
    public fun <init> ()V
+    public final fun getBodySerializationMode
+ ()Lcloud/mws/devp/access/logging/spring/webflux/common/impl/BodySer
+ ializationMode;
    public final fun getClientLogEnabled ()Z
    public final fun getServerLogEnabled ()Z
    public final fun getStrategy ()Ljava/lang/String;
    public final fun getWithHeaders ()Ljava/util/List;

```

```

        public final fun getWithoutBodyEndpoints ()Ljava/util/List;
+       public final fun setBodySerializationMode
(Lcloud/mws/devp/access/logging/spring/webflux/common/impl/BodySeria
alizationMode;)V
        public final fun setClientLogEnabled (Z)V
        public final fun setServerLogEnabled (Z)V
        public final fun setStrategy (Ljava/lang/String;)V
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
index
4866ab2503de7cd9132a474ab2239c1eb0761460..6925b85d030e05ae2fc470ee4
44ddafb72e7e3bc 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
@@ -4,8 +4,11 @@ import
cloud.mws.devp.access.logging.AccessLogWriter
import cloud.mws.devp.access.logging.LoggerAccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilt
erFunction
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessi
ngStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySe
rializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySerial
izationMode
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultReq
uestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySer
ializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodySt
rategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithoutBod
yStrategy
import org.springframework.boot.autoconfigure.AutoConfiguration
@@ -43,7 +46,19 @@ open class AccessLogClientAutoConfiguration {

```

```

        "WITH_BODY" ->
        WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
        else -> WithoutBodyStrategy()
    }
    -    return DefaultRequestProcessingStage(strategy = strategy,
    sink = LoggerSink(accessLogWriter,
    accessLogProperties.withHeaders))
    +
    +    val bodySerializationStrategy = when
    (accessLogProperties.bodySerializationMode) {
    +        BodySerializationMode.RAW ->
    RawBodySerializationStrategy()
    +        BodySerializationMode.AUTO ->
    AutoBodySerializationStrategy()
    +    }
    +
    +    val sink = LoggerSink(
    +        accessLogWriter = accessLogWriter,
    +        headersWhiteList = accessLogProperties.withHeaders,
    +        bodySerializationStrategy = bodySerializationStrategy
    +    )
    +
    +    return DefaultRequestProcessingStage(strategy = strategy,
    sink = sink)
    }

    /**
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
index
b6257d559757d4a0c3859cd6abdb012ea7fab7c..b0bf91153a992fde6bf9888f4
bd7a913b6a05bfc 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
@@ -1,5 +1,6 @@
    package
    cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter

    +import
    cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySerial
    izationMode
    import
    org.springframework.boot.context.properties.ConfigurationProperties

    @ConfigurationProperties(prefix = "devp.api.access-log")

```

```

@@ -30,4 +31,11 @@ open class AccessLogProperties {
    * Список header'ов, которые будут логироваться.
    */
    var withHeaders: List<String> = emptyList()
+
+    /**
+     * Режим сериализации тела запроса/ответа.
+     * RAW - тело логируется как есть
+     * AUTO - маскирует CloudSensitive поля в JSON контенте
+     */
+    var bodySerializationMode: BodySerializationMode =
BodySerializationMode.AUTO
    }
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/resources/META-INF/additional-spring-configuration-metadat
a.json
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/resources/META-INF/additional-spring-configuration-metadat
a.json
index
8c99168b96ecf578a2626985f2dc52397a71d0d2..fe4aa8817ac32c97e592d1abc
76aabcd026229bf 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/resources/META-INF/additional-spring-configuration-metadat
a.json
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/resources/META-INF/additional-spring-configuration-metadat
a.json
@@ -10,6 +10,27 @@
        "value": "WITHOUT_BODY"
    }
    ]
+
+    },
+    {
+        "name": "devp.api.access-log.body-serialization-mode",
+        "values": [
+            {
+                "value": "RAW",
+                "description": "Тело логируется как есть"
+            },
+            {
+                "value": "AUTO",
+                "description": "Маскирует CloudSensitive поля в JSON
контенте"
+            }
+        ]
+    }
+
+    ],
+    "properties": [
+        {
+            "name": "devp.api.access-log.body-serialization-mode",
+            "type":
"cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySeria

```

```

lizationMode",
+     "description": "Режим сериализации тела запроса/ответа",
+     "defaultValue": "AUTO"
    }
  ]
}
diff --git
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
api/devp-access-logging-webflux-server-spring-boot-starter.api
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
api/devp-access-logging-webflux-server-spring-boot-starter.api
index
57b739f524edbf73daf3a7a4aa6f50d886b3640d..597b259898961c8f906046433
b9calad3f47e5f1 100644
---
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
api/devp-access-logging-webflux-server-spring-boot-starter.api
+++
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
api/devp-access-logging-webflux-server-spring-boot-starter.api
@@ -1,5 +1,7 @@
    public class
cloud/mws/devp/access/logging/spring/webflux/spring/boot/starter/AccessLogServerAutoConfiguration {
    public fun <init> ()V
    public fun accessLogWebFilter
(Lcloud/mws/devp/access/logging/spring/webflux/common/RequestProcessingStage;Lorg/springframework/web/reactive/result/method/annotation/RequestMappingHandlerMapping;Lorg/springframework/core/env/Environment;)Lcloud/mws/devp/access/logging/spring/webflux/AccessLogWebFilter;
+   public fun loggerSink
(Lcloud/mws/devp/access/logging/AccessLogWriter;Lcloud/mws/devp/access/logging/spring/webflux/spring/boot/starter/AccessLogProperties;)Lcloud/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink;
+   public fun requestProcessingStage
(Lcloud/mws/devp/access/logging/spring/webflux/common/impl/LoggerSink;Lcloud/mws/devp/access/logging/spring/webflux/spring/boot/starter/AccessLogProperties;)Lcloud/mws/devp/access/logging/spring/webflux/common/RequestProcessingStage;
    }

diff --git
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
index
bfc81fcb6eb6f5fb1aa9c51be1888c31bda6e1ab..676186f17466719ef30244f2fa44ef82edfd2103 100644
---
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring

```

```

/boot/starter/AccessLogServerAutoConfiguration.kt
+++
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
@@ -1,7 +1,15 @@
package
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter

+import cloud.mws.devp.access.logging.AccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessi
ngStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySe
rializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySerial
izationMode
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultReq
uestProcessingStage
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySer
ializationStrategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodySt
rategy
+import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithoutBod
yStrategy
import org.springframework.boot.autoconfigure.AutoConfiguration
import
org.springframework.boot.autoconfigure.condition.ConditionalOnPrope
rty
import
org.springframework.boot.autoconfigure.condition.ConditionalOnWebAp
plication
@@ -33,4 +41,36 @@ open class AccessLogServerAutoConfiguration {
    requestMappingHandlerMapping:
RequestMappingHandlerMapping,
    environment: Environment,
    ) = AccessLogWebFilter(requestProcessingStage,
requestMappingHandlerMapping, environment)
+
+    /**
+     * Бин для построения процесса обработки request'а и записи
его в лог.
+     */
+    @Bean
+    open fun requestProcessingStage(
+        loggerSink: LoggerSink,

```

```

+         accessLogProperties: AccessLogProperties
+     ): RequestProcessingStage {
+         val strategy = when (accessLogProperties.strategy) {
+             "WITHOUT_BODY" -> WithoutBodyStrategy()
+             "WITH_BODY" ->
+ WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
+             else -> WithoutBodyStrategy()
+         }
+         return DefaultRequestProcessingStage(strategy = strategy,
+ sink = loggerSink)
+     }
+
+     @Bean
+     open fun loggerSink(
+         accessLogWriter: AccessLogWriter,
+         accessLogProperties: AccessLogProperties
+     ): LoggerSink {
+         val bodySerializationStrategy = when
+ (accessLogProperties.bodySerializationMode) {
+             BodySerializationMode.RAW ->
+ RawBodySerializationStrategy()
+             BodySerializationMode.AUTO ->
+ AutoBodySerializationStrategy()
+         }
+         return LoggerSink(
+             accessLogWriter = accessLogWriter,
+             headersWhiteList = accessLogProperties.withHeaders,
+             bodySerializationStrategy = bodySerializationStrategy
+         )
+     }
+ }

```

Листинг Б.3 – Изменения, влитые по задаче MWSDEVP-5799

```
diff --git a/.changes/next/MWSDEVP-4267.yaml
b/.changes/next/MWSDEVP-4267.yaml
new file mode 100644
index
0000000000000000000000000000000000..353d736ba3e3f4bfd7fd0dba9
1b8a44934553de7
--- /dev/null
+++ b/.changes/next/MWSDEVP-4267.yaml
@@ -0,0 +1,2 @@
+Feature:
+  - "Для access логов по умолчанию включена запись body. Для
отключения необходимо явно устанавливать
`devp.api.access-log.strategy: WITHOUT_BODY`"
diff --git a/docs/site/user/pages/reference/obs/access-logging.md
b/docs/site/user/pages/reference/obs/access-logging.md
index
a004b1ab48a741ea80bf2d76313dd2408228a9e1..395aa5c12a221f7a67884c641
701ec78eafc15c3 100644
--- a/docs/site/user/pages/reference/obs/access-logging.md
```



```

+++ b/docs/site/user/pages/reference/obs/access-logging.md
@@ -48,7 +48,7 @@ devp:
    access-log:
        server-log-enabled: true # По умолчанию логирование запросов
к серверу включено
        client-log-enabled: true # По умолчанию логирование запросов
клиентов включено
-        strategy: WITHOUT_BODY # По умолчанию логирование body
выключено
+        strategy: WITH_BODY # По умолчанию логирование body включено
        body-serialization-mode: AUTO # Режим сериализации тела:
AUTO (по умолчанию) или RAW
        without-body-endpoints: # Список endpoints, в формате regex,
для которых HTTP body не логируется,
                                # по умолчанию содержит /metrics,
/customer-metrics
diff --git
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerial
izationStrategy.kt
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerial
izationStrategy.kt
index
d12df6f11008ae7a0d898170ce16662a5fe4772c..0899502e8ba7398872d8ef342
eeac17418a39ff5 100644
---
a/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerial
izationStrategy.kt
+++
b/libraries/devp-access-logging-spring-webflux/src/main/kotlin/clou
d/mws/devp/access/logging/spring/webflux/common/impl/AutoBodySerial
izationStrategy.kt
@@ -38,11 +38,13 @@ class AutoBodySerializationStrategy :
BodySerializationStrategy {
    )

    override fun serializeBody(body: String, bodyClass: Class<*>?,
contentType: String?): String {
+        if (body.isBlank()) return body
+
        if (!ContentType.isJsonMediaType(contentType)) {
            return binaryContentMarker
        }

-        if (body.isBlank() || bodyClass == null) return body
+        if (bodyClass == null) return body

        return try {
            val obj = secureMapper.readValue(body, bodyClass)
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/

```

```

src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
index
6925b85d030e05ae2fc470ee444ddafb72e7e3bc..2d741b5b34dfb944ad04d5bdc
2ef4d2a364fd6b8 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogClientAutoConfiguration.kt
@@ -35,6 +35,7 @@ open class AccessLogClientAutoConfiguration {

    /**
     * Бин для построения процесса обработки request'а и записи
его в лог.
+    * По умолчанию access логи записываются вместе с body
    */
    @Bean("devp.RequestProcessingStage")
    open fun requestProcessingStage(
@@ -44,7 +45,7 @@ open class AccessLogClientAutoConfiguration {
        val strategy = when (accessLogProperties.strategy) {
            "WITHOUT_BODY" -> WithoutBodyStrategy()
            "WITH_BODY" ->
WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
-            else -> WithoutBodyStrategy()
+            else ->
WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
        }

        val bodySerializationStrategy = when
(accessLogProperties.bodySerializationMode) {
diff --git
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
index
b0bf91153a992fde6bf9888f4bd7a913b6a05bfc..aaf322958811495395462a49a
15dbc5e51365b7f 100644
---
a/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
+++
b/libraries/devp-access-logging-webflux-client-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogProperties.kt
@@ -7,7 +7,7 @@ import
org.springframework.boot.context.properties.ConfigurationProperties
open class AccessLogProperties {
    /**

```

```

        * Выбранная стратегия записи http request и response в access
log.
-      * Возможно 2 варианта: WITH_BODY, WITHOUT_BODY.
+      * Возможно 2 варианта: WITH_BODY (по умолчанию),
WITHOUT_BODY.
        */
        var strategy: String? = null

diff --git
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
index
676186f17466719ef30244f2fa44ef82edfd2103..9e7690b5cee4c5a54959752fd
b6f4d17659d75de 100644
---
a/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
+++
b/libraries/devp-access-logging-webflux-server-spring-boot-starter/
src/main/kotlin/cloud/mws/devp/access/logging/spring/webflux/spring
/boot/starter/AccessLogServerAutoConfiguration.kt
@@ -44,6 +44,7 @@ open class AccessLogServerAutoConfiguration {

    /**
     * Бин для построения процесса обработки request'a и записи
его в лог.
+    * По умолчанию access логи записываются вместе с body
    */
    @Bean
    open fun requestProcessingStage(
@@ -53,7 +54,7 @@ open class AccessLogServerAutoConfiguration {
        val strategy = when (accessLogProperties.strategy) {
            "WITHOUT_BODY" -> WithoutBodyStrategy()
            "WITH_BODY" ->
WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
-            else -> WithoutBodyStrategy()
+            else ->
WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
        }
        return DefaultRequestProcessingStage(strategy = strategy,
sink = loggerSink)
    }
diff --git
a/tests/consumer/test-cpl/app/src/test/kotlin/cloud/mws/cpl/FailedF
lowDataLoadTest.kt
b/tests/consumer/test-cpl/app/src/test/kotlin/cloud/mws/cpl/FailedF
lowDataLoadTest.kt
index
9e8cbc6a899bec90c34a70194ee7134546bccc20..e09b1a47d9adec07234cef787
e0ec4e65add47a7 100644
---

```

```

a/tests/consumer/test-cpl/app/src/test/kotlin/cloud/mws/cpl/FailedFlowDataLoadTest.kt
+++
b/tests/consumer/test-cpl/app/src/test/kotlin/cloud/mws/cpl/FailedFlowDataLoadTest.kt
@@ -3,7 +3,7 @@ package cloud.mws.cpl
import cloud.mws.cpl.api.CplApiConfiguration
import cloud.mws.cpl.repository.CplRepositoryConfiguration
import cloud.mws.cpl.service.CplServiceConfiguration
- import cloud.mws.devp.cloud.client.ResponseParseClientException
+ import cloud.mws.devp.cloud.client.ClientException
import cloud.mws.devp.cloud.client.impl.BaseCloudApiClient
import
cloud.mws.devp.cloud.client.impl.CloudApiWebClientConfiguration
import cloud.mws.devp.devenv.ExternalComponent.POSTGRESQL
@@ -57,13 +57,13 @@ class FailedFlowDataLoadTest :
ExternalComponentsTest() {
    val cloudApiConfiguration =
CloudApiWebClientConfiguration(webClientBuilder, mapper)
    val webClient =
cloudApiConfiguration.createWebClient("http://localhost:$randomServerPort")
    val testApiClientImpl = TestApiClientImpl(webClient,
mapper)
-    val answer = assertThrows<ResponseParseClientException> {
+    val answer = assertThrows<ClientException> {
        runBlocking {
            testApiClientImpl.getFlow().body
        }
    }
    answer.shouldNotBeNull()
-    answer.message!!.shouldBeEqual("Connection prematurely
closed DURING response")
+    answer.message!!.shouldBeEqual("Internal server error")
}

@@ -73,7 +73,7 @@ class TestApiClientImpl(
) : BaseCloudApiClient(webClient, mapper) {
    suspend fun getFlow(): ResponseEntity<List<String>> = request(
        method = GET,
-        responseMediaType = "text/plain",
+        responseMediaType = "application/json",
        headerList = mapOf(),
        path = "/splitResponse",
        pathList = mapOf(),

```

Листинг Б.4 – Изменения, влитые по задаче MWSDEVP-4267



Рисунок Б.5 – Архитектурная схема первой реализации

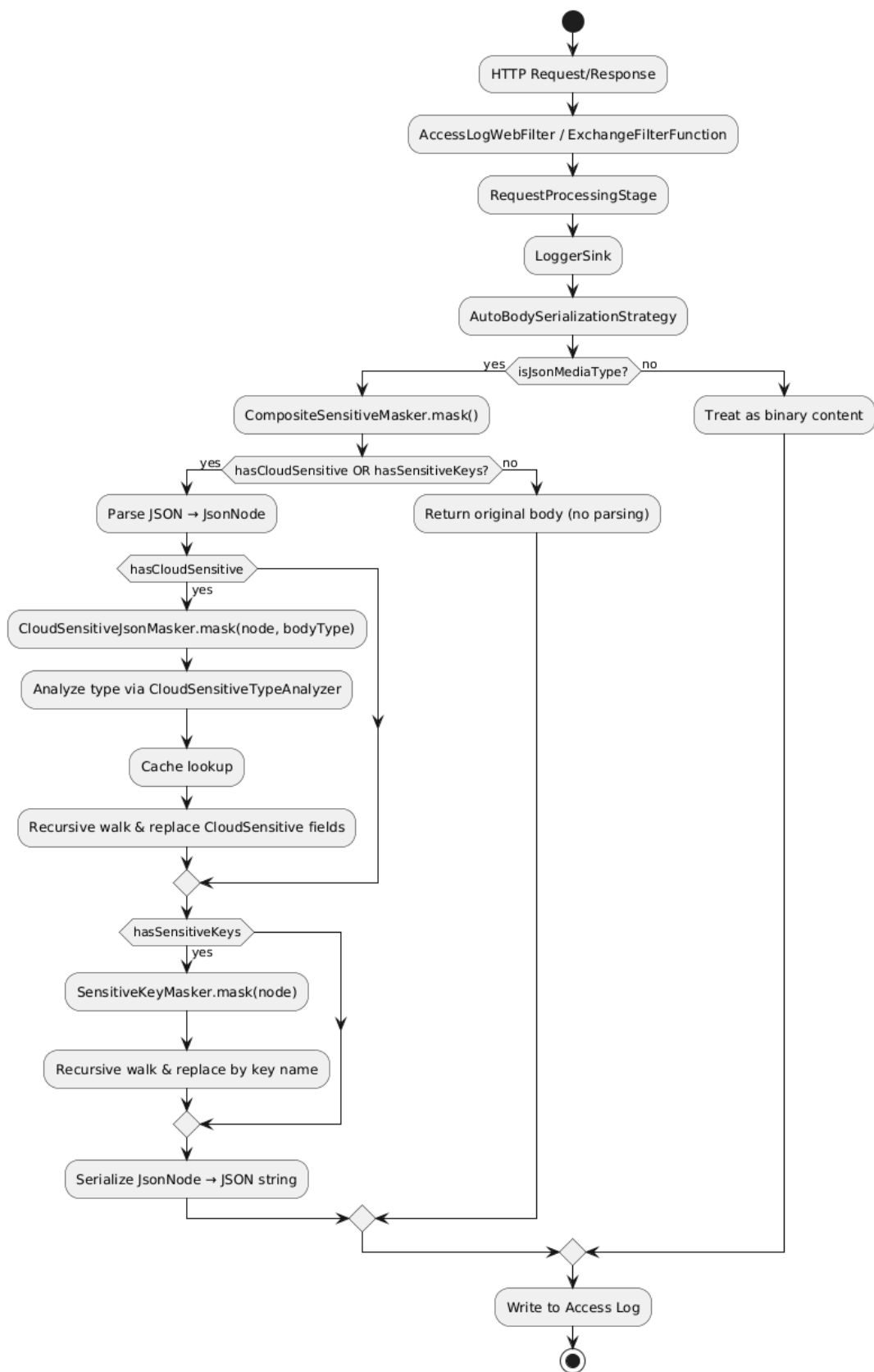


Рисунок Б.6 – Схема второй реализации

ПРИЛОЖЕНИЕ В

```
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import cloud.mws.devp.base.usage.DevpInternal
import cloud.mws.devp.cloud.optional.CloudOptional
import cloud.mws.devp.cloud.sensitive.CloudSensitive
import com.fasterxml.jackson.annotation.JsonProperty
import org.reactivestreams.Publisher
import org.springframework.core.ResolvableType
import org.springframework.http.ResponseEntity
import reactor.core.publisher.Flux
import reactor.core.publisher.Mono
import java.util.LinkedList
import java.util.concurrent.ConcurrentHashMap
import kotlin.reflect.KClass
import kotlin.reflect.KProperty1
import kotlin.reflect.KType
import kotlin.reflect.full.createType
import kotlin.reflect.full.isSubclassOf
import kotlin.reflect.full.memberProperties
import kotlin.reflect.jvm.internal.KotlinReflectionInternalError
import kotlin.reflect.jvm.jvmErasure

@DevpInternal
class CloudSensitiveTypeAnalyzer {

    private val metadataCache = ConcurrentHashMap<KClass<*>,
ClassMetadata>()

    private val cloudSensitiveClass = CloudSensitive::class
    private val wrapperTypes: Set<KClass<*>> = setOf(
        CloudOptional::class,
        Mono::class,
        Publisher::class,
        ResponseEntity::class
    )

    @DevpInternal
    data class ClassMetadata(
        val cloudSensitiveFields: Set<String> = emptySet(),
        val nestedObjectFields: Map<String, KType> = emptyMap(),
        val collectionFields: Map<String, KType> = emptyMap(),
        val mapFields: Map<String, KType> = emptyMap(),
        val mapsWithSensitiveKeys: Set<String> = emptySet()
    ) {
        fun definitelyHasCloudSensitive(): Boolean {
            return cloudSensitiveFields.isNotEmpty() ||
mapsWithSensitiveKeys.isNotEmpty()
        }
    }

    fun extractMetadata(clazz: KClass<*>): ClassMetadata {
```

```

        return metadataCache.getOrPut(clazz) {
            analyzeClassMetadata(clazz)
        }
    }

    fun mayContainSensitiveFields(type: ResolvableType): Boolean {
        checkResolvableType(type)

        if (type.hasGenerics()) {
            val genericResults = LinkedList<Boolean>()
            type.generics.forEach { arg ->
                if (arg != ResolvableType.NONE) {
genericResults.add(mayContainSensitiveFields(arg))
                }
            }
            return isCloudSensitiveType(type) ||
genericResults.contains(true)
        }

        if (type.rawClass!!.isArray) {
            val componentType = type.componentType
            if (componentType != ResolvableType.NONE) {
                return isCloudSensitiveType(type) ||
mayContainSensitiveFields(componentType)
            }
        }

        val rawResult = mayContainSensitiveFields(type.rawClass!!)
        return rawResult || isCloudSensitiveType(type)
    }

    private fun mayContainSensitiveFields(bodyClass: Class<*>):
Boolean {
        val type = extractKTypeFromClass(bodyClass)
        return mayContainSensitiveFields(type)
    }

    private fun analyzeClassMetadata(clazz: KClass<*>):
ClassMetadata {
        return try {
            val cloudSensitiveFields = mutableSetOf<String>()
            val nestedObjectFields = mutableMapOf<String, KType>()
            val collectionFields = mutableMapOf<String, KType>()
            val mapFields = mutableMapOf<String, KType>()
            val mapsWithSensitiveKeys = mutableSetOf<String>()

            clazz.memberProperties.forEach { property ->
                try {
                    val fieldName = getJsonFieldName(property)
                    var propertyType = property.returnType

                    while (isWrapperType(propertyType.jvmErasure))
{
                        val unwrapped =
extractWrappedType(propertyType)

```



```

        if (unwrapped != null) {
            propertyType = unwrapped
        } else {
            break
        }
    }

    when {
        isCloudSensitiveType(propertyType) ->
            cloudSensitiveFields.add(fieldName)

        isCollectionType(propertyType) ->
            processCollectionField(fieldName,
propertyType, collectionFields)

        isMapType(propertyType) ->
            processMapField(fieldName,
propertyType, mapFields, mapsWithSensitiveKeys)

        canProcessAsObject(propertyType.jvmErasure)
->
            nestedObjectFields[fieldName] =
propertyType
    }
    } catch (_: Exception) {
        // Ignore problematic properties
    }
}

ClassMetadata(
    cloudSensitiveFields,
    nestedObjectFields,
    collectionFields,
    mapFields,
    mapsWithSensitiveKeys
)
} catch (_: Exception) {
    ClassMetadata()
}
}

private fun processCollectionField(
    fieldName: String,
    propertyType: KType,
    collectionFields: MutableMap<String, KType>
) {
    val elementType =
extractElementTypeFromCollection(propertyType) ?: return

    when {
        isCloudSensitiveType(elementType) ||
            canProcessAsObject(elementType.jvmErasure) ||
        isCollectionType(elementType) ->
            collectionFields[fieldName] = elementType
    }
}

```

```

private fun processMapField(
    fieldName: String,
    propertyType: KType,
    mapFields: MutableMap<String, KType>,
    mapsWithSensitiveKeys: MutableSet<String>
) {
    val keyType = extractKeyTypeFromMap(propertyType)
    val valueType = extractValueTypeFromMap(propertyType)

    if (keyType != null && isCloudSensitiveType(keyType)) {
        mapsWithSensitiveKeys += fieldName
    }

    if (valueType != null) {
        when {
            isCloudSensitiveType(valueType) ||
            canProcessAsObject(valueType.jvmErasure) ||
            isCollectionType(valueType) ->
            mapFields[fieldName] = valueType
        }
    }
}

private fun mayContainSensitiveFields(type: KType): Boolean {
    val metadata = extractMetadata(type.jvmErasure)

    if (metadata.definitelyHasCloudSensitive()) {
        return true
    }

    val visiting = mutableSetOf<KClass<*>>()

    return metadata.nestedObjectFields.values.any {
        it.containsCloudSensitive(visiting) } ||
        metadata.collectionFields.values.any {
            it.containsCloudSensitive(visiting) } ||
        metadata.mapFields.values.any {
            it.containsCloudSensitive(visiting) }
    }

    private fun KType.containsCloudSensitive(visiting:
MutableSet<KClass<*>>): Boolean {
        val erasure = this.jvmErasure

        if (erasure in visiting) {
            // Because if CloudSensitive is indeed present in the
            loop structure,
            // it will be detected before entering the loop (at the
            first nesting level).
            // If it's not present, then false is correct.
            return false
        }

        visiting.add(erasure)
    }

```

```

        try {
            if (isCloudSensitiveType(this)) return true
            if (!canProcessAsObject(erasure)) return false

            if (arguments.any { arg ->
                arg.type?.containsCloudSensitive(visiting) ==
true
                }) {
                return true
            }

            val nestedMetadata = extractMetadata(erasure)
            if (nestedMetadata.definitelyHasCloudSensitive())
return true

                return nestedMetadata.nestedObjectFields.values.any {
it.containsCloudSensitive(visiting) } ||
                    nestedMetadata.collectionFields.values.any {
it.containsCloudSensitive(visiting) } ||
                    nestedMetadata.mapFields.values.any {
it.containsCloudSensitive(visiting) }
                } finally {
                    visiting.remove(erasure)
                }
        }

        fun extractKTypeFromClass(bodyClass: Class<*>): KType {
            return try {
                bodyClass.kotlin.createType()
            } catch (e: IllegalArgumentException) {
                throw UnsupportedTypeForMaskingException(
                    "Unsupported class for masking: ${bodyClass.name}.
" +
                    "Cannot create type from raw generic class
(e.g., List::class, Array::class). " +
                    "Generic type information is erased at runtime.
" +
                    "To mask sensitive data, use a concrete
non-generic class or provide full type reference",
                    e
                )
            } catch (e: KotlinReflectionInternalError) {
                throw UnsupportedTypeForMaskingException(
                    "Unsupported class for masking: ${bodyClass.name}.
" +
                    "Make sure it's a Kotlin class or properly
annotated Java class with retained generic info." +
                    "Cannot create type for an unsupported
classifier",
                    e
                )
            }
        }

        fun isCloudSensitiveType(type: KType): Boolean {
            return type.jvmErasure == cloudSensitiveClass

```

```

}

fun isCloudSensitiveType(type: ResolvableType): Boolean {
    return type.rawClass == cloudSensitiveClass.java
}

fun isCollectionType(type: KType): Boolean {
    return try {
        val erasure = type.jvmErasure
        erasure.isSubclassOf(List::class) ||
        erasure.isSubclassOf(Set::class) ||
        erasure.isSubclassOf(Collection::class) ||
        erasure.isSubclassOf(Array::class) ||
        erasure.isSubclassOf(Flux::class) ||
        erasure.java.isArray
    } catch (_: Exception) {
        false
    }
}

fun isMapType(type: KType): Boolean {
    val erasure = type.jvmErasure
    return erasure.isSubclassOf(Map::class) ||
        erasure == Map::class ||
        erasure == MutableMap::class
}

fun canProcessAsObject(clazz: KClass<*>): Boolean {
    return when {
        isWrapperType(clazz) -> false

        clazz == Any::class ||
        clazz == Unit::class ||
        clazz == Nothing::class ||
        clazz.javaPrimitiveType != null ||
        clazz == String::class ||
        clazz == ByteArray::class ||
        clazz == Boolean::class ||
        clazz == Int::class ||
        clazz == Long::class ||
        clazz == Double::class ||
        clazz == Float::class ||
        clazz == Short::class ||
        clazz == Char::class ||
        clazz == Byte::class ||
        clazz.isSubclassOf(Enum::class) ||
        clazz == cloudSensitiveClass -> false

        else -> true
    }
}

fun checkResolvableType(type: ResolvableType) {
    if (type.rawClass == null) {
        throw UnsupportedTypeForMaskingException(
            "Cannot analyze type $type with null rawClass:

```

```

    ${type.type}. " +
        "This usually happens when generic type
information is incomplete or unresolved. " +
        "Ensure that the provided ResolvableType is
fully specified " +
        "(e.g., not a raw TypeVariable or unbounded
wildcard)"
    )
}

fun isWrapperType(clazz: KClass<*>): Boolean {
    return clazz in wrapperTypes
}

fun isWrapperType(type: ResolvableType): Boolean {
    return type.rawClass in wrapperTypes.map { it.java }
}

private fun getJsonFieldName(property: KProperty1<*, *>):
String {
    property.getter.annotations.find { it is JsonProperty
}?.let { ann ->
        (ann as JsonProperty).value.takeIf { it.isNotEmpty()
}?.let { return it }
    }
    property.annotations.find { it is JsonProperty }?.let { ann
->
        (ann as JsonProperty).value.takeIf { it.isNotEmpty()
}?.let { return it }
    }
    return property.name
}

private fun extractWrappedType(type: KType): KType? {
    return type.arguments.getOrNull(0)?.type
}

private fun extractElementTypeFromCollection(type: KType):
KType? {
    return type.arguments.getOrNull(0)?.type
}

private fun extractKeyTypeFromMap(type: KType): KType? {
    return type.arguments.getOrNull(0)?.type
}

private fun extractValueTypeFromMap(type: KType): KType? {
    return type.arguments.getOrNull(1)?.type
}
}

```

Листинг В.1 – Исходный код CloudSensitiveTypeAnalyzer

```

...
@Test
fun `mayContainSensitiveFields should correctly detect presence
of CloudSensitive in all test structures`() {

mayContainSensitiveFieldsWithoutGenerics (SimpleRequest::class.java)
.shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NestedRequest::class.java)
.shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (Credentials::class.java).s
houldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (CollectionRequest::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NullableFieldsRequest::cla
ss.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MixedCollectionsRequest::c
lass.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NullableCollectionContaine
r::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MapRequest::class.java).sh
ouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MapWithNonStringKeys::clas
s.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithArrays::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NestedCollectionsRequest::
class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (DeepNestingRequest::class.
java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level1::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level2::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level3::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (InheritedFromBase::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithOptionals::clas

```

```

s.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(RequestWithRecursiveReference::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(RequestWithJsonProperty::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(RequestWithDifferentSensitiveTypes::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(RequestWithEnum::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(ProfileRequest::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(ListOfMapsRequest::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(MapOfListsRequest::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(DoubleWrappedRequest::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(RequestWithMultipleOptions::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics(NightmareRequest::class.java).shouldBeTrue()

    data class PurePublicRequest(
        val name: String,
        val age: Int,
        val tags: List<String>,
        val metadata: Map<String, String>
    )

mayContainSensitiveFieldsWithoutGenerics(PurePublicRequest::class.java).shouldBeFalse()

    data class NestedPublic(val inner: PurePublicRequest)

mayContainSensitiveFieldsWithoutGenerics(NestedPublic::class.java).shouldBeFalse()

    data class CollectionOfPublic(val items: List<PurePublicRequest>)

mayContainSensitiveFieldsWithoutGenerics(CollectionOfPublic::class.java).shouldBeFalse()

    data class MapOfPublic(val map: Map<String, PurePublicRequest>)

```

```

mayContainSensitiveFieldsWithoutGenerics (MapOfPublic::class.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (String::class.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (Int::class.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (Boolean::class.java).shouldBeFalse()

        assertThrows<UnsupportedTypeForMaskingException> {
mayContainSensitiveFieldsWithoutGenerics (List::class.java)
mayContainSensitiveFieldsWithoutGenerics (Map::class.java)
        }

        data class PublicOnlyOptional(val data:
CloudOptional<String>)

mayContainSensitiveFieldsWithoutGenerics (PublicOnlyOptional::class.java).shouldBeFalse()

        data class NestedPublicOptional(val wrapper:
CloudOptional<SimpleRequest>)

mayContainSensitiveFieldsWithoutGenerics (NestedPublicOptional::class.java).shouldBeTrue()

        data class RequestWithOnlyEnum(val status: Status)

mayContainSensitiveFieldsWithoutGenerics (RequestWithOnlyEnum::class.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (Any::class.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (Unit::class.java).shouldBeFalse()

        class EmptyClass

mayContainSensitiveFieldsWithoutGenerics (EmptyClass::class.java).shouldBeFalse()

        data class OnlyNullableStrings(val a: String?, val b:
String?)

mayContainSensitiveFieldsWithoutGenerics (OnlyNullableStrings::class.java).shouldBeFalse()

```



```

        data class MixedNested(
            val public: PurePublicRequest,
            val secret: CloudSensitive<String>
        )

mayContainSensitiveFieldsWithoutGenerics(MixedNested::class.java).shouldBeTrue()

        data class CyclicPublic(val next: CyclicPublic?)

mayContainSensitiveFieldsWithoutGenerics(CyclicPublic::class.java).shouldBeFalse()

        data class CyclicSensitive(val next: CyclicSensitive?, val secret: CloudSensitive<String>)

mayContainSensitiveFieldsWithoutGenerics(CyclicSensitive::class.java).shouldBeTrue()
    }

    private fun mayContainSensitiveFieldsWithoutGenerics(clazz: Class<*>): Boolean {
        return
        CloudSensitiveTypeAnalyzer().mayContainSensitiveFields(ResolvableType.forRawClass(clazz))
    }
    ...

    @Test
    fun `should cache class metadata efficiently`() {
        val analyzer = CloudSensitiveTypeAnalyzer()
        val metadata1 =
        analyzer.extractMetadata(UserRequest::class)
        metadata1.definitelyHasCloudSensitive().shouldBeTrue()
        val metadata2 =
        analyzer.extractMetadata(UserRequest::class)
        (metadata1 === metadata2).shouldBeTrue()
        val simpleMetadata =
        analyzer.extractMetadata(SimpleRequest::class)
        val nestedMetadata =
        analyzer.extractMetadata(NestedRequest::class)
        simpleMetadata.definitelyHasCloudSensitive().shouldBeTrue()

        nestedMetadata.definitelyHasCloudSensitive().shouldBeFalse()

        analyzer.mayContainSensitiveFields(ResolvableType.forClass(NestedRequest::class.java)).shouldBeTrue()
        data class PublicData(val name: String, val age: Int)
        val publicMetadata1 =
        analyzer.extractMetadata(PublicData::class)
        val publicMetadata2 =
        analyzer.extractMetadata(PublicData::class)
        (publicMetadata1 === publicMetadata2).shouldBeTrue()

        publicMetadata1.definitelyHasCloudSensitive().shouldBeFalse()
    }

```



Листинг В.2 – Набор юнит-тестов, демонстрирующих работу анализатора на различных типах данных

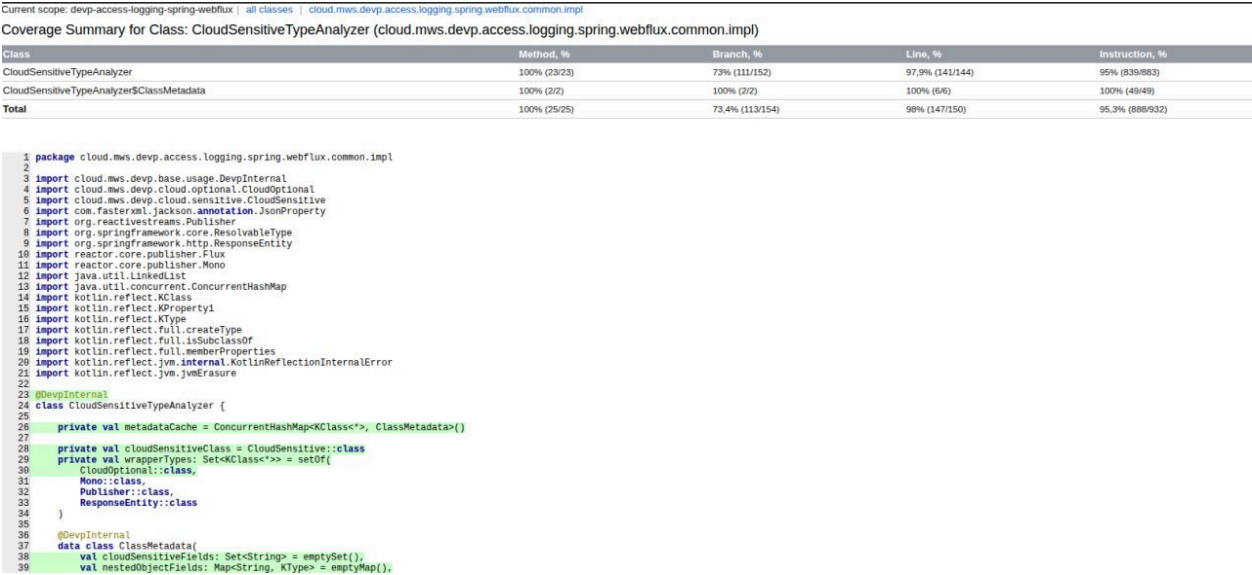


Рисунок В.3 – Отчёт о покрытии тестами

ПРИЛОЖЕНИЕ Г

```
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import
cloud.mws.devp.access.logging.spring.webflux.common.SensitiveValues
import cloud.mws.devp.base.usage.DevpInternal
import com.fasterxml.jackson.databind.JsonNode
import com.fasterxml.jackson.databind.ObjectMapper
import com.fasterxml.jackson.databind.node.ArrayNode
import com.fasterxml.jackson.databind.node.NullNode
import com.fasterxml.jackson.databind.node.ObjectNode
import com.fasterxml.jackson.databind.node.TextNode
import org.reactivestreams.Publisher
import org.springframework.core.ResolvableType
import kotlin.reflect.KType
import kotlin.reflect.jvm.jvmErasure

@DevpInternal
class CloudSensitiveJsonMasker(
    private val objectMapper: ObjectMapper,
    private val typeAnalyzer: CloudSensitiveTypeAnalyzer
) {

    fun mask(node: JsonNode, bodyType: ResolvableType): JsonNode =
        maskJsonNode(node, bodyType)

    private fun maskJsonNode(node: JsonNode, type: ResolvableType):
        JsonNode {
        typeAnalyzer.checkResolvableType(type)

        if (type == ResolvableType.NONE) {
            return node
        }

        if (node.isNull) {
            return NullNode.getInstance()
        }

        if (typeAnalyzer.isCloudSensitiveType(type)) {
            return TextNode.valueOf(SensitiveValues.MASKED_VALUE)
        }

        if (typeAnalyzer.isWrapperType(type)) {
            val wrappedType = type.getGeneric(0)
            return if (wrappedType != ResolvableType.NONE) {
                maskJsonNode(node, wrappedType)
            } else {
                node
            }
        }

        if (type.rawClass!!.isArray) {
```

```

        return if (node.isArray) {
            val elementType = type.componentType
            maskArrayNode(node as ArrayNode, elementType)
        } else {
            node
        }
    }

    if (
Collection::class.java.isAssignableFrom(type.rawClass!!) ||
        Array::class.java.isAssignableFrom(type.rawClass!!) ||
        Publisher::class.java.isAssignableFrom(type.rawClass!!)
    ) {
        return if (node.isArray) {
            maskArrayNode(node as ArrayNode,
type.getGeneric(0))
        } else {
            node
        }
    }

    if (Map::class.java.isAssignableFrom(type.rawClass!!)) {
        return if (node.isObject) {
            val keyType = type.getGeneric(0)
            val valueType = type.getGeneric(1)
            maskMapNode(node as ObjectNode, keyType, valueType)
        } else {
            node
        }
    }

    if (node.isObject) {
        val kType =
typeAnalyzer.extractKTypeFromClass(type.rawClass!!)
        return maskObjectNode(node as ObjectNode, kType)
    }

    return node
}

private fun maskObjectNode(objNode: ObjectNode, type: KType):
ObjectNode {
    val clazz = type.jvmErasure

    val metadata = try {
        typeAnalyzer.extractMetadata(clazz)
    } catch (_: Exception) {
        return objNode
    }

    var result: ObjectNode? = null

    fun copyIfNeeded(): ObjectNode {
        if (result == null) {
            result = objectMapper.createObjectNode()

```

```

        objNode.fieldNames().forEach { name ->
            result.set<JsonNode>(name, objNode[name])
        }
    }
    return result
}

metadata.cloudSensitiveFields.forEach { fieldName ->
    if (objNode.has(fieldName)) {
        val node = objNode.get(fieldName)
        if (node != null && !node.isNull) {
            copyIfNeeded().put(fieldName,
SensitiveValues.MASKED_VALUE)
        }
    }
}

processNestedObjects(objNode, metadata, ::copyIfNeeded)
processMaps(objNode, result, metadata, ::copyIfNeeded)
processCollections(objNode, metadata, ::copyIfNeeded)

return result ?: objNode
}

private fun processNestedObjects(
    objNode: ObjectNode,
    metadata: CloudSensitiveTypeAnalyzer.ClassMetadata,
    copyIfNeeded: () -> ObjectNode
) {
    metadata.nestedObjectFields.forEach { (fieldName,
fieldType) ->
        if (objNode.has(fieldName)) {
            val node = objNode[fieldName]
            if (node.isObject) {
                val maskedChild = maskObjectNode(node as
ObjectNode, fieldType)
                if (maskedChild != node) {
                    copyIfNeeded().set<ObjectNode>(fieldName,
maskedChild)
                }
            }
        }
    }
}

private fun processMaps(
    objNode: ObjectNode,
    result: ObjectNode?,
    metadata: CloudSensitiveTypeAnalyzer.ClassMetadata,
    copyIfNeeded: () -> ObjectNode
) {
    metadata.mapFields.forEach { (fieldName, valueType) ->
        if (objNode.has(fieldName)) {
            val fieldValue = objNode[fieldName]
            if (fieldValue.isObject) {
                val mapNode = fieldValue as ObjectNode

```

```

        val maskedMap = objectMapper.createObjectNode()
        var changed = false

        mapNode.fieldNames().forEach { key ->
            val originalValue = mapNode[key]
            val maskedValue =
maskMapValue(originalValue, valueType)
            maskedMap.set<JsonNode>(key, maskedValue)
            if (maskedValue != originalValue) {
                changed = true
            }
        }

        if (changed) {
            copyIfNeeded().set<ObjectNode>(fieldName,
maskedMap)
        }
    }
}

metadata.mapsWithSensitiveKeys.forEach { fieldName ->
    if (objNode.has(fieldName)) {
        val sourceNode = if (result != null &&
result.has(fieldName)) {
            result[fieldName]
        } else {
            objNode[fieldName]
        }

        if (sourceNode.isObject) {
            val mapNode = sourceNode as ObjectNode
            val maskedMap = objectMapper.createObjectNode()
            var idx = 0
            mapNode.fieldNames().forEach { key ->

maskedMap.set<JsonNode>("${SensitiveValues.MASKED_VALUE}_$idx",
mapNode.get(key))
                idx++
            }
            copyIfNeeded().set<ObjectNode>(fieldName,
maskedMap)
        }
    }
}

private fun maskMapNode(mapNode: ObjectNode, keyType:
ResolvableType, valueType: ResolvableType): ObjectNode {
    var result: ObjectNode? = null
    var changed = false

    val maskKeys = keyType != ResolvableType.NONE &&
typeAnalyzer.isCloudSensitiveType(keyType)

    mapNode.fieldNames().forEach { key ->

```

```

        val originalValue = mapNode[key]
        val maskedValue = if (valueType != ResolvableType.NONE)
    {
        maskJsonNode(originalValue, valueType)
    } else {
        originalValue
    }

        val finalKey = if (maskKeys)
"$${SensitiveValues.MASKED_VALUE}_${result?.size() ?: 0}" else key

        if (!changed && (maskedValue != originalValue ||
maskKeys)) {
            changed = true
            result = objectMapper.createObjectNode()
            for (prevKey in mapNode.fieldNames()) {
                if (prevKey == key) break
                val prevVal = mapNode[prevKey]

                val maskedPrev = if (valueType !=
ResolvableType.NONE) {
                    maskJsonNode(prevVal, valueType)
                } else {
                    prevVal
                }

                val outKey = if (maskKeys)
"$${SensitiveValues.MASKED_VALUE}_${result.size()}" else prevKey
                result.set<JsonNode>(outKey, maskedPrev)
            }

            if (changed) {
                result!!.set<JsonNode>(finalKey, maskedValue)
            }

            return if (changed) result!! else mapNode
        }

        private fun processCollections(
            objNode: ObjectNode,
            metadata: CloudSensitiveTypeAnalyzer.ClassMetadata,
            copyIfNeeded: () -> ObjectNode
        ) {
            metadata.collectionFields.forEach { (fieldName,
elementType) ->
                if (objNode.has(fieldName)) {
                    val fieldValue = objNode[fieldName]
                    if (fieldValue.isArray) {
                        val originalArray = fieldValue as ArrayNode
                        val maskedArray = maskArrayNode(originalArray,
elementType)

                        if (maskedArray != originalArray) {
                            copyIfNeeded().set<ArrayNode>(fieldName,
maskedArray)

```

```

    }
    }
}

private fun maskMapValue(value: JsonNode?, valueType: KType):
JsonNode {
    if (value == null || value.isNull) {
        return NullNode.getInstance()
    }

    if (typeAnalyzer.isCloudSensitiveType(valueType)) {
        return TextNode.valueOf(SensitiveValues.MASKED_VALUE)
    }

    if (value.isArray &&
typeAnalyzer.isCollectionType(valueType)) {
        val elementType =
extractElementTypeFromCollection(valueType)
        if (elementType != null) {
            return maskArrayNode(value as ArrayNode,
elementType)
        }
        return value
    }

    if (value.isObject && typeAnalyzer.isMapType(valueType)) {
        val innerValueType =
valueType.arguments.getOrNull(1)?.type
        if (innerValueType != null) {
            val mapNode = value as ObjectNode
            var maskedMap: ObjectNode? = null
            var changed = false

            mapNode.fieldNames().forEach { key ->
                val originalVal = mapNode[key]
                val maskedVal = maskMapValue(originalVal,
innerValueType)

                if (!changed && maskedVal !== originalVal) {
                    changed = true
                    maskedMap = objectMapper.createObjectNode()
                    for (prevKey in mapNode.fieldNames()) {
                        if (prevKey == key) break
                        maskedMap.set<JsonNode>(prevKey,
mapNode[prevKey])
                    }
                }

                if (changed) {
                    maskedMap!!.set<JsonNode>(key, maskedVal)
                }
            }

            return if (changed) maskedMap!! else value
        }
    }
}

```



```

        }
        return value
    }

    if (value.isObject &&
typeAnalyzer.canProcessAsObject(valueType.jvmErasure)) {
        return maskObjectNode(value as ObjectNode, valueType)
    }

    return value
}

private fun maskArrayNode(arrayNode: ArrayNode, elementType:
ResolvableType): ArrayNode {
    if (elementType == ResolvableType.NONE) return arrayNode

    var result: ArrayNode? = null
    var changed = false

    for (i in 0 until arrayNode.size()) {
        val element = arrayNode[i]
        val maskedElement = maskJsonNode(element, elementType)

        if (maskedElement != element) {
            if (result == null) {
                result = objectMapper.createArrayNode()
                for (j in 0 until i) {
                    result.add(arrayNode[j])
                }
            }
            changed = true
        }

        result?.add(maskedElement)
    }

    return if (changed) result!! else arrayNode
}

private fun maskArrayNode(arrayNode: ArrayNode, elementType:
KType): ArrayNode {
    var result: ArrayNode? = null
    var changed = false

    for (i in 0 until arrayNode.size()) {
        val element = arrayNode[i]
        var maskedElement: JsonNode? = null

        try {
            when {
                element.isObject &&
typeAnalyzer.isMapType(elementType) -> {
                    val maskedMap = maskMapValue(element as
ObjectNode, elementType)
                    if (maskedMap != element) {
                        maskedElement = maskedMap
                    }
                }
            }
        }
    }

```

```

        changed = true
    }
}
element.isObject &&
typeAnalyzer.canProcessAsObject(elementType.jvmErasure) -> {
    val maskedObj = maskObjectNode(element as
ObjectNode, elementType)
    if (maskedObj != element) {
        maskedElement = maskedObj
        changed = true
    }
}
element.isArray &&
typeAnalyzer.isCollectionType(elementType) -> {
    val innerElementType =
extractElementTypeFromCollection(elementType)
    if (innerElementType != null) {
        val maskedArr = maskArrayNode(element
as ArrayNode, innerElementType)
        if (maskedArr != element) {
            maskedElement = maskedArr
            changed = true
        }
    }
}
typeAnalyzer.isCloudSensitiveType(elementType)
&& !element.isNull -> {
    maskedElement =
TextNode.valueOf(SensitiveValues.MASKED_VALUE)
    changed = true
}
else -> {
    // Nothing to mask
}
}
} catch (_: Exception) {
    // Keep original
}

if (changed && result == null) {
    result = objectMapper.createArrayNode()
    for (j in 0 until i) {
        result.add(arrayNode[j])
    }
}

result?.add(maskedElement ?: element)
}

return if (changed) result!! else arrayNode
}

private fun extractElementTypeFromCollection(type: KType):
KType? {
    return type.arguments.getOrNull(0)?.type
}

```

```
}
```

Листинг Г.1 – Исходный код модуля маскирования

```
=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/impl/CloudSensitiveMaskerResolvableTypeTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import cloud.mws.devp.cloud.optional.CloudOptional
import cloud.mws.devp.cloud.sensitive.CloudSensitive
import com.fasterxml.jackson.databind.ObjectMapper
import io.kotest.matchers.booleans.shouldBeFalse
import io.kotest.matchers.booleans.shouldBeTrue
import io.kotest.matchers.equals.shouldBeEqual
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.junit.jupiter.api.Test
import org.junit.jupiter.api.assertThrows
import org.springframework.core.ResolvableType
import org.springframework.http.ResponseEntity
import reactor.core.publisher.Flux
import reactor.core.publisher.Mono

@Suppress("detekt:StringTemplateIndent")
class CloudSensitiveMaskerResolvableTypeTest {

    private val objectMapper = ObjectMapper()
    private val typeAnalyzer = CloudSensitiveTypeAnalyzer()
    private val masker = CompositeSensitiveMasker(objectMapper, SensitiveKeyMasker(objectMapper))

    data class SimpleDto(
        val id: Long,
        val name: String,
        val secret: CloudSensitive<String>
    )

    data class PublicDto(
        val id: Long,
        val name: String,
        val description: String
    )

    @Test
    fun `should handle invalid ResolvableType with null rawClass`() {
        val invalidType =
            ResolvableType.forType(Object::class.java.genericSuperclass)

        val json = """"{"test": "value"}""""

        assertThrows<UnsupportedTypeForMaskingException> {

```

```

        masker.mask(json, invalidType)
    }
}

@Test
fun `should handle list without generic type`() {
    val invalidType =
ResolvableType.forRawClass(List::class.java)

    val json = ""[]""

    assertThrows<UnsupportedTypeForMaskingException> {
        masker.mask(json, invalidType)
    }
}

@Test
fun `should handle type mismatch between JSON and
ResolvableType`() {
    val json = ""{"id":1,"name":"Test"}""

    val type =
ResolvableType.forClassWithGenerics(List::class.java,
String::class.java)

    val result = masker.mask(json, type)
    result.shouldBeEqual(json)
}

@Test
fun `should handle CloudOptional empty with generic type`() {
    val json = "null"

    val type =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldBeEqual("null")
}

@Test
fun `should handle Mono empty with generic type`() {
    val json = "null"

    val type =
ResolvableType.forClassWithGenerics(Mono::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldBeEqual("null")
}

@Test
fun `should handle bodyClass as Unit`() {
    val json = "null"

```

```

        val type = ResolvableType.forClass(Unit::class.java)
        val result = masker.mask(json, type)

        result.shouldBeEqual("null")
    }

    @Test
    fun `should not mask anything when bodyClass is Unit`() {
        val json = """"{"id":123,"name":"Test
User","secret":"super-secret-value"}""".trimIndent()

        val type = ResolvableType.forClass(Unit::class.java)
        val result = masker.mask(json, type)

        result.shouldBeEqual(json)
        result.shouldContain("super-secret-value")
    }

    @Test
    fun `should mask CloudSensitive in List of DTO`() {
        val json = """"
        [
            {"id": 1, "name": "Alice", "secret":
"alice-secret"},
            {"id": 2, "name": "Bob", "secret": "bob-secret"},
            {"id": 3, "name": "Charlie", "secret":
"charlie-secret"}
        ]
        """.trimIndent()

        val type =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
        val result = masker.mask(json, type)

        result.shouldContain("\"name\": \"Alice\"")
        result.shouldContain("\"name\": \"Bob\"")
        result.shouldContain("\"secret\": \"****\"")
        result.shouldNotContain("alice-secret")
        result.shouldNotContain("bob-secret")
        result.shouldNotContain("charlie-secret")
    }

    @Test
    fun `should mask CloudSensitive in Set of DTO`() {
        val json = """"
        [
            {"id": 1, "name": "Item1", "secret": "secret1"},
            {"id": 2, "name": "Item2", "secret": "secret2"}
        ]
        """.trimIndent()

        val type =
ResolvableType.forClassWithGenerics(Set::class.java,
SimpleDto::class.java)

```

```

        val result = masker.mask(json, type)

        result.shouldContain("\\"name\\":\\"Item1\\"")
        result.shouldContain("\\"name\\":\\"Item2\\"")
        result.shouldContain("\\"secret\\":\\"*****\\"")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
    }

    @Test
    fun `should mask CloudSensitive inside CloudSensitive inside
List`() {
        val json = """"["inner-secret"]""""
        val innerList = ResolvableType.forClassWithGenerics(
            CloudSensitive::class.java,
            String::class.java
        )
        val outerList = ResolvableType.forClassWithGenerics(
            List::class.java,
            innerList
        )
        val type = ResolvableType.forClassWithGenerics(
            CloudSensitive::class.java,
            outerList
        )
        val result = masker.mask(json, type)
        result.shouldBeEqual("\\"*****\\"")
    }

    @Test
    fun `should mask CloudSensitive at root level`() {
        val json1 = "\\"top-secret-token\\"
        val type1 =
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
String::class.java)
        val result1 = masker.mask(json1, type1)
        result1.shouldBeEqual("\\"*****\\"")
        result1.shouldNotContain("top-secret-token")

        val json2 = "42"
        val type2 =
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
Int::class.java)
        val result2 = masker.mask(json2, type2)
        result2.shouldBeEqual("\\"*****\\"")
        result2.shouldNotContain("42")

        val json3 = """"["a", "b", "c"]""""
        val listType =
ResolvableType.forClassWithGenerics(List::class.java,
String::class.java)
        val type3 =
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
listType)
        val result3 = masker.mask(json3, type3)
        result3.shouldBeEqual("\\"*****\\"")
    }

```

```

        result3.shouldNotContain("a")
    }

    @Test
    fun `should mask CloudSensitive in Map with String keys`() {
        val json = """
            {
                "user1": {"id": 1, "name": "Alice", "secret":
"secret1"},
                "user2": {"id": 2, "name": "Bob", "secret":
"secret2"}
            }
        """.trimIndent()

        val type = ResolvableType.forClassWithGenerics(
            Map::class.java,
            String::class.java,
            SimpleDto::class.java
        )

        val result = masker.mask(json, type)

        result.shouldContain("\"user1\":{\"")
        result.shouldContain("\"user2\":{\"")
        result.shouldContain("\"name\": \"Alice\"")
        result.shouldContain("\"name\": \"Bob\"")
        result.shouldContain("\"secret\": \"****\"")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
    }

    @Test
    fun `should mask CloudSensitive in Map with CloudSensitive
keys`() {
        val json = """
            {
                "secretKey1": "value1",
                "secretKey2": "value2"
            }
        """.trimIndent()

        val type = ResolvableType.forClassWithGenerics(
            Map::class.java,
            ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
            String::class.java),
            ResolvableType.forClass(String::class.java)
        )

        val result = masker.mask(json, type)

        result.shouldContain("\"****_0\": \"value1\"")
        result.shouldContain("\"****_1\": \"value2\"")
        result.shouldNotContain("secretKey1")
        result.shouldNotContain("secretKey2")
    }

```

```

@Test
fun `should mask CloudSensitive in array of DTO`() {
    val json = """
        [
            {"id": 1, "name": "Item1", "secret": "secret1"},
            {"id": 2, "name": "Item2", "secret": "secret2"}
        ]
    """.trimIndent()

    val type =
ResolvableType.forArrayComponent(ResolvableType.forClass(SimpleDto:
:class.java))
    val result = masker.mask(json, type)

    result.shouldContain("\\"name\\":\\"Item1\\"")
    result.shouldContain("\\"name\\":\\"Item2\\"")
    result.shouldContain("\\"secret\\":\\"****\\"")
    result.shouldNotContain("secret1")
    result.shouldNotContain("secret2")
}

@Test
fun `should mask CloudSensitive in primitive array`() {
    val json = """["secret1", "secret2", "secret3"]"""

    val type = ResolvableType.forArrayComponent(
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
String::class.java)
    )
    val result = masker.mask(json, type)

    result.shouldContain("[\\"****\\",\\"****\\",\\"****\\"]")
    result.shouldNotContain("secret1")
    result.shouldNotContain("secret2")
}

@Test
fun `should mask CloudSensitive in Mono of DTO`() {
    val json = """{"id": 1, "name": "Alice", "secret":
"alice-secret"}"""

    val type =
ResolvableType.forClassWithGenerics(Mono::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("\\"name\\":\\"Alice\\"")
    result.shouldContain("\\"secret\\":\\"****\\"")
    result.shouldNotContain("alice-secret")
}

@Test
fun `should mask CloudSensitive in Flux of DTO`() {
    val json = """

```



```

        [
            {"id": 1, "name": "Alice", "secret": "secret1"},
            {"id": 2, "name": "Bob", "secret": "secret2"}
        ]
    """.trimIndent()

    val type =
ResolvableType.forClassWithGenerics(Flux::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("\"name\": \"Alice\"")
    result.shouldContain("\"name\": \"Bob\"")
    result.shouldContain("\"secret\": \"****\")
    result.shouldNotContain("secret1")
    result.shouldNotContain("secret2")
}

@Test
fun `should mask CloudSensitive in ResponseEntity of DTO`() {
    val json = """{"id": 1, "name": "Alice", "secret":
"alice-secret"}"""

    val type = ResolvableType.forClassWithGenerics(
        ResponseEntity::class.java,
        SimpleDto::class.java
    )

    val result = masker.mask(json, type)

    result.shouldContain("\"name\": \"Alice\"")
    result.shouldContain("\"secret\": \"****\")
    result.shouldNotContain("alice-secret")
}

@Test
fun `should mask CloudSensitive in ResponseEntity of List`() {
    val json = """
        [
            {"id": 1, "name": "Alice", "secret": "secret1"},
            {"id": 2, "name": "Bob", "secret": "secret2"}
        ]
    """.trimIndent()

    val listType =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
    val type =
ResolvableType.forClassWithGenerics(ResponseEntity::class.java,
listType)

    val result = masker.mask(json, type)

    result.shouldContain("\"name\": \"Alice\"")
    result.shouldContain("\"name\": \"Bob\"")
    result.shouldContain("\"secret\": \"****\")

```

```

        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
    }

    @Test
    fun `should mask CloudSensitive in CloudOptional of DTO`() {
        val json = """{"id": 1, "name": "Alice", "secret":
"alice-secret"}"""

        val type =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
SimpleDto::class.java)
        val result = masker.mask(json, type)

        result.shouldContain("\\"name\\":\\"Alice\\"")
        result.shouldContain("\\"secret\\":\\"****\\"")
        result.shouldNotContain("alice-secret")
    }

    @Test
    fun `should handle CloudOptional with generic List`() {
        val json = """
        [
            {"id": 1, "name": "Alice", "secret": "secret1"},
            {"id": 2, "name": "Bob", "secret": "secret2"}
        ]
        """.trimIndent()

        val listType =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
        val optionalType =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
listType)

        val result = masker.mask(json, optionalType)

        result.shouldContain("\\"name\\":\\"Alice\\"")
        result.shouldContain("\\"name\\":\\"Bob\\"")
        result.shouldContain("\\"secret\\":\\"****\\"")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
    }

    @Test
    fun `should handle nested CloudOptional`() {
        val json = """{"id": 1, "name": "Test", "secret":
"my-secret"}"""

        val innerOptional =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
SimpleDto::class.java)
        val outerOptional =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
innerOptional)

```

```

        val result = masker.mask(json, outerOptional)

        result.shouldContain("\"name\": \"Test\"")
        result.shouldContain("\"secret\": \"****\")
        result.shouldNotContain("my-secret")
    }

    @Test
    fun `should mask CloudSensitive in CloudOptional of List`() {
        val json = """
            [
                {"id": 1, "name": "Alice", "secret": "secret1"},
                {"id": 2, "name": "Bob", "secret": "secret2"}
            ]
        """.trimIndent()

        val listType =
            ResolvableType.forClassWithGenerics(List::class.java,
            SimpleDto::class.java)
        val type =
            ResolvableType.forClassWithGenerics(CloudOptional::class.java,
            listType)

        val result = masker.mask(json, type)

        result.shouldContain("\"name\": \"Alice\"")
        result.shouldContain("\"name\": \"Bob\"")
        result.shouldContain("\"secret\": \"****\")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
    }

    @Test
    fun `should mask CloudSensitive in List of List of DTO`() {
        val json = """
            [
                [
                    {"id": 1, "name": "Alice", "secret":
"secret1"},
                    {"id": 2, "name": "Bob", "secret": "secret2"}
                ],
                [
                    {"id": 3, "name": "Charlie", "secret":
"secret3"}
                ]
            ]
        """.trimIndent()

        val innerType =
            ResolvableType.forClassWithGenerics(List::class.java,
            SimpleDto::class.java)
        val outerType =
            ResolvableType.forClassWithGenerics(List::class.java, innerType)

        val result = masker.mask(json, outerType)
    }

```

```

        result.shouldContain("\\"name\\":\\"Alice\\")
        result.shouldContain("\\"name\\":\\"Bob\\")
        result.shouldContain("\\"name\\":\\"Charlie\\")
        result.shouldContain("\\"secret\\":\\"*****\\")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
        result.shouldNotContain("secret3")
    }

    @Test
    fun `should mask CloudSensitive in Map of List of DTO`() {
        val json = """
            {
                "group1": [
                    {"id": 1, "name": "Alice", "secret":
"secret1"},
                    {"id": 2, "name": "Bob", "secret": "secret2"}
                ],
                "group2": [
                    {"id": 3, "name": "Charlie", "secret":
"secret3"}
                ]
            }
        """.trimIndent()

        val listType =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
        val mapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            ResolvableType.forClass(String::class.java),
            listType
        )

        val result = masker.mask(json, mapType)

        result.shouldContain("\\"group1\\":[")
        result.shouldContain("\\"group2\\":[")
        result.shouldContain("\\"name\\":\\"Alice\\")
        result.shouldContain("\\"name\\":\\"Bob\\")
        result.shouldContain("\\"name\\":\\"Charlie\\")
        result.shouldContain("\\"secret\\":\\"*****\\")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
        result.shouldNotContain("secret3")
    }

    @Test
    fun `should mask CloudSensitive in Flux of List of Map`() {
        val json = """
            [
                [
                    {
                        "user1": {"id": 1, "name": "Alice",
"secret": "secret1"},
                        "user2": {"id": 2, "name": "Bob", "secret":

```

```

"secret2"}
        ],
        [
            {
                "user3": {"id": 3, "name": "Charlie",
"secret": "secret3"}
            }
        ]
    ]
    """.trimIndent()

    val mapType = ResolvableType.forClassWithGenerics(
        Map::class.java,
        String::class.java,
        SimpleDto::class.java
    )
    val listType =
ResolvableType.forClassWithGenerics(List::class.java, mapType)
    val fluxType =
ResolvableType.forClassWithGenerics(Flux::class.java, listType)

    val result = masker.mask(json, fluxType)

    result.shouldContain("\\"user1\\":{")
    result.shouldContain("\\"user2\\":{")
    result.shouldContain("\\"user3\\":{")
    result.shouldContain("\\"name\\":\\"Alice\\"")
    result.shouldContain("\\"name\\":\\"Bob\\"")
    result.shouldContain("\\"name\\":\\"Charlie\\"")
    result.shouldContain("\\"secret\\":\\"*****\\"")
    result.shouldNotContain("secret1")
    result.shouldNotContain("secret2")
    result.shouldNotContain("secret3")
}

@Test
fun `should mask entire List when wrapped in CloudSensitive`() {
    {
        val json = """["item1", "item2", "item3"]"""

        val type = ResolvableType.forClassWithGenerics(
            CloudSensitive::class.java,
            ResolvableType.forClassWithGenerics(List::class.java,
String::class.java)
        )

        val result = masker.mask(json, type)

        result.shouldBeEqual("\\"*****\\"")
    }
}

@Test
fun `should mask entire Map when wrapped in CloudSensitive`() {
    val json = """{"key1": "value1", "key2": "value2"}"""

```

```

        val mapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            String::class.java,
            String::class.java
        )
        val type =
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
mapType)

        val result = masker.mask(json, type)

        result.shouldBeEqual("\*****\")
    }

@Test
fun `should return original JSON for List of public DTO`() {
    val json = """
        [
            {"id": 1, "name": "Alice", "description": "Desc1"},
            {"id": 2, "name": "Bob", "description": "Desc2"}
        ]
    """.replace(" ", "").replace("\n", "")

    val type =
ResolvableType.forClassWithGenerics(List::class.java,
PublicDto::class.java)
    val result = masker.mask(json, type)

    result.shouldBeEqual(json)
}

@Test
fun `should return original JSON for Map of public DTO`() {
    val json = """
        {
            "item1": {"id": 1, "name": "Alice", "description":
"Desc1"},
            "item2": {"id": 2, "name": "Bob", "description":
"Desc2"}
        }
    """.replace(" ", "").replace("\n", "")

    val type = ResolvableType.forClassWithGenerics(
        Map::class.java,
        String::class.java,
        PublicDto::class.java
    )

    val result = masker.mask(json, type)

    result.shouldBeEqual(json)
}

@Test
fun `should return original JSON for Mono of public DTO`() {
    val json =

```

```

""{"id":1,"name":"Alice","description":"Description"}""

        val type =
ResolvableType.forClassWithGenerics(Mono::class.java,
PublicDto::class.java)
        val result = masker.mask(json, type)

        result.shouldBeEqual(json)
    }

    @Test
    fun `mayContainSensitiveFields should detect CloudSensitive in
generic types`() {
        val listType =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)

typeAnalyzer.mayContainSensitiveFields(listType).shouldBeTrue()

        val mapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            String::class.java,
            SimpleDto::class.java
        )

typeAnalyzer.mayContainSensitiveFields(mapType).shouldBeTrue()

        val monoType =
ResolvableType.forClassWithGenerics(Mono::class.java,
SimpleDto::class.java)

typeAnalyzer.mayContainSensitiveFields(monoType).shouldBeTrue()

        val publicListType =
ResolvableType.forClassWithGenerics(List::class.java,
PublicDto::class.java)

typeAnalyzer.mayContainSensitiveFields(publicListType).shouldBeFalse()

        val sensitiveListType =
ResolvableType.forClassWithGenerics(
            CloudSensitive::class.java,
            ResolvableType.forClassWithGenerics(List::class.java,
String::class.java)
        )

typeAnalyzer.mayContainSensitiveFields(sensitiveListType).shouldBeTrue()
    }

    @Test
    fun `mayContainSensitiveFields should detect nested generic
types with CloudSensitive`() {
        val innerList =
ResolvableType.forClassWithGenerics(List::class.java,

```

```

SimpleDto::class.java)
    val outerList =
ResolvableType.forClassWithGenerics(List::class.java, innerList)

typeAnalyzer.mayContainSensitiveFields(outerList).shouldBeTrue()

    val listType =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
    val mapType = ResolvableType.forClassWithGenerics(
        Map::class.java,
        ResolvableType.forClass(String::class.java),
        listType
    )

typeAnalyzer.mayContainSensitiveFields(mapType).shouldBeTrue()

    val responseType =
ResolvableType.forClassWithGenerics(ResponseEntity::class.java,
SimpleDto::class.java)
    val fluxType =
ResolvableType.forClassWithGenerics(Flux::class.java, responseType)

typeAnalyzer.mayContainSensitiveFields(fluxType).shouldBeTrue()
}

@Test
fun `mayContainSensitiveFields should handle arrays`() {
    val arrayType =
ResolvableType.forArrayComponent(ResolvableType.forClass(SimpleDto::class.java))

typeAnalyzer.mayContainSensitiveFields(arrayType).shouldBeTrue()

    val publicArrayType =
ResolvableType.forArrayComponent(ResolvableType.forClass(PublicDto::class.java))

typeAnalyzer.mayContainSensitiveFields(publicArrayType).shouldBeFalse()

    val sensitiveArrayType = ResolvableType.forArrayComponent(
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
String::class.java)
    )

typeAnalyzer.mayContainSensitiveFields(sensitiveArrayType).shouldBeTrue()
}

@Test
fun `should handle custom collection implementations`() {
    class CustomList<T> : ArrayList<T>()

    val json = ""

```



```

        [
            {"id": 1, "name": "Item1", "secret": "secret1"},
            {"id": 2, "name": "Item2", "secret": "secret2"}
        ]
    """.trimIndent()

    val type =
ResolvableType.forClassWithGenerics(CustomList::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("\"name\": \"Item1\"")
    result.shouldContain("\"name\": \"Item2\"")
    result.shouldContain("\"secret\": \"****\"")
    result.shouldNotContain("secret1")
    result.shouldNotContain("secret2")
}

@Test
fun `should handle empty collections in generic types`() {
    val json = "[]"

    val type =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldBeEqual("[]")
}

@Test
fun `should handle null values in generic collections`() {
    val json = """[null, {"id": 1, "name": "Alice", "secret":
"secret1"}, null]"""

    val type =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("[null,")
    result.shouldContain(",null]")
    result.shouldContain("\"name\": \"Alice\"")
    result.shouldContain("\"secret\": \"****\"")
    result.shouldNotContain("secret1")
}

@Test
fun `should handle raw types without generics`() {
    val json = """{"id": 1, "name": "Test", "secret":
"my-secret"}"""

    val type =
ResolvableType.forRawClass(SimpleDto::class.java)
    val result = masker.mask(json, type)

```

```

        result.shouldContain("\\"name\\":\\"Test\\")
        result.shouldContain("\\"secret\\":\\"*****\\")
        result.shouldNotContain("my-secret")
    }

    @Test
    fun `should handle malformed JSON with generic type`() {
        val malformedJson = ""["id": 1, "name": "Test", "secret":
        ""

        val type =
        ResolvableType.forClassWithGenerics(List::class.java,
        SimpleDto::class.java)

        assertThrows<IllegalStateException> {
            masker.mask(malformedJson, type)
        }
    }

    @Test
    fun `should handle JSON type mismatch with generic type
    declaration`() {
        val json = ""["key":"value"]""

        val type = ResolvableType.forClassWithGenerics(
            Map::class.java,
            String::class.java,
            String::class.java
        )

        val result = masker.mask(json, type)
        result.shouldBeEqual(json)
    }

    @Test
    fun `should handle wildcard generic types`() {
        val json = ""["id": 1, "name": "Alice", "secret":
        "secret1"]""

        val wildcardType =
        ResolvableType.forClass(SimpleDto::class.java)
        val listType =
        ResolvableType.forClassWithGenerics(List::class.java, wildcardType)

        val result = masker.mask(json, listType)

        result.shouldContain("\\"name\\":\\"Alice\\")
        result.shouldContain("\\"secret\\":\\"*****\\")
        result.shouldNotContain("secret1")
    }

    @Test
    fun `should cache analysis results for generic types`() {
        val type1 =
        ResolvableType.forClassWithGenerics(List::class.java,
        SimpleDto::class.java)

```

```

        val type2 =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)

        val result1 = typeAnalyzer.mayContainSensitiveFields(type1)
        val result2 = typeAnalyzer.mayContainSensitiveFields(type2)

        result1.shouldBeTrue()
        result2.shouldBeTrue()
    }

    @Test
    fun `should handle CloudSensitive with primitive types in
collections`() {
        data class PrimitiveDto(
            val id: Long,
            val secretInt: CloudSensitive<Int>,
            val secretBoolean: CloudSensitive<Boolean>,
            val secretDouble: CloudOptional<CloudSensitive<Double>>
        )

        val json = """
        [
            {"id": 1, "secretInt": 42, "secretBoolean": true,
"secretDouble": 3.14},
            {"id": 2, "secretInt": 100, "secretBoolean": false,
"secretDouble": 2.71}
        ]
        """.trimIndent()

        val type =
ResolvableType.forClassWithGenerics(List::class.java,
PrimitiveDto::class.java)
        val result = masker.mask(json, type)

        result.shouldContain("\"id\":1")
        result.shouldContain("\"id\":2")
        result.shouldContain("\"secretInt\": \"****\"")
        result.shouldContain("\"secretBoolean\": \"****\"")
        result.shouldContain("\"secretDouble\": \"****\"")
        result.shouldNotContain("42")
        result.shouldNotContain("3.14")
    }

    @Test
    fun `should handle very deep nested generic types`() {
        val innerList =
ResolvableType.forClassWithGenerics(List::class.java,
SimpleDto::class.java)
        val mapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            ResolvableType.forClass(String::class.java),
            innerList
        )
        val outerList =
ResolvableType.forClassWithGenerics(List::class.java, mapType)

```

```

val json = """
    [
      {
        "group1": [
          {"id": 1, "name": "A", "secret": "s1"},
          {"id": 2, "name": "B", "secret": "s2"}
        ],
        "group2": [
          {"id": 3, "name": "C", "secret": "s3"}
        ]
      },
      {
        "group3": [
          {"id": 4, "name": "D", "secret": "s4"}
        ]
      }
    ]
  """.trimIndent()

val result = masker.mask(json, outerList)

result.shouldContain("\"group1\": [")
result.shouldContain("\"group2\": [")
result.shouldContain("\"group3\": [")
result.shouldContain("\"name\": \"A\"")
result.shouldContain("\"name\": \"B\"")
result.shouldContain("\"name\": \"C\"")
result.shouldContain("\"name\": \"D\"")
result.shouldContain("\"secret\": \"****\")
result.shouldNotContain("s1")
result.shouldNotContain("s2")
result.shouldNotContain("s3")
result.shouldNotContain("s4")
}

@Test
fun `should handle extremely nested generic structure with all
wrapper types`() {
  val json = """
    [
      [
        {
          "group1": {"token": "secret1"},
          "group2": {"token": "secret2"}
        }
      ],
      null,
      [
        {
          "group3": {"token": "secret3"}
        }
      ]
    ]
  """.trimIndent()

```

```

        val sensitiveString =
ResolvableType.forClassWithGenerics(CloudSensitive::class.java,
String::class.java)
        val innerMapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            ResolvableType.forClass(String::class.java),
            sensitiveString
        )
        val mapType = ResolvableType.forClassWithGenerics(
            Map::class.java,
            ResolvableType.forClass(String::class.java),
            innerMapType
        )
        val listType =
ResolvableType.forClassWithGenerics(List::class.java, mapType)
        val optionalType =
ResolvableType.forClassWithGenerics(CloudOptional::class.java,
listType)
        val fluxType =
ResolvableType.forClassWithGenerics(Flux::class.java, optionalType)

        val result = masker.mask(json, fluxType)

        result.shouldContain("\"token\":\"****\"")
        result.shouldNotContain("secret1")
        result.shouldNotContain("secret2")
        result.shouldNotContain("secret3")
    }
}

```

```

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/impl/CloudSensitiveMaskerTest.kt =====
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import
cloud.mws.devp.access.logging.spring.webflux.common.SensitiveValues
import cloud.mws.devp.cloud.optional.CloudOptional
import cloud.mws.devp.cloud.sensitive.CloudSensitive
import com.fasterxml.jackson.annotation.JsonProperty
import com.fasterxml.jackson.databind.ObjectMapper
import io.kotest.matchers.booleans.shouldBeFalse
import io.kotest.matchers.booleans.shouldBeTrue
import io.kotest.matchers.equals.shouldBeEqual
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.junit.jupiter.api.Test
import org.junit.jupiter.api.assertDoesNotThrow
import org.junit.jupiter.api.assertThrows
import org.springframework.core.ResolvableType
import java.util.Collections
import java.util.UUID
import java.util.concurrent.CountDownLatch
import java.util.concurrent.TimeUnit

```

```

import kotlin.concurrent.thread

@Suppress("detekt:StringTemplateIndent")
class CloudSensitiveMaskerTest {

    private val objectMapper = ObjectMapper()
    private val masker = CompositeSensitiveMasker(objectMapper,
SensitiveKeyMasker(objectMapper))

    data class SimpleRequest(
        val publicField: String,
        val secretToken: CloudSensitive<String>
    )

    data class NestedRequest(
        val id: Long,
        val credentials: Credentials,
        val metadata: Map<String, CloudSensitive<String>>
    )

    data class Credentials(
        val username: String,
        val password: CloudSensitive<String>
    )

    data class CollectionRequest(
        val publicList: List<String>,
        val secretList: List<CloudSensitive<String>>,
        val sensitiveList: CloudSensitive<List<String>>,
        val nestedList: List<Credentials>
    )

    data class NullableFieldsRequest(
        val requiredField: CloudSensitive<String>,
        val optionalField: CloudSensitive<String>?,
        val nullableString: String?
    )

    data class MixedCollectionsRequest(
        val stringList: List<String>,

        val sensitiveList: List<CloudSensitive<String>>,
        val nullableSensitiveList: List<CloudSensitive<String>?>,
        val secretList: CloudSensitive<List<String>>,

        val sensitiveSet: Set<CloudSensitive<Int>>,
        val nullableSensitiveSet: Set<CloudSensitive<Int>?>,
        val secretSet: CloudSensitive<Set<Int>>,

        val sensitiveArray: Array<CloudSensitive<Boolean>>,
        val nullableSensitiveArray:
Array<CloudSensitive<Boolean>?>,
        val secretArray: CloudSensitive<Array<Boolean>>,

        val mapWithSensitiveValues: Map<String,
CloudSensitive<String>>,

```

```

        val mapWithNullableSensitiveValues: Map<String,
CloudSensitive<String>?>,
        val mapWithRegularValues: Map<String, String>
    )

    data class NullableCollectionContainer(
        val listNotNullableElements: List<CloudSensitive<String>>,
        val listNullableElements: List<CloudSensitive<String>?>,
        val listNullableItself1: List<CloudSensitive<String>>?,
        val listNullableItself2: List<CloudSensitive<String>>?,
        val listFullyNullable1: List<CloudSensitive<String>?>?,
        val listFullyNullable2: List<CloudSensitive<String>?>?,
        val mapNotNullableValues: Map<String,
CloudSensitive<String>>,
        val mapNullableValues: Map<String,
CloudSensitive<String>?>,
        val mapNullableItself1: Map<String,
CloudSensitive<String>>?,
        val mapNullableItself2: Map<String,
CloudSensitive<String>>?,
        val mapFullyNullable1: Map<String,
CloudSensitive<String>?>?,
        val mapFullyNullable2: Map<String,
CloudSensitive<String>?>?
    )

    data class MapRequest(
        val map: Map<String, String>,
        val valuesAreSensitive: Map<String,
CloudSensitive<String>>,
        val keysAreSensitive: Map<CloudSensitive<String>, String>,
        val keysAndValuesAreSensitive: Map<CloudSensitive<String>,
CloudSensitive<String>>,
        val wholeMapIsSensitive: CloudSensitive<Map<String,
String>>,
        val nestedMap: Map<String, Map<String,
CloudSensitive<Int>>>,
        val nestedMapObject: Map<String, Credentials>,
        val nullableValues: Map<String, CloudSensitive<String>?>
    )

    data class MapWithNonStringKeys(
        val intKeyMap: Map<Int, CloudSensitive<String>>,
        val uuidKeyMap: Map<UUID, CloudSensitive<String>>
    )

    data class RequestWithArrays(
        val stringArray: Array<String>,
        val sensitiveArray: Array<CloudSensitive<String>>,
        val intArray: IntArray
    ) {
        override fun equals(other: Any?): Boolean {
            if (this === other) return true
            if (javaClass != other?.javaClass) return false

            other as RequestWithArrays

```

```

        if (!stringArray.contentEquals(other.stringArray))
return false
        if
(!sensitiveArray.contentEquals(other.sensitiveArray)) return false
        if (!intArray.contentEquals(other.intArray)) return
false

        return true
    }

    override fun hashCode(): Int {
        var result = stringArray.contentHashCode()
        result = 31 * result + sensitiveArray.contentHashCode()
        result = 31 * result + intArray.contentHashCode()
        return result
    }
}

data class NestedCollectionsRequest(
    val matrix: List<List<CloudSensitive<String>>>
)

data class DeepNestingRequest(
    val level1: Level1
)

data class Level1(
    val level2: Level2,
    val sensitive: CloudSensitive<String>
)

data class Level2(
    val level3: Level3,
    val list: List<CloudSensitive<String>>
)

data class Level3(
    val sensitive: CloudSensitive<Int>,
    val regular: String
)

abstract class BaseClass {
    abstract val baseSensitive: CloudSensitive<String>
}

data class InheritedFromBase(
    val additionalField: String,
    override val baseSensitive: CloudSensitive<String>
) : BaseClass()

data class RequestWithOptionals(
    val presentSensitive: CloudSensitive<String>? =
CloudSensitive("present"),
    val nullSensitive: CloudSensitive<String>? = null,
    val regularOptional: String? = null
)

```



```

)

data class RequestWithRecursiveReference(
    val name: String,
    val sensitive: CloudSensitive<String>,
    val children: List<RequestWithRecursiveReference>? = null
)

data class RequestWithJsonProperty(
    @get:JsonProperty("api_token") val secretToken:
CloudSensitive<String>,
    @get:JsonProperty("user_name") val username: String
)

data class RequestWithDifferentSensitiveTypes(
    val stringSensitive: CloudSensitive<String>,
    val intSensitive: CloudSensitive<Int>,
    val booleanSensitive: CloudSensitive<Boolean>,
    val listSensitive: CloudSensitive<List<String>>,
    val mapSensitive: CloudSensitive<Map<String, String>>
)

data class RequestWithAnyPayload(val payload: Any)

enum class Status { ACTIVE, INACTIVE }

data class RequestWithEnum(
    val status: Status,
    val secretStatus: CloudSensitive<Status>
)

data class UserRequest(
    val email: String,
    val password: CloudSensitive<String>
)

data class ListOfMapsRequest(
    val items: List<Map<String, UserRequest>>
)

data class MapOfListsRequest(
    val groups: Map<String, List<UserRequest>>
)

data class ProfileRequest(
    val userId: Long,
    val user: CloudOptional<UserRequest>
)

data class DoubleWrappedRequest(
    val data: CloudOptional<CloudOptional<UserRequest>>
)

data class RequestWithMultipleOptionals(
    val primary: CloudOptional<UserRequest> =
CloudOptional.empty(),

```

```

        val secondary: CloudOptional<Credentials> =
CloudOptional.empty(),
        val emptyOptional: CloudOptional<String> =
CloudOptional.empty()
    )

    data class NightmareRequest(
        val list: CloudOptional<
            List<
                Map<
                    CloudOptional<Int?>?,
                    ProfileRequest?
                >?
            >?
        >? = CloudOptional.default(emptyList())
    )

    @Test
    fun `should mask CloudSensitive fields in simple object`() {
        val json =
        """{"publicField":"test","secretToken":"super-secret"}"""
        val expected =
        """{"publicField":"test","secretToken":"****"}"""

        val result = masker.mask(json,
ResolvableType.forClass(SimpleRequest::class.java))

        result.shouldBeEqual(expected)
        result.shouldContain(SensitiveValues.MASKED_VALUE)
        result.shouldNotContain("super-secret")
    }

    @Test
    fun `should mask CloudSensitive fields in nested objects`() {
        val json = """
        {
            "id": 123,
            "credentials": {
                "username": "john",
                "password": "my-password"
            },
            "metadata": {"role": "admin"}
        }
        """.trimIndent()

        val result = masker.mask(json,
ResolvableType.forClass(NestedRequest::class.java))

        result.shouldContain("\password\":"****")
        result.shouldContain("\username\":"john")
        result.shouldContain("\role\":"****")
        result.shouldNotContain("my-password")
        result.shouldNotContain("admin")
    }

    @Test

```

```

fun `should mask CloudSensitive inside CloudOptional`() {
    val json = """
        {
            "userId": 42,
            "user": {
                "email": "john@example.com",
                "password": "my-secret-password"
            }
        }
    """.trimIndent()

    val result = masker.mask(json,
ResolvableType.forClass(ProfileRequest::class.java))

    result.shouldContain("\\"email\\":\\"john@example.com\\"")
    result.shouldContain("\\"password\\":\\"*****\\"")
    result.shouldNotContain("my-secret-password")
}

@Test
fun `should handle CloudOptional with null value`() {
    val json = """{"userId":100,"user":null}"""

    val result = masker.mask(json,
ResolvableType.forClass(ProfileRequest::class.java))

    result.shouldBeEqual("""{"userId":100,"user":null}""")
}

@Test
fun `should handle deeply nested CloudOptional wrappers`() {
    val json = """
        {
            "data": {
                "email": "nested@example.com",
                "password": "deep-secret"
            }
        }
    """.trimIndent()

    val result = masker.mask(
        json,
ResolvableType.forClass(DoubleWrappedRequest::class.java)
    )

    result.shouldContain("\\"email\\":\\"nested@example.com\\"")
    result.shouldContain("\\"password\\":\\"*****\\"")
    result.shouldNotContain("deep-secret")
}

@Test
fun `should mask multiple CloudOptional fields with different
inner types`() {
    val json = """
        {

```

```

        "primary": {
            "email": "primary@example.com",
            "password": "primary-secret"
        },
        "secondary": {
            "username": "secondary-user",
            "password": "secondary-secret"
        },
        "emptyOptional": null
    }
    """.trimIndent()

    val result = masker.mask(
        json,

ResolvableType.forClass(RequestWithMultipleOptionals::class.java)
    )

    result.shouldContain("\"email\": \"primary@example.com\"")

result.shouldContain("\"primary\": {\"email\": \"primary@example.com\\", \"password\": \"****\"}")

    result.shouldContain("\"username\": \"secondary-user\"")

result.shouldContain("\"secondary\": {\"username\": \"secondary-user\\", \"password\": \"****\"}")

    result.shouldContain("\"emptyOptional\": null")

    result.shouldNotContain("primary-secret")
    result.shouldNotContain("secondary-secret")
}

@Test
fun `should handle all-null nightmare structure without failure`() {
    val json = """{"list": null}"""
    val result = masker.mask(json,
ResolvableType.forClass(NightmareRequest::class.java))
    result.shouldBeEqual("""{"list": null}""")
}

@Test
fun `should handle empty list in nightmare structure`() {
    val json = """{"list": []}"""
    val result = masker.mask(json,
ResolvableType.forClass(NightmareRequest::class.java))
    result.shouldBeEqual("""{"list": []}""")
}

@Test
fun `should correctly mask CloudSensitive inside deeply nested optional structures`() {
    val json = """
    {

```

```

        "list": [
            {
                "123": {
                    "userId": 1,
                    "user": {
                        "email": "user1@example.com",
                        "password": "password1"
                    }
                },
                "null": null,
                "456": {
                    "userId": 2,
                    "user": {
                        "email": "user2@example.com",
                        "password": "password2"
                    }
                }
            },
            null,
            {
                "789": null,
                "100": {
                    "userId": 3,
                    "user": {
                        "email": "user3@example.com",
                        "password": "password3"
                    }
                }
            }
        ]
    }
}
""".trimIndent()

val result = masker.mask(json,
ResolvableType.forClass(NightmareRequest::class.java))

result.shouldContain("\"email\": \"user1@example.com\"")
result.shouldContain("\"email\": \"user2@example.com\"")
result.shouldContain("\"email\": \"user3@example.com\"")

result.shouldContain("\"password\": \"****\"")
result.shouldNotContain("password1")
result.shouldNotContain("password2")
result.shouldNotContain("password3")

result.shouldContain("""123":{"""")
result.shouldContain("""456":{"""")
result.shouldContain("""789:null""")
result.shouldContain("""100":{"""")
result.shouldContain(""" ,null, """)
}

@Test
fun `should mask all CloudSensitive elements in collections`()
{
    val json = """

```

```

        {
            "publicList": ["a", "b", "c"],
            "secretList": ["secret1", "secret2", "secret3"],
            "sensitiveList": ["sensitive1", "sensitive2"],
            "nestedList": [
                {"username": "user1", "password": "pass1"},
                {"username": "user2", "password": "pass2"}
            ]
        }
    """.trimIndent()

    val result = masker.mask(json,
ResolvableType.forClass(CollectionRequest::class.java))

    result.shouldContain("\\"publicList\\":[\\\"a\\\",\\\"b\\\",\\\"c\\\"]")
    result.shouldContain("\\"sensitiveList\\":\\\"****\\\"")

result.shouldContain("\\"secretList\\":[\\\"****\\\",\\\"****\\\",\\\"****\\\"]")
    result.shouldNotContain("secret1")

    result.shouldContain("\\"nestedList\\":[")
    result.shouldContain("\\"password\\":\\\"****\\\"")
    result.shouldNotContain("pass1")
    result.shouldNotContain("pass2")
}

@Test
fun `should handle nullable CloudSensitive fields`() {
    val json =
    """{"requiredField":"req-secret","optionalField":"opt-secret","null
ableString":null}"""

    val result = masker.mask(
        json,

ResolvableType.forClass(NullableFieldsRequest::class.java)
    )

    result.shouldContain("\\"requiredField\\":\\\"****\\\"")
    result.shouldContain("\\"optionalField\\":\\\"****\\\"")
    result.shouldContain("\\"nullableString\\":null")
    result.shouldNotContain("req-secret")
    result.shouldNotContain("opt-secret")
}

@Test
fun `should return original string when no CloudSensitive
fields`() {
    data class PublicRequest(
        val name: String,
        val age: Int,
        val tags: List<String>
    )

    val json =
    """{"name":"John","age":30,"tags":["dev","kotlin"]}"""

```

```

        val result = masker.mask(
            json,
            ResolvableType.forClass(PublicRequest::class.java)
        )

        result.shouldBeEqual(json)
    }

    @Test
    fun `should handle empty or null body gracefully`() {
        val type =
            ResolvableType.forClass(SimpleRequest::class.java)
        masker.mask("", type).shouldBeEqual("")
        masker.mask("null", type).shouldBeEqual("null")
        masker.mask("{}", type).shouldBeEqual("{}")
    }

    @Test
    fun `should keep not found properties in body`() {
        val type =
            ResolvableType.forClass(SimpleRequest::class.java)
        masker.mask("{\"data\":\"12\"}",
            type).shouldBeEqual("{\"data\":\"12\"}")
    }

    @Test
    fun `should handle JSON primitives at root level`() {
        val type =
            ResolvableType.forClass(SimpleRequest::class.java)
        masker.mask("", type).shouldBeEqual("")

        masker.mask(
            "\"plain string\"",
            ResolvableType.forClass(String::class.java)
        ).shouldBeEqual("\"plain string\"")

        masker.mask(
            "123",
            ResolvableType.forClass(Int::class.java)
        ).shouldBeEqual("123")

        masker.mask(
            "true",
            ResolvableType.forClass(Boolean::class.java)
        ).shouldBeEqual("true")

        masker.mask(
            "null",
            ResolvableType.forClass(Any::class.java)
        ).shouldBeEqual("null")
    }

    @Test
    fun `should not fail on JSON with extra fields not in class`() {

```

```

        val json = """
            {
                "publicField": "test",
                "secretToken": "secret",
                "extraField": "extra-value",
                "anotherExtra": {"nested": "value"}
            }
        """.trimIndent()

        val type =
        ResolvableType.forClass(SimpleRequest::class.java)
        assertDoesNotThrow {
            val result = masker.mask(json, type)
            result.shouldContain("\"secretToken\": \"****\")
            result.shouldContain("\"extraField\": \"extra-value\")

        result.shouldContain("\"anotherExtra\": {\"nested\": \"value\"}")
        }
    }

    @Test
    fun `should not fail on JSON with missing fields from class`()
    {
        val json = """{"publicField": "test"}"""

        val type =
        ResolvableType.forClass(SimpleRequest::class.java)
        assertDoesNotThrow {
            val result = masker.mask(json, type)
            result.shouldBeEqual("""{"publicField": "test"}""")
        }
    }

    @Test
    fun `should handle malformed JSON`() {
        val malformedJson = """{"publicField": "test",
"secretToken": """"

        val type =
        ResolvableType.forClass(SimpleRequest::class.java)
        assertThrows<IllegalStateException> {
            masker.mask(malformedJson, type)
        }
    }

    @Test
    fun `should handle mixed collection types correctly`() {
        val json = """
            {
                "stringList": ["a", "b", "c"],
                "sensitiveList": ["sensitive1", "sensitive2"],
                "nullableSensitiveList": ["present-secret", null],
                "secretList": ["secret1", "secret2"],
                "sensitiveSet": [42, 100],
                "nullableSensitiveSet": [999, null],
                "secretSet": [42, 100],
            }
        """
    }

```



```

        "sensitiveArray": [true, false, true],
        "nullableSensitiveArray": [true, null, null,
false],
        "secretArray": [true, false, true],
        "mapWithSensitiveValues": {
            "apiKey": "key-123",
            "token": "token-456"
        },
        "mapWithNullableSensitiveValues": {
            "presentKey": "present-value",
            "nullKey": null
        },
        "mapWithRegularValues": {
            "role": "admin",
            "department": "IT"
        }
    }
}
"".trimIndent()

val result = masker.mask(
    json,
    ResolvableType.forClass(MixedCollectionsRequest::class.java)
)

result.shouldContain("\stringList\":[\"a\", \"b\", \"c\"]")

result.shouldContain("\sensitiveList\":[\"****\", \"****\"]")
result.shouldContain("\nullableSensitiveList\":[\"****\", null]")
result.shouldContain("\secretList\":[\"****\"]")

result.shouldContain("\sensitiveSet\":[\"****\", \"****\"]")
result.shouldContain("\nullableSensitiveSet\":[\"****\", null]")
result.shouldContain("\secretSet\":[\"****\"]")

result.shouldContain("\sensitiveArray\":[\"****\", \"****\", \"****\"]")
result.shouldContain("\nullableSensitiveArray\":[\"****\", null, null, \"****\"]")
result.shouldContain("\secretArray\":[\"****\"]")

result.shouldContain("\apiKey\":[\"****\"]")
result.shouldContain("\token\":[\"****\"]")
result.shouldContain("\role\":[\"admin\"]")
result.shouldContain("\department\":[\"IT\"]")
result.shouldContain("\presentKey\":[\"****\"]")
result.shouldContain("\nullKey\":[null]")

result.shouldNotContain("secret1")
result.shouldNotContain("sensitive1")

```

```

        result.shouldNotContain("key-123")
        result.shouldNotContain("token-456")
        result.shouldNotContain("present-value")
        result.shouldNotContain("999")
    }

    @Test
    fun `should handle all nullable combinations for collections
and maps`() {
        val json = """
        {
            "listNotNullableElements": ["s1", "s2"],
            "listNullableElements": ["s3", null, "s4"],
            "listNullableItself1": ["s5", "s6"],
            "listNullableItself2": null,
            "listFullyNullable1": ["s7", null],
            "listFullyNullable2": null,
            "mapNotNullableValues": {"k1": "v1", "k2": "v2"},
            "mapNullableValues": {"k3": "v3", "k4": null},
            "mapNullableItself1": {"k5": "v5"},
            "mapNullableItself2": null,
            "mapFullyNullable1": {"k6": "v6", "k7": null},
            "mapFullyNullable2": null
        }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(NullableCollectionContainer::class.java)
        )

        result.shouldContain("""{"listNotNullableElements":["*****","*****"]}""")
        result.shouldContain("""{"listNullableElements":["*****",null,"*****"]}""")
        result.shouldContain("""{"listNullableItself1":["*****","*****"]}""")
        result.shouldContain("""{"listNullableItself2":null}""")
        result.shouldContain("""{"listFullyNullable1":["*****",null]}""")
        result.shouldContain("""{"listFullyNullable2":null}""")
        result.shouldContain("""{"mapNotNullableValues":{"k1":"*****","k2":"*****"}}""")
        result.shouldContain("""{"mapNullableValues":{"k3":"*****","k4":null}}""")
        result.shouldContain("""{"mapNullableItself1":{"k5":"*****"}}""")
        result.shouldContain("""{"mapNullableItself2":null}""")
        result.shouldContain("""{"mapFullyNullable1":{"k6":"*****","k7":null}}""")
    }

```

```

        result.shouldContain("""mapFullyNullable2:null""")
    }

    @Test
    fun `should handle all CloudSensitive Map combinations`() {
        val json = """
        {
            "map": {
                "public": "public",
                "message": "Hello"
            },
            "valuesAreSensitive": {
                "apiKey": "secret-key",
                "token": "bearer-token"
            },
            "keysAreSensitive": {
                "secret-user-id": "john",
                "secret-session": "abc123"
            },
            "keysAndValuesAreSensitive": {
                "super-secret-key": "super-secret-key",
                "super-secret-value": "super-secret-value"
            },
            "wholeMapIsSensitive": {
                "internal": "data"
            },
            "nestedMap": {
                "level1": {
                    "level2": 999
                }
            },
            "nestedMapObject": {
                "user1": {
                    "username": "john1",
                    "password": "john-password1"
                },
                "user2": {
                    "username": "john2",
                    "password": "john-password2"
                }
            },
            "nullableValues": {
                "present": "here",
                "absent": null
            }
        }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(MapRequest::class.java)
        )

        result.shouldContain("\"public\": \"public\"")
        result.shouldContain("\"message\": \"Hello\"")
    }

```

```

        result.shouldContain("\"apiKey\": \"****\")
        result.shouldContain("\"token\": \"****\")

        result.shouldContain("\"****_0\": \"john\")
        result.shouldContain("\"****_1\": \"abc123\")

        result.shouldContain("\"****_0\": \"****\")
        result.shouldContain("\"****_1\": \"****\")

        result.shouldContain("\"wholeMapIsSensitive\": \"****\")

        result.shouldContain("\"level2\": \"****\")

result.shouldContain("\"user1\": {\"username\": \"john1\", \"password\": \"****\"}")

result.shouldContain("\"user2\": {\"username\": \"john2\", \"password\": \"****\"}")

        result.shouldContain("\"present\": \"****\")
        result.shouldContain("\"absent\": null")

        result.shouldNotContain("secret-key")
        result.shouldNotContain("secret-user-id")
        result.shouldNotContain("secret-session")
        result.shouldNotContain("super-secret")
        result.shouldNotContain("john-password")
        result.shouldNotContain("bearer-token")
        result.shouldNotContain("999")
        result.shouldNotContain("here")
    }

    @Test
    fun `should handle Map with non-string keys (Int, UUID)`() {
        val uuid = UUID.randomUUID()
        val json = """
            {
                "intKeyMap": { "123": "secret-int", "456":
"another-secret" },
                "uuidKeyMap": { "$uuid": "secret-uuid" }
            }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(MapWithNonStringKeys::class.java)
        )

        result.shouldContain("\"123\": \"****\")
        result.shouldContain("\"456\": \"****\")
        result.shouldContain("\"$uuid\": \"****\")

        result.shouldNotContain("secret-int")
        result.shouldNotContain("another-secret")

```

```

        result.shouldNotContain("secret-uuid")
    }

    @Test
    fun `should mask CloudSensitive inside List of Map`() {
        val json = """
        {
            "items": [
                {
                    "key1": {
                        "username": "alice",
                        "password": "alice-secret"
                    },
                    "key2": {
                        "username": "bob",
                        "password": "bob-secret"
                    }
                },
                {
                    "key3": {
                        "username": "charlie",
                        "password": "charlie-secret"
                    }
                }
            ]
        }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(ListOfMapsRequest::class.java)
        )

        result.shouldContain("\\"username\\":\\"alice\\"")
        result.shouldContain("\\"username\\":\\"bob\\"")
        result.shouldContain("\\"username\\":\\"charlie\\"")

        result.shouldContain("\\"password\\":\\"****\\"")
        result.shouldNotContain("alice-secret")
        result.shouldNotContain("bob-secret")
        result.shouldNotContain("charlie-secret")

        result.shouldContain("key1":{"")
        result.shouldContain("key2":{"")
        result.shouldContain("key3":{"")
    }

    @Test
    fun `should mask CloudSensitive inside Map of List`() {
        val json = """
        {
            "groups": {
                "admin": [
                    {
                        "username": "admin1",
                        "password": "admin1-secret"
                    }
                ]
            }
        }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(MapOfListRequest::class.java)
        )

        result.shouldContain("\\"username\\":\\"admin1\\"")
        result.shouldContain("\\"password\\":\\"admin1-secret\\"")
    }
}

```

```

        },
        {
            "username": "admin2",
            "password": "admin2-secret"
        }
    ],
    "user": [
        {
            "username": "user1",
            "password": "user1-secret"
        }
    ]
}
}
"".trimIndent()

val result = masker.mask(
    json,
    ResolvableType.forClass(MapOfListsRequest::class.java)
)

result.shouldContain("\\"username\\":\\"admin1\\"")
result.shouldContain("\\"username\\":\\"user1\\"")

result.shouldContain("\\"password\\":\\"*****\\"")
result.shouldNotContain("admin1-secret")
result.shouldNotContain("user1-secret")

result.shouldContain("""admin":[{ """)
result.shouldContain("""user":[{ """)
}

@Test
fun `should handle nested collections like List of List of
CloudSensitive`() {
    val json = """"{"matrix": [[ "aaaaa", "bbbbbb"], ["ccccc"]]}""

    val result = masker.mask(
        json,
        ResolvableType.forClass(NestedCollectionsRequest::class.java)
    )

    result.shouldContain("\\"matrix\\":[[\\"*****\\",\\"*****\\"],[\\"*****\\"]]")
    result.shouldNotContain("aaaaa")
    result.shouldNotContain("bbbbbb")
    result.shouldNotContain("ccccc")
}

@Test
fun `should handle deep nesting with collections`() {
    val json = ""
    {
        "level1": {
            "level2": {

```

```

        "level3": {
            "sensitive": 12345,
            "regular": "normal"
        },
        "list": ["deep1", "deep2", "deep3"]
    },
    "sensitive": "top-secret"
}
"".trimIndent()

val result = masker.mask(
    json,
    ResolvableType.forClass(DeepNestingRequest::class.java)
)

result.shouldContain("\sensitive\s:\s*****\s")
result.shouldContain("\slist\s:[\s*****\s,\s*****\s,\s*****\s]")
result.shouldContain("\sensitive\s:\s*****\s")
result.shouldContain("\sregular\s:\snormal\s")

result.shouldNotContain("top-secret")
result.shouldNotContain("deep1")
result.shouldNotContain("12345")
}

@Test
fun `should handle inheritance correctly`() {
    val json =
        """"{"baseSensitive":"inherited-secret","additionalField":"public"}""
        ""

    val result = masker.mask(
        json,
        ResolvableType.forClass(InheritedFromBase::class.java)
    )

    result.shouldContain("\sbaseSensitive\s:\s*****\s")
    result.shouldContain("\sadditionalField\s:\spublic\s")
    result.shouldNotContain("inherited-secret")
}

@Test
fun `should handle optional fields with defaults`() {
    val json1 = """"{"presentSensitive":
    "im-here","nullSensitive": null,"regularOptional": "regular"}""

    val result1 = masker.mask(
        json1,
        ResolvableType.forClass(RequestWithOptionals::class.java)
    )

    result1.shouldContain("\spresentSensitive\s:\s*****\s")
    result1.shouldContain("\snullSensitive\s:null")
}

```

```

        result1.shouldContain("\regularOptional\":"regular\")
        result1.shouldNotContain("im-here")

        val json2 = "{}"
        val result2 = masker.mask(
            json2,

ResolvableType.forClass(RequestWithOptionals::class.java)
        )

        result2.shouldBeEqual("{}")
    }

    @Test
    fun `should prevent infinite recursion with self-referencing
types`() {
        val json = """
            {
                "name": "root",
                "sensitive": "root-secret",
                "children": [
                    {
                        "name": "child1",
                        "sensitive": "child1-secret",
                        "children": null
                    },
                    {
                        "name": "child2",
                        "sensitive": "child2-secret",
                        "children": [
                            {
                                "name": "child3",
                                "sensitive": "child3-secret",
                                "children": null
                            },
                            {
                                "name": "child4",
                                "sensitive": "child4-secret",
                                "children": []
                            }
                        ]
                    }
                ]
            }
        """.trimIndent()

        assertDoesNotThrow {
            val result = masker.mask(
                json,

ResolvableType.forClass(RequestWithRecursiveReference::class.java)
            )

            result.shouldContain("\name\":"root\")
            result.shouldContain("\name\":"child1\")
            result.shouldContain("\name\":"child4\")

```



```

        result.shouldContain("\sensitive\":"*****")
        result.shouldNotContain("root-secret")
        result.shouldNotContain("-secret")
    }
}

@Test
fun `should respect JsonProperty annotations`() {
    val json =
        """"{"api_token":"my-api-key","user_name":"john_doe"}""""
    val result = masker.mask(
        json,

ResolvableType.forClass(RequestWithJsonProperty::class.java)
    )

    result.shouldContain("\sapi_token\":"*****")
    result.shouldContain("\suser_name\":"john_doe")
    result.shouldNotContain("my-api-key")
}

@Test
fun `should handle different CloudSensitive generic types`() {
    val json = """"
        {
            "stringSensitive": "text-secret",
            "intSensitive": 42,
            "booleanSensitive": true,
            "listSensitive": ["item1", "item2"],
            "mapSensitive": {"key": "value"}
        }
    """".trimIndent()

    val result = masker.mask(
        json,

ResolvableType.forClass(RequestWithDifferentSensitiveTypes::class.j
ava)
    )

    result.shouldContain("\stringSensitive\":"*****")
    result.shouldContain("\intSensitive\":"*****")
    result.shouldContain("\booleanSensitive\":"*****")
    result.shouldContain("\listSensitive\":"*****")
    result.shouldContain("\mapSensitive\":"*****")

    result.shouldNotContain("text-secret")
    result.shouldNotContain("42")
    result.shouldNotContain("true")
    result.shouldNotContain("item1")
    result.shouldNotContain("value")
}

@Test
fun `should not crash and not mask CloudSensitive inside Any
type`() {

```

```

        val json = ""{"payload":{"token":"super-secret"}}""
        val result = masker.mask(
            json,

ResolvableType.forClass(RequestWithAnyPayload::class.java)
        )

        result.shouldBeEqual(json)
        result.shouldContain("super-secret") // as expected
    }

    @Test
    fun `should handle empty collections and null values in
collections`() {
        data class RequestWithEmptyCollections(
            val emptyList: List<CloudSensitive<String>> =
emptyList(),
            val nullList: List<CloudSensitive<String>>? = null,
            val listWithNulls: List<CloudSensitive<String>?> =
listOf(null, CloudSensitive("secret"), null)
        )

        val json = ""{"emptyList": [], "nullList":
null, "listWithNulls": [null, "secret-value", null]}""

        val result = masker.mask(
            json,

ResolvableType.forClass(RequestWithEmptyCollections::class.java)
        )

        result.shouldContain("\\"emptyList\\": []")
        result.shouldContain("\\"nullList\\": null")

result.shouldContain("\\"listWithNulls\\": [null, \\"****\\", null]")
        result.shouldNotContain("secret-value")
    }

    @Test
    fun `should handle primitive arrays`() {
        val json = ""
        {
            "stringArray": ["a", "b", "c"],
            "sensitiveArray": ["s1", "s2", "s3"],
            "intArray": [1, 2, 3]
        }
        """.trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(RequestWithArrays::class.java)
        )

        result.shouldContain("\\"stringArray\\": [\\"a\\", \\"b\\", \\"c\\"]")
result.shouldContain("\\"sensitiveArray\\": [\\"****\\", \\"****\\", \\"****\\")

```

```

    ])")
        result.shouldContain("\intArray\":[1,2,3]")
        result.shouldNotContain("s1")
    }

    @Test
    fun `should handle data classes with default values in
    constructor`() {
        data class RequestWithDefaults(
            val required: CloudSensitive<String>,
            val optionalWithDefault: CloudSensitive<String> =
CloudSensitive("default-secret"),
            val regularWithDefault: String = "default-regular"
        )

        val json1 = """{"required": "provided-secret"}"""
        val result1 = masker.mask(
            json1,
ResolvableType.forClass(RequestWithDefaults::class.java)
        )

        result1.shouldContain("\required\":"*****\")
        result1.shouldNotContain("provided-secret")

        val json2 = """{"required": "secret1",
"optionalWithDefault": "secret2", "regularWithDefault":
"regular2"}"""
        val result2 = masker.mask(
            json2,
ResolvableType.forClass(RequestWithDefaults::class.java)
        )

        result2.shouldContain("\required\":"*****\")
        result2.shouldContain("\optionalWithDefault\":"*****\")

        result2.shouldContain("\regularWithDefault\":"regular2\")
        result2.shouldNotContain("secret1")
        result2.shouldNotContain("secret2")
    }

    @Test
    fun `should handle enums`() {
        val json = """{"status": "ACTIVE", "secretStatus":
"ACTIVE"}""".trimIndent()

        val result = masker.mask(
            json,
            ResolvableType.forClass(RequestWithEnum::class.java)
        )

        result.shouldContain("\status\":"ACTIVE\")
        result.shouldContain("\secretStatus\":"*****\")
    }

```

```

@Test
fun `should handle CloudSensitive with Enum type in generic
collections`() {
    val json = """
        [
            {"status": "ACTIVE", "secretStatus": "ACTIVE"},
            {"status": "INACTIVE", "secretStatus": "PENDING"}
        ]
    """.trimIndent()

    val type =
        ResolvableType.forClassWithGenerics(List::class.java,
        RequestWithEnum::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("\"status\": \"ACTIVE\"")
    result.shouldContain("\"status\": \"INACTIVE\"")
    result.shouldContain("\"secretStatus\": \"****\"")
    result.shouldNotContain("\"secretStatus\": \"ACTIVE\"")
    result.shouldNotContain("\"secretStatus\": \"PENDING\"")
}

@Test
fun `should handle mixed nullability in complex generic
structure`() {
    data class MixedNullableDto(
        val required: CloudSensitive<String>,
        val optional: CloudSensitive<String>?,
        val list: List<CloudSensitive<String>?>,
        val map: Map<String, CloudSensitive<String>?>
    )

    val json = """
        {
            "required": "req-secret",
            "optional": null,
            "list": ["list1", null, "list3"],
            "map": {"key1": "map1", "key2": null}
        }
    """.trimIndent()

    val type =
        ResolvableType.forClass(MixedNullableDto::class.java)
    val result = masker.mask(json, type)

    result.shouldContain("\"required\": \"****\"")
    result.shouldContain("\"optional\": null")
    result.shouldContain("\"list\": [\"****\", null, \"****\"]")
    result.shouldContain("\"map\": {\"key1\": \"****\", \"key2\": null}")
    result.shouldNotContain("req-secret")
    result.shouldNotContain("list1")
    result.shouldNotContain("map1")
}

@Test

```

```

    fun `should handle deep nested structures without stack
overflow`() {
        data class DeepNode(
            val id: Int,
            val sensitive: CloudSensitive<String>,
            val next: DeepNode?
        )

        val depth = 200
        var json = ""{"id":0,"sensitive":"secret0","next":null}""

        for (i in 1..depth) {
            json =
""{"id":$i,"sensitive":"secret$i","next":$json}""
        }

        assertDoesNotThrow {
            val result = masker.mask(
                json,
                ResolvableType.forClass(DeepNode::class.java)
            )

            for (i in 0..depth) {
                result.shouldNotContain("secret$i")
            }
            result.shouldContain("""id":200""")
            result.shouldContain("""id":101""")
            result.shouldContain("""id":100""")
            result.shouldContain("""id":0""")
            result.shouldContain("\sensitive\":"\*****\")
        }
    }

    @Test
    fun `mayContainSensitiveFields should correctly detect presence
of CloudSensitive in all test structures`() {

        mayContainSensitiveFieldsWithoutGenerics(SimpleRequest::class.java)
            .shouldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(NestedRequest::class.java)
            .shouldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(Credentials::class.java).s
            houldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(CollectionRequest::class.j
            ava).shouldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(NullableFieldsRequest::cla
            ss.java).shouldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(MixedCollectionsRequest::c
            lass.java).shouldBeTrue()

        mayContainSensitiveFieldsWithoutGenerics(NullableCollectionContaine

```

```

r::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MapRequest::class.java).sh
ouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MapWithNonStringKeys::clas
s.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithArrays::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NestedCollectionsRequest::
class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (DeepNestingRequest::class.
java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level1::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level2::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (Level3::class.java).should
BeTrue()

mayContainSensitiveFieldsWithoutGenerics (InheritedFromBase::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithOptionals::clas
s.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithRecursiveRefere
nce::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithJsonProperty::c
lass.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithDifferentSensit
iveTypes::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (RequestWithEnum::class.jav
a).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (ProfileRequest::class.java
).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (ListOfMapsRequest::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (MapOfListsRequest::class.j
ava).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (DoubleWrappedRequest::clas
s.java).shouldBeTrue()

```

```

mayContainSensitiveFieldsWithoutGenerics (RequestWithMultipleOptiona
ls::class.java).shouldBeTrue()

mayContainSensitiveFieldsWithoutGenerics (NightmareRequest::class.ja
va).shouldBeTrue()

    data class PurePublicRequest (
        val name: String,
        val age: Int,
        val tags: List<String>,
        val metadata: Map<String, String>
    )

mayContainSensitiveFieldsWithoutGenerics (PurePublicRequest::class.j
ava).shouldBeFalse()

    data class NestedPublic(val inner: PurePublicRequest)

mayContainSensitiveFieldsWithoutGenerics (NestedPublic::class.java) .
shouldBeFalse()

    data class CollectionOfPublic(val items:
List<PurePublicRequest>)

mayContainSensitiveFieldsWithoutGenerics (CollectionOfPublic::class.
java).shouldBeFalse()

    data class MapOfPublic(val map: Map<String,
PurePublicRequest>)

mayContainSensitiveFieldsWithoutGenerics (MapOfPublic::class.java) .s
houldBeFalse()

mayContainSensitiveFieldsWithoutGenerics (String::class.java) .should
BeFalse()

mayContainSensitiveFieldsWithoutGenerics (Int::class.java) .shouldBeF
alse()

mayContainSensitiveFieldsWithoutGenerics (Boolean::class.java) .shoul
dBeFalse()

    assertThrows<UnsupportedTypeForMaskingException> {

mayContainSensitiveFieldsWithoutGenerics (List::class.java)

mayContainSensitiveFieldsWithoutGenerics (Map::class.java)
    }

    data class PublicOnlyOptional(val data:
CloudOptional<String>)

mayContainSensitiveFieldsWithoutGenerics (PublicOnlyOptional::class.
java) .shouldBeFalse()

```

```

        data class NestedPublicOptional(val wrapper:
CloudOptional<SimpleRequest>)

mayContainSensitiveFieldsWithoutGenerics(NestedPublicOptional::class
s.java).shouldBeTrue()

        data class RequestWithOnlyEnum(val status: Status)

mayContainSensitiveFieldsWithoutGenerics(RequestWithOnlyEnum::class
.java).shouldBeFalse()

mayContainSensitiveFieldsWithoutGenerics(Any::class.java).shouldBeF
alse()

mayContainSensitiveFieldsWithoutGenerics(Unit::class.java).shouldBe
False()

        class EmptyClass

mayContainSensitiveFieldsWithoutGenerics(EmptyClass::class.java).sh
ouldBeFalse()

        data class OnlyNullableStrings(val a: String?, val b:
String?)

mayContainSensitiveFieldsWithoutGenerics(OnlyNullableStrings::class
.java).shouldBeFalse()

        data class MixedNested(
            val public: PurePublicRequest,
            val secret: CloudSensitive<String>
        )

mayContainSensitiveFieldsWithoutGenerics(MixedNested::class.java).s
houldBeTrue()

        data class CyclicPublic(val next: CyclicPublic?)

mayContainSensitiveFieldsWithoutGenerics(CyclicPublic::class.java).
shouldBeFalse()

        data class CyclicSensitive(val next: CyclicSensitive?, val
secret: CloudSensitive<String>)

mayContainSensitiveFieldsWithoutGenerics(CyclicSensitive::class.jav
a).shouldBeTrue()
    }

    private fun mayContainSensitiveFieldsWithoutGenerics(clazz:
Class<*>): Boolean {
        return
CloudSensitiveTypeAnalyzer().mayContainSensitiveFields(ResolvableTy
pe.forRawClass(clazz))
    }

```



```

@Test
fun `should be thread safe when processing multiple requests
concurrently`() {
    val threadCount = 50
    val iterations = 100
    val latch = CountDownLatch(threadCount)
    val errors =
Collections.synchronizedList(mutableListOf<Throwable>())

    val testJson =
"""{"email":"test@example.com","password":"secret123"}"""
    val expectedMasked =
"""{"email":"test@example.com","password":"*****"}"""

    repeat(threadCount) {
        thread {
            try {
                repeat(iterations) {
                    val result = masker.mask(
                        testJson,
ResolvableType.forClass(UserRequest::class.java)
                        )
                    result.shouldBeEqual(expectedMasked)
                    Thread.yield()
                }
            } catch (e: Throwable) {
                errors.add(e)
            } finally {
                latch.countDown()
            }
        }
    }

    latch.await(10, TimeUnit.SECONDS)

    if (errors.isNotEmpty()) {
        throw AssertionError("Found ${errors.size} errors in
concurrent execution: ${errors[0]}")
    }
}

@Test
fun `should cache class metadata efficiently`() {
    val analyzer = CloudSensitiveTypeAnalyzer()

    val metadata1 =
analyzer.extractMetadata(UserRequest::class)
    metadata1.definitelyHasCloudSensitive().shouldBeTrue()

    val metadata2 =
analyzer.extractMetadata(UserRequest::class)

    (metadata1 === metadata2).shouldBeTrue()

```

```

        val simpleMetadata =
            analyzer.extractMetadata(SimpleRequest::class)
        val nestedMetadata =
            analyzer.extractMetadata(NestedRequest::class)

        simpleMetadata.definitelyHasCloudSensitive().shouldBeTrue()
        nestedMetadata.definitelyHasCloudSensitive().shouldBeFalse()

        analyzer.mayContainSensitiveFields(ResolvableType.forClass(NestedRequest::class.java)).shouldBeTrue()

        data class PublicData(val name: String, val age: Int)
        val publicMetadata1 =
            analyzer.extractMetadata(PublicData::class)
        val publicMetadata2 =
            analyzer.extractMetadata(PublicData::class)

        (publicMetadata1 === publicMetadata2).shouldBeTrue()

        publicMetadata1.definitelyHasCloudSensitive().shouldBeFalse()
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/common/impl/CompositeSensitiveMaskerCoWTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import cloud.mws.devp.cloud.sensitive.CloudSensitive
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
import io.kotest.matchers.shouldBe
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.junit.jupiter.api.Nested
import org.junit.jupiter.api.Test
import org.junit.jupiter.api.assertThrows
import org.mockito.kotlin.any
import org.mockito.kotlin.doAnswer
import org.mockito.kotlin.spy
import org.mockito.kotlin.whenever
import org.springframework.core.ResolvableType

class CompositeSensitiveMaskerCoWTest {

    private val objectMapper = jacksonObjectMapper()

    data class PublicDto(val name: String, val value: String)
    data class MixedDto(val name: String, val secret:
        CloudSensitive<String>, val password: String)

    @Nested
    inner class MinimalOverheadTests {

```

```

        private val masker = CompositeSensitiveMasker(
            objectMapper,
            SensitiveKeyMasker(objectMapper,
                NoOpSensitiveKeyStrategy)
        )
        private val type =
            ResolvableType.forClass(PublicDto::class.java)

        @Test
        fun `should return SAME string reference when no masking
        needed`() {
            val json = """"{"name":"test","value":"data"}""""
            val result = masker.mask(json, type)

            result shouldBe json
            (result === json) shouldBe true
        }

        @Test
        fun `should return SAME reference even if JSON has fields
        matching NO keys`() {
            val json = """"{"name":"test","token":"data"}""""
            val result = masker.mask(json, type)

            (result === json) shouldBe true
        }

        @Test
        fun `should return SAME reference for invalid JSON when no
        masking needed`() {
            val invalidJson = "{ invalid json }"
            val result = masker.mask(invalidJson, type)

            (result === invalidJson) shouldBe true
        }
    }

    @Nested
    inner class KeyBasedMaskingTests {

        private val strategy =
            ExactMatchStrategy(setOf("password"))
        private val masker = CompositeSensitiveMasker(
            objectMapper,
            SensitiveKeyMasker(objectMapper, strategy)
        )
        private val type =
            ResolvableType.forClass(PublicDto::class.java)

        @Test
        fun `should return NEW string when key matches`() {
            val json = """"{"name":"test","password":"secret"}""""
            val result = masker.mask(json, type)

            (result === json) shouldBe false
        }
    }

```

```

        result shouldContain "\"password\":"*****\"
        result shouldNotContain "secret"
    }

    @Test
    fun `should return SAME reference when keys do not match`() {
        val json = """"{"name":"test","token":"data"}""
        val result = masker.mask(json, type)

        (result === json) shouldBe true
    }

    @Test
    fun `should throw on invalid JSON when key masking is active`() {
        val invalidJson = "{ invalid json }"
        assertThrows<IllegalStateException> {
            masker.mask(invalidJson, type)
        }
    }

    @Nested
    inner class CloudSensitiveBasedMaskingTests {

        private val masker = CompositeSensitiveMasker(
            objectMapper,
            SensitiveKeyMasker(objectMapper,
                NoOpSensitiveKeyStrategy)
        )

        private val type =
            ResolvableType.forClass(MixedDto::class.java)

        @Test
        fun `should return NEW string when CloudSensitive field is present in JSON`() {
            val json = """"{"name":"test","secret":"my-secret","password":"plain"}""
            val result = masker.mask(json, type)

            (result === json) shouldBe false
            result shouldContain "\"secret\":"*****\"
            result shouldNotContain "my-secret"
            result shouldContain "\"password\":"plain\"
        }

        @Test
        fun `should return SAME reference when CloudSensitive field is ABSENT in JSON`() {
            val json = """"{"name":"test","password":"plain"}""
            val result = masker.mask(json, type)

            (result === json) shouldBe true
        }
    }

```

```

        @Test
        fun `should throw on invalid JSON when CloudSensitive
masking is active`() {
            val invalidJson = "{ invalid json }"
            assertThrows<IllegalStateException> {
                masker.mask(invalidJson, type)
            }
        }
    }

    @Nested
    inner class HybridMaskingTests {

        private val strategy = ExactMatchStrategy(setOf("password",
"foobar"))
        private val masker = CompositeSensitiveMasker(objectMapper,
SensitiveKeyMasker(objectMapper, strategy))
        private val type =
ResolvableType.forClass(MixedDto::class.java)

        @Test
        fun `should mask keys not listed in class`() {
            val json =
"""{"name":"test","secret":"anno-secret","password":"key-secret","f
oobar":"hello"}"""
            val result = masker.mask(json, type)

            (result == json) shouldBe false
            result shouldContain "\"secret\": \"****\""
            result shouldContain "\"password\": \"****\""
            result shouldContain "\"foobar\": \"****\""
            result shouldNotContain "anno-secret"
            result shouldNotContain "key-secret"
            result shouldNotContain "hello"
        }

        @Test
        fun `should parse JSON exactly once when both mechanisms
are active`() {
            val keyStrategy = ExactMatchStrategy(setOf("password"))
            val keyMasker = SensitiveKeyMasker(objectMapper,
keyStrategy)

            val spyingMapper = spy(objectMapper)
            var parseCount = 0

            spyingMapper.apply {
                doAnswer { invocation ->
                    parseCount++
                    invocation.callRealMethod()
                }.whenever(this).readTree(any<String>())
            }

            val masker = CompositeSensitiveMasker(spyingMapper,
keyMasker)

```

```

        val json =
        """{"name":"test","secret":"data","password":"pass"}"""
        val type =
        ResolvableType.forClass(MixedDto::class.java)

        val result = masker.mask(json, type)

        result shouldContain "\"secret\": \"****\""
        result shouldContain "\"password\": \"****\""
        parseCount shouldBe 1
    }

    @Test
    fun `should NOT parse JSON on repeated calls for
    non-CloudSensitive type`() {
        val objectMapper = jacksonObjectMapper()
        val spyingMapper = spy(objectMapper)
        var parseCount = 0

        spyingMapper.apply {
            doAnswer { invocation ->
                parseCount++
                invocation.callRealMethod()
            }.whenever(this).readTree(any<String>())
        }

        val masker = CompositeSensitiveMasker(
            spyingMapper,
            SensitiveKeyMasker(spyingMapper,
            NoOpSensitiveKeyStrategy)
        )
        val type =
        ResolvableType.forClass(PublicDto::class.java)
        val json = """{"name":"test"}"""

        masker.mask(json, type)
        parseCount shouldBe 0

        masker.mask(json, type)
        parseCount shouldBe 0
    }

    @Test
    fun `should mask BOTH CloudSensitive-wrapped field and
    key-matched field`() {
        val json =
        """{"name":"test","secret":"anno-secret","password":"key-secret"}"""
        val result = masker.mask(json, type)

        (result === json) shouldBe false
        result shouldContain "\"secret\": \"****\""
        result shouldContain "\"password\": \"****\""
        result shouldNotContain "anno-secret"
        result shouldNotContain "key-secret"
    }

```

```

        @Test
        fun `should return SAME reference if neither CloudSensitive
nor keys match`() {
            val json = """{"name":"test","other":"data"}"""
            val result = masker.mask(json, type)

            (result === json) shouldBe true
        }

        @Test
        fun `should mask ONLY CloudSensitive when keys do not
match`() {
            val json =
            """{"name":"test","secret":"anno-secret","other":"data"}"""
            val result = masker.mask(json, type)

            (result === json) shouldBe false
            result shouldContain "\"secret\": \"****\""
            result shouldNotContain "anno-secret"
            result shouldContain "\"other\": \"data\""
        }
    }

    @Nested
    inner class CachingTests {

        @Test
        fun `should handle repeated calls efficiently`() {
            val masker = CompositeSensitiveMasker(objectMapper,
SensitiveKeyMasker(objectMapper, NoOpSensitiveKeyStrategy))
            val type =
ResolvableType.forClass(PublicDto::class.java)
            val json = """{"name":"test"}"""

            val res1 = masker.mask(json, type)
            val res2 = masker.mask(json, type)

            (res1 === json) shouldBe true
            (res2 === json) shouldBe true
            (res1 === res2) shouldBe true
        }
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/common/impl/SensitiveKeyMaskingTest.kt =====
package cloud.mws.devp.access.logging.spring.webflux.common.impl

import
cloud.mws.devp.access.logging.spring.webflux.dto.KeyBasedSensitiveR
equest
import cloud.mws.devp.access.logging.spring.webflux.dto.NestedData

```

```

import
cloud.mws.devp.access.logging.spring.webflux.dto.NestedKeyBasedRequ
est
import cloud.mws.devp.access.logging.spring.webflux.dto.UserData
import cloud.mws.devp.access.logging.spring.webflux.dto.UserInfo
import cloud.mws.devp.access.logging.spring.webflux.dto.UserList
import
cloud.mws.devp.access.logging.spring.webflux.dto.UserWithPassword
import com.fasterxml.jackson.databind.ObjectMapper
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
import io.kotest.matchers.shouldBe
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.junit.jupiter.api.Test
import org.springframework.core.ResolvableType

class SensitiveKeyMaskingTest {

    private val objectMapper: ObjectMapper = jacksonObjectMapper()

    private fun masker(strategy: SensitiveKeyNameStrategy):
CompositeSensitiveMasker {
        return CompositeSensitiveMasker(objectMapper,
SensitiveKeyMasker(objectMapper, strategy))
    }

    @Test
    fun `EXACT strategy should mask exact key matches`() {
        val strategy = ExactMatchStrategy(
            setOf("password", "apiKey", "creditCardNumber",
"innerPassword", "innerToken")
        )
        val masker = masker(strategy)

        val json = objectMapper.writeValueAsString(
            UserData(
                username = "john_doe",
                userPassword = "secret123",
                email = "john@example.com",
                apiKey = "abc123",
                creditCardNumber = "4111111111111111",
                nested = NestedData(
                    innerPassword = "inner-secret",
                    innerToken = "token123",
                    publicField = "visible"
                )
            )
        )

        val result = masker.mask(json,
ResolvableType.forClass(UserData::class.java))

        result.shouldContain("\"apiKey\": \"****\"")
        result.shouldNotContain("abc123")
        result.shouldContain("\"creditCardNumber\": \"****\"")
        result.shouldNotContain("4111111111111111")
    }
}

```



```

        result.shouldContain("\\"innerPassword\\":\\"*****\\")
        result.shouldNotContain("inner-secret")
        result.shouldContain("\\"innerToken\\":\\"*****\\")
        result.shouldNotContain("token123")

        result.shouldContain("\\"userPassword\\":\\"secret123\\")
        result.shouldContain("\\"username\\":\\"john_doe\\")
        result.shouldContain("\\"email\\":\\"john@example.com\\")
        result.shouldContain("\\"publicField\\":\\"visible\\")
    }

    @Test
    fun `CONTAINS strategy should mask keys containing substrings`() {
        val strategy = ContainsMatchStrategy(listOf("pass",
"token", "credit", "api"))
        val masker = masker(strategy)

        val json = objectMapper.writeValueAsString(
            KeyBasedSensitiveRequest(
                username = "john_doe",
                password = "secret123",
                apiKey = "abc123",
                description = "some description"
            )
        )

        val result = masker.mask(json,
ResolvableType.forClass(KeyBasedSensitiveRequest::class.java))

        result.shouldContain("\\"username\\":\\"john_doe\\")
        result.shouldContain("\\"password\\":\\"*****\\")
        result.shouldContain("\\"apiKey\\":\\"*****\\")
        result.shouldContain("\\"description\\":\\"some
description\\")
        result.shouldNotContain("secret123")
        result.shouldNotContain("abc123")
    }

    @Test
    fun `CONTAINS strategy should be case-insensitive`() {
        val strategy = ContainsMatchStrategy(listOf("PASSWORD",
"token"))
        val masker = masker(strategy)

        data class CaseInsensitiveTest(
            val password: String,
            val userToken: String,
            val username: String
        )

        val json = objectMapper.writeValueAsString(
            CaseInsensitiveTest(
                password = "secret123",
                userToken = "abc123",
                username = "john_doe"
            )
        )
    }

```

```

        )
    )

    val result = masker.mask(json,
ResolvableType.forClass(CaseInsensitiveTest::class.java))

    result.shouldContain("\password\":"*****")
    result.shouldNotContain("secret123")
    result.shouldContain("\userToken\":"*****")
    result.shouldNotContain("abc123")
    result.shouldContain("\username\":"john_doe")
}

@Test
fun `should mask nested fields by key name`() {
    val strategy = ContainsMatchStrategy(listOf("cod",
"token"))
    val masker = masker(strategy)

    val json = objectMapper.writeValueAsString(
        NestedKeyBasedRequest(
            user = UserInfo(
                login = "john_doe",
                code = "secret123"
            ),
            secrets = mapOf(
                "apiToken" to "xyz789",
                "publicInfo" to "visible"
            )
        )
    )

    val result = masker.mask(json,
ResolvableType.forClass(NestedKeyBasedRequest::class.java))

    result.shouldContain("\login\":"john_doe")
    result.shouldContain("\code\":"*****")
    result.shouldNotContain("secret123")
    result.shouldContain("\apiToken\":"*****")
    result.shouldNotContain("xyz789")
    result.shouldContain("\publicInfo\":"visible")
}

@Test
fun `should mask entire objects by key name`() {
    val strategy = ContainsMatchStrategy(listOf("secret",
"oDE"))
    val masker = masker(strategy)

    val json = objectMapper.writeValueAsString(
        NestedKeyBasedRequest(
            user = UserInfo(
                login = "john_doe",
                code = "secret123"
            ),
            secrets = mapOf(

```

```

        "apiToken" to "xyz789",
        "publicInfo" to "visible"
    )
)

val result = masker.mask(json,
ResolvableType.forClass(NestedKeyBasedRequest::class.java))

result.shouldContain("\login\":"john_doe\")
result.shouldContain("\code\":"****\")
result.shouldNotContain("secret123")
result.shouldContain("\secrets\":"****\")
result.shouldNotContain("apiToken")
result.shouldNotContain("xyz789")
result.shouldNotContain("publicInfo")
result.shouldNotContain("visible")
}

@Test
fun `should mask fields in arrays`() {
    val strategy = ContainsMatchStrategy(listOf("password"))
    val masker = masker(strategy)

    val json = objectMapper.writeValueAsString(
        UserList(
            users = listOf(
                UserWithPassword("Alice", "alice123"),
                UserWithPassword("Bob", "bob456"),
                UserWithPassword("Charlie", "charlie789")
            )
        )
    )

    val result = masker.mask(json,
ResolvableType.forClass(UserList::class.java))

    result.shouldContain("\name\":"Alice\")
    result.shouldContain("\name\":"Bob\")
    result.shouldContain("\name\":"Charlie\")
    result.shouldContain("\password\":"****\")
    result.shouldNotContain("alice123")
    result.shouldNotContain("bob456")
    result.shouldNotContain("charlie789")
}

@Test
@Suppress("detekt:StringTemplateIndent")
fun `should mask keys inside arrays within maps`() {
    val strategy = ContainsMatchStrategy(listOf("token"))
    val masker = masker(strategy)

    val json = """
    {
        "items": [
            {"token": "secret1", "name": "item1"},

```

```

        {"token": "secret2", "name": "item2"}
    ]
}
"".trimIndent()

val resolvableType =
ResolvableType.forClass(Any::class.java)
val result = masker.mask(json, resolvableType)

result.shouldContain("\token\":"****")
result.shouldNotContain("secret1")
result.shouldNotContain("secret2")
result.shouldContain("\name\":"item1")
}

@Test
fun `should not mask when sensitiveKeys list is empty`() {
    val strategy = NoOpSensitiveKeyStrategy
    val masker = masker(strategy)

    val json = objectMapper.writeValueAsString(
        KeyBasedSensitiveRequest(
            username = "john_doe",
            password = "secret123",
            apiKey = "abc123",
            description = "description"
        )
    )

    val result = masker.mask(json,
ResolvableType.forClass(KeyBasedSensitiveRequest::class.java))

    result.shouldContain("\password\":"secret123")
    result.shouldContain("\apiKey\":"abc123")
    result.shouldContain("\username\":"john_doe")
    result.shouldContain("\description\":"description")
}

@Test
fun `should produce valid JSON after masking`() {
    val strategy = ContainsMatchStrategy(listOf("code",
"token"))
    val masker = masker(strategy)

    val json = objectMapper.writeValueAsString(
        NestedKeyBasedRequest(
            user = UserInfo(
                login = "john_doe",
                code = "secret123"
            ),
            secrets = mapOf(
                "apiToken" to "xyz789",
                "publicInfo" to "visible"
            )
        )
    )
}

```

```

        val result = masker.mask(json,
ResolvableType.forClass(NestedKeyBasedRequest::class.java))

        val parsed = objectMapper.readTree(result)
        parsed["user"]["code"].asText().shouldBe("****")
        parsed["user"]["login"].asText().shouldBe("john_doe")
        parsed["secrets"]["apiToken"].asText().shouldBe("****")

parsed["secrets"]["publicInfo"].asText().shouldBe("visible")
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/dto/KeyBasedSensitiveDtos.kt =====
package cloud.mws.devp.access.logging.spring.webflux.dto

data class KeyBasedSensitiveRequest(
    val username: String,
    val password: String,
    val apiKey: String,
    val description: String
)

data class NestedKeyBasedRequest(
    val user: UserInfo,
    val secrets: Map<String, String>
)

data class UserInfo(
    val login: String,
    val code: String
)

data class UserData(
    val username: String,
    val userPassword: String,
    val email: String,
    val apiKey: String,
    val creditCardNumber: String,
    val nested: NestedData
)

data class NestedData(
    val innerPassword: String,
    val innerToken: String,
    val publicField: String
)

data class UserWithPassword(
    val name: String,
    val password: String
)

```

```

data class UserList(
    val users: List<UserWithPassword>
)

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/dto/PublicDtos.kt =====
package cloud.mws.devp.access.logging.spring.webflux.dto

data class PublicRequest(
    val publicField: String,
    val publicToken: String
)

data class PublicResponse(
    val message: String,
    val publicToken: String
)

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/dto/SecretDtos.kt =====
package cloud.mws.devp.access.logging.spring.webflux.dto

import cloud.mws.devp.cloud.sensitive.CloudSensitive

data class SecretRequest(
    val publicField: String,
    val secretToken: CloudSensitive<String>
)

data class SecretResponse(
    val message: String,
    val maskedToken: CloudSensitive<String>
)

data class SecretRequestWithPassword(
    val publicField: String,
    val secretToken: CloudSensitive<String>,
    val password: String
)

data class SecretResponseWithApiKey(
    val message: String,
    val maskedToken: CloudSensitive<String>,
    val apiKey: String
)

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin

```

```

g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogClientAutoMaskingTest.kt
=====
package
cloud.mws.devp.access.logging.spring.webflux.exchange.filter

import cloud.mws.devp.access.logging.AccessLog
import cloud.mws.devp.access.logging.AccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilterFunction
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySerializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.CompositeSensitiveMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.SensitiveKeyMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretRequest
import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretResponse
import cloud.mws.devp.cloud.sensitive.CloudSensitive
import cloud.mws.devp.cloud.sensitive.cloudSensitive
import com.fasterxml.jackson.core.JsonGenerator
import com.fasterxml.jackson.core.JsonParser
import com.fasterxml.jackson.databind.DeserializationContext
import com.fasterxml.jackson.databind.ObjectMapper
import com.fasterxml.jackson.databind.SerializerProvider
import com.fasterxml.jackson.databind.deser.std.StdDeserializer
import com.fasterxml.jackson.databind.module.SimpleModule
import com.fasterxml.jackson.databind.ser.std.StdSerializer
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
import com.fasterxml.jackson.module.kotlin.registerKotlinModule
import com.github.tomakehurst.wiremock.WireMockServer
import com.github.tomakehurst.wiremock.client.WireMock.aResponse
import com.github.tomakehurst.wiremock.client.WireMock.post
import
com.github.tomakehurst.wiremock.core.WireMockConfiguration.options
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.assertj.core.api.Assertions.assertThat
import org.junit.jupiter.api.AfterEach

```

```

import org.junit.jupiter.api.BeforeEach
import org.junit.jupiter.api.Test
import org.springframework.core.ResolvableType
import org.springframework.http.MediaType
import
org.springframework.http.client.reactive.ReactorClientHttpConnector
import org.springframework.http.codec.json.Jackson2JsonDecoder
import org.springframework.http.codec.json.Jackson2JsonEncoder
import org.springframework.web.reactive.function.client.WebClient
import reactor.netty.http.client.HttpClient

class AccessLogClientAutoMaskingTest {

    private val server = WireMockServer(options().dynamicPort())

    private lateinit var accessLogWriter: AccessLogWriter
    private lateinit var requestProcessingStage:
RequestProcessingStage

    private lateinit var client: WebClient
    private lateinit var capturedAccessLogs: MutableList<String>

    @BeforeEach
    fun setup() {
        server.start()
        capturedAccessLogs = mutableListOf()

        accessLogWriter = object : AccessLogWriter {
            override fun write(accessLog: AccessLog) {
                capturedAccessLogs.add(accessLog.body ?: "")
            }
        }

        val mapper = jacksonObjectMapper()
        val sink = LoggerSink(
            accessLogWriter = accessLogWriter,
            bodySerializationStrategy =
AutoBodySerializationStrategy(
                false,
                CompositeSensitiveMasker(mapper,
SensitiveKeyMasker(mapper))
            )
        )

        requestProcessingStage = DefaultRequestProcessingStage(
            strategy = WithBodyStrategy(emptyList()),
            sink = sink
        )

        val objectMapper: ObjectMapper = jacksonObjectMapper()
            .registerKotlinModule()
            .registerModule(
                SimpleModule().apply {
                    addSerializer(
                        object :
StdSerializer<CloudSensitive<*>>(CloudSensitive::class.java) {

```



```

        override fun serialize(
            value: CloudSensitive<*>,
            gen: JsonGenerator,
            serializers: SerializerProvider
        ) {
            gen.writeString(value.toString())
        }
    })

    addDeserializer(
        CloudSensitive::class.java,
        object :
StdDeserializer<CloudSensitive<*>>(CloudSensitive::class.java) {
            override fun deserialize(
                parser: JsonParser,
                context: DeserializationContext
            ): CloudSensitive<*> {
                val value = parser.text
                return value.cloudSensitive()
            }
        })
    }
)

client = WebClient.builder()

    .clientConnector(ReactorClientHttpConnector(HttpClient.create()))
        .baseUrl(server.baseUrl())
        .codecs { configurer ->

configurer.defaultCodecs().jackson2JsonDecoder(Jackson2JsonDecoder(
objectMapper))

configurer.defaultCodecs().jackson2JsonEncoder(Jackson2JsonEncoder(
objectMapper))
        }
        .filter { request, next ->
            val newRequest =
org.springframework.web.reactive.function.client.ClientRequest.from
(request)
                .attributes { attrs ->
                    attrs["requestType"] =
ResolvableType.forClass(SecretRequest::class.java)
                    attrs["responseType"] =
ResolvableType.forClass(SecretResponse::class.java)
                }
                .build()
            next.exchange(newRequest)
        }

    .filter(AccessLogExchangeFilterFunction(requestProcessingStage))
        .build()
    }

@AfterEach
fun tearDown() {

```

```

        server.stop()
    }

    @Test
    fun `AUTO mode masks CloudSensitive in client request and
response`() {
        server.stubFor(
            post("/secret")
                .willReturn(
                    aResponse()
                        .withStatus(200)
                        .withHeader("Content-Type",
"application/json")
                )
                .withBody("""{"message":"OK","maskedToken":"server-masked"}""")
            )

        val requestDto = SecretRequest(
            publicField = "test-public",
            secretToken = "test-secret".cloudSensitive()
        )

        client.post()
            .uri("/secret")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(requestDto)
            .retrieve()
            .bodyToMono(SecretResponse::class.java)
            .block()

        assertThat(capturedAccessLogs).hasSize(2)

        val requestLog = capturedAccessLogs[0]
        val responseLog = capturedAccessLogs[1]

        requestLog.shouldContain("""secretToken:"****""")
        requestLog.shouldContain("""publicField:"test-public""")
        responseLog.shouldContain("""maskedToken:"****""")
        responseLog.shouldContain("""message:"OK""")
        requestLog.shouldNotContain("test-secret")
        responseLog.shouldNotContain("server-masked")
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/exchange/filter/AccessLogClientRawModeTest.kt
=====
package
cloud.mws.devp.access.logging.spring.webflux.exchange.filter

import cloud.mws.devp.access.logging.AccessLog
import cloud.mws.devp.access.logging.AccessLogWriter

```

```

import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilterFunction
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySerializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretRequest
import
cloud.mws.devp.access.logging.spring.webflux.dto.SecretResponse
import com.github.tomakehurst.wiremock.WireMockServer
import com.github.tomakehurst.wiremock.client.WireMock.aResponse
import com.github.tomakehurst.wiremock.client.WireMock.equalToJson
import com.github.tomakehurst.wiremock.client.WireMock.post
import
com.github.tomakehurst.wiremock.core.WireMockConfiguration.options
import io.kotest.matchers.string.shouldContain
import io.kotest.matchers.string.shouldNotContain
import org.assertj.core.api.Assertions.assertThat
import org.junit.jupiter.api.AfterEach
import org.junit.jupiter.api.BeforeEach
import org.junit.jupiter.api.Test
import org.springframework.http.MediaType
import
org.springframework.http.client.reactive.ReactorClientHttpConnector
import org.springframework.web.reactive.function.client.WebClient
import reactor.netty.http.client.HttpClient

class AccessLogClientRawModeTest {

    private val server = WireMockServer(options().dynamicPort())

    private lateinit var accessLogWriter: AccessLogWriter
    private lateinit var requestProcessingStage:
RequestProcessingStage

    private lateinit var client: WebClient
    private lateinit var capturedAccessLogs: MutableList<String>

    @BeforeEach
    fun setup() {
        server.start()
        capturedAccessLogs = mutableListOf()

        accessLogWriter = object : AccessLogWriter {

```

```

        override fun write(accessLog: AccessLog) {
            capturedAccessLogs.add(accessLog.body ?: "")
        }
    }

    val sink = LoggerSink(
        accessLogWriter = accessLogWriter,
        bodySerializationStrategy =
RawBodySerializationStrategy()
    )

    requestProcessingStage = DefaultRequestProcessingStage(
        strategy = WithBodyStrategy(emptyList()),
        sink = sink
    )

    client = WebClient.builder()

        .clientConnector(ReactorClientHttpConnector(HttpClient.create()))
        .baseUrl(server.baseUrl())
        .filter { request, next ->
            val newRequest =
org.springframework.web.reactive.function.client.ClientRequest.from
(request)
                .attributes { attrs ->
                    attrs["requestType"] =
SecretRequest::class.java
                    attrs["responseType"] =
SecretResponse::class.java
                }
                .build()
                next.exchange(newRequest)
            }

        .filter(AccessLogExchangeFilterFunction(requestProcessingStage))
        .build()
    }

    @AfterEach
    fun tearDown() {
        server.stop()
    }

    @Test
    fun `RAW mode should not mask CloudSensitive in client logs`()
    {
        val requestJson =
"""{"publicField":"test","secretToken":"input-secret"}"""
        val responseJson =
"""{"message":"OK","maskedToken":"server-masked"}"""

        server.stubFor(
            post("/secret")
                .withRequestBody(equalToJson(requestJson))
                .willReturn(
                    aResponse()

```

```

        .withStatus(200)
        .withHeader("Content-Type",
"application/json")
        .withBody(responseJson)
    )
)

client.post()
    .uri("/secret")
    .contentType(MediaType.APPLICATION_JSON)
    .bodyValue(requestJson)
    .retrieve()
    .bodyToMono(String::class.java)
    .block()

assertThat(capturedAccessLogs).hasSize(2)

val requestLog = capturedAccessLogs[0]
val responseLog = capturedAccessLogs[1]

requestLog.shouldNotContain("****")
requestLog.shouldContain("input-secret")
responseLog.shouldNotContain("****")
responseLog.shouldContain("server-masked")
}
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/exchange/filter/AccessLogExchangeFilterFunctionBaseTest.kt =====
package
cloud.mws.devp.access.logging.spring.webflux.exchange.filter

import cloud.mws.devp.access.logging.AccessLogWriter
import cloud.mws.devp.access.logging.Origin
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilterFunction
import cloud.mws.devp.access.logging.spring.webflux.TestUtil
import
cloud.mws.devp.access.logging.spring.webflux.common.PreCorrelation
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
import
cloud.mws.devp.access.logging.spring.webflux.common.http.HttpRequest
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import com.fasterxml.jackson.databind.ObjectMapper
import com.github.tomakehurst.wiremock.WireMockServer

```

```

import com.github.tomakehurst.wiremock.client.WireMock
import com.github.tomakehurst.wiremock.client.WireMock.aResponse
import com.github.tomakehurst.wiremock.client.WireMock.post
import com.github.tomakehurst.wiremock.core.Options
import
com.github.tomakehurst.wiremock.core.WireMockConfiguration.options
import io.kotest.assertions.throwables.shouldThrow
import io.kotest.matchers.equals.shouldBeEqual
import org.junit.jupiter.api.AfterEach
import org.junit.jupiter.api.BeforeEach
import org.junit.jupiter.api.Test
import org.mockito.Mockito
import org.mockito.Mockito.mock
import org.mockito.kotlin.argumentCaptor
import
org.springframework.http.client.reactive.ReactorClientHttpConnector
import org.springframework.http.codec.json.Jackson2JsonDecoder
import org.springframework.http.codec.json.Jackson2JsonEncoder
import org.springframework.web.reactive.function.client.WebClient
import
org.springframework.web.reactive.function.client.WebClientResponseException
import reactor.netty.http.client.HttpClient

class AccessLogExchangeFilterFunctionBaseTest {

    private val server = WireMockServer(options().dynamicPort())
    private val serverWithoutChunkEncoding = WireMockServer(
        options().dynamicPort()

        .useChunkedTransferEncoding(Options.ChunkedEncodingPolicy.BODY_FILE
        )
    )

    private val sink = mock(Sink::class.java)
    private val accessLogWriterMock = mock<AccessLogWriter>()
    private val accessLogWriter =
DefaultRequestProcessingStage(strategy =
WithBodyStrategy(listOf("metrics")), sink = sink)

    private lateinit var client: WebClient
    private lateinit var util: TestUtil

    @BeforeEach
    fun setup() {
        server.start()
        serverWithoutChunkEncoding.start()
        Mockito.`when`(sink.isActive).thenReturn(true)

        client = WebClient.builder()

        .clientConnector(ReactorClientHttpConnector(HttpClient.create()))
            .baseUrl(server.baseUrl())

        .filter(AccessLogExchangeFilterFunction(accessLogWriter))
            .codecs {

```

```

it.customCodecs().register(Jackson2JsonEncoder(ObjectMapper()))
    }
    .codecs {

it.customCodecs().register(Jackson2JsonDecoder(ObjectMapper()))
    }
    .build()
    util = TestUtil(sink, client, accessLogWriterMock)
}

@AfterEach
fun tearDown() {
    server.stop()
    serverWithoutChunkEncoding.stop()
}

@Test
fun shouldLogRequestWithoutBody() {

server.stubFor(post("/echo").willReturn(aResponse().withStatus(200)
))

    util.sendAndReceive()

    val (request, _) = util.captureResponse()
    request.method.shouldBeEqual("POST")

request.requestUri.shouldBeEqual("${server.baseUrl()}/echo")
    request.bodyAsString.shouldBeEqual("")
}

@Test
fun shouldLogEmptyResponseWithTransferEncodingChunked() {

server.stubFor(WireMock.get("/empty-chunked").willReturn(aResponse(
).withStatus(400)))
    shouldThrow<WebClientResponseException> {
        client
            .get()
            .uri(server.baseUrl() + "/empty-chunked")
            .retrieve()
            .toBodilessEntity()
            .block()
    }.message.shouldBeEqual("400 Bad Request from GET
${server.baseUrl()}/empty-chunked")

    val (request, response) = util.captureResponse()
    request.method.shouldBeEqual("GET")

request.requestUri.shouldBeEqual("${server.baseUrl()}/empty-chunked
")
    response.bodyAsString.shouldBeEqual("")
    response.getStatus().shouldBeEqual(400)
}

```

```

@Test
fun shouldLogRequestWithBody() {

server.stubFor(post("/discard").willReturn(aResponse().withStatus(200)))

    client.post()
        .uri("/discard")
        .bodyValue("Hello, world!")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val request = util.captureRequest()
    request.method.shouldBeEqual("POST")

request.requestUri.shouldBeEqual("${server.baseUrl()}/discard")
    request.bodyAsString.shouldBeEqual("Hello, world!")
}

@Test
fun shouldNotLogRequestIfInactive() {

server.stubFor(post("/echo").willReturn(aResponse().withStatus(200)))

    Mockito.`when`(sink.isActive).thenReturn(false)

    util.sendAndReceive()
    Mockito.verify(sink, Mockito.never()).write(
        argumentCaptor<PreCorrelation>().capture(),
        argumentCaptor<HttpRequest>().capture(),
        Mockito.isNull()
    )
}

@Test
fun shouldLogResponseWithoutBody() {

serverWithoutChunkEncoding.stubFor(post("/discard").willReturn(aResponse().withStatus(200)))

    client.post()
        .uri(serverWithoutChunkEncoding.baseUrl() + "/discard")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val (_, response) = util.captureResponse()
    response.getStatus().shouldBeEqual(200)

response.headers["Content-Length"]!!.first().shouldBeEqual("0")
    response.bodyAsString.shouldBeEqual("")
}

@Test

```



```

    fun shouldLogResponseWithBody() {
        serverWithoutChunkEncoding.stubFor(
            post("/echo").willReturn(
                aResponse().withStatus(200).withBody("Hello,
world!")
            )
        )

        val response = client.post()
            .uri(serverWithoutChunkEncoding.baseUrl() + "/echo")
            .bodyValue("Hello, world!")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()

        response!!.shouldBeEqual("Hello, world!")

        val (_, captureResponse) = util.captureResponse()
        captureResponse.bodyAsString.shouldBeEqual("Hello, world!")
        captureResponse.getStatus().shouldBeEqual(200)

        captureResponse.headers["Content-Length"]!!.first().shouldBeEqual("
13")
    }

    @Test
    fun shouldNotLogResponseIfInactive() {

server.stubFor(post("/echo").willReturn(aResponse().withStatus(200)
))

        Mockito.`when`(sink.isActive).thenReturn(false)

        util.sendAndReceive()

        Mockito.verify(sink, Mockito.never()).write(
            argumentCaptor<PreCorrelation>().capture(),
            argumentCaptor<HttpRequest>().capture(),
            Mockito.isNull()
        )
    }

    @Test
    fun shouldLogChunkedResponseWithBody() {

server.stubFor(WireMock.get("/chunked").willReturn(aResponse().with
Status(200).withBody("Hello, world!")))

        val response = client.get()
            .uri("/chunked")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()

        response!!.shouldBeEqual("Hello, world!")
    }

```

```

        val (_, capturedResponse) = util.captureResponse()

        capturedResponse.getStatus().shouldBeEqual(200)

        capturedResponse.headers["Transfer-Encoding"]!!.first().shouldBeEqual("chunked")
        capturedResponse.bodyAsString.shouldBeEqual("Hello, world!")
    }

    @Test
    fun shouldLogIncomingRequestWithBody() {

        server.stubFor(post("/echo").willReturn(aResponse().withStatus(200).withBody("response")))

        client.post()
            .uri("/echo")
            .bodyValue("""{"test": "data"}""")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()

        val request = util.captureRequest()
        request.method.shouldBeEqual("POST")
        request.origin.shouldBeEqual(Origin.OUTGOING)
        request.bodyAsString.shouldBeEqual("""{"test": "data"}""")
    }

    @Test
    fun shouldBufferBodyBeforeLoggingForIncomingRequests() {

        server.stubFor(post("/discard").willReturn(aResponse().withStatus(200)))

        val requestBody = """{"test": "data"}"""

        client.post()
            .uri("/discard")
            .bodyValue(requestBody)
            .retrieve()
            .bodyToMono(String::class.java)
            .block()

        val request = util.captureRequest()
        request.bodyAsString.shouldBeEqual(requestBody)
        request.origin.shouldBeEqual(Origin.OUTGOING)
    }

    @Test
    fun shouldHandleIncomingRequestWithEmptyBody() {

        server.stubFor(post("/discard").willReturn(aResponse().withStatus(200)))

        client.post()

```

```

        .uri("/discard")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val (request, _) = util.captureResponse()
    request.bodyAsString.shouldBeEqual("")
    request.origin.shouldBeEqual(Origin.OUTGOING)
}

@Test
fun shouldHandleIncomingRequestWithLargeBody() {
    val largeBody = "x".repeat(10000)

server.stubFor(post("/discard").willReturn(aResponse().withStatus(200)))

    client.post()
        .uri("/discard")
        .bodyValue(largeBody)
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val request = util.captureRequest()
    request.bodyAsString.shouldBeEqual(largeBody)
    request.origin.shouldBeEqual(Origin.OUTGOING)
}

@Test
fun shouldLogIncomingRequestWithJsonBody() {

server.stubFor(post("/echo").willReturn(aResponse().withStatus(200)
.withBody("ok")))

    client.post()
        .uri("/echo")
        .header("Content-Type", "application/json")
        .bodyValue("""{"key": "value"}""")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val request = util.captureRequest()
    request.bodyAsString.shouldBeEqual("""{"key": "value"}""")
    request.contentType?.shouldBeEqual("application/json")
    request.origin.shouldBeEqual(Origin.OUTGOING)
}
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogAutoMaskingTest.kt =====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

```

```

import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import org.assertj.core.api.Assertions.assertThat
import org.junit.jupiter.api.Test
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.http.MediaType
import reactor.core.publisher.Mono

@SpringBootTest(
    classes = [TestApplication::class,
AutoMaskingTestConfig::class],
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = [
        "devp.api.access-log.body-serialization-mode=AUTO",
        "devp.api.access-log.strategy=WITH_BODY"
    ]
)
class AccessLogAutoMaskingTest : AccessLogBaseTest() {

    @Test
    fun `test secret endpoint with full response`() {
        val jsonRequest = """"{"publicField": "public-value",
"secretToken": "super-secret-password"}""".trimIndent()

        val response = client.post()
            .uri("/secret")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(jsonRequest)
            .exchangeToMono { clientResponse ->
                println("Status: ${clientResponse.statusCode()}")
                println("Headers:
${clientResponse.headers().asHttpHeaders()}")
                clientResponse.bodyToMono(String::class.java)
                    .doOnNext { body -> println("Body: $body") }
                    .thenReturn(clientResponse)
            }
            .block()

        println("Final response status: ${response?.statusCode()}")
    }

    @Test
    fun `AUTO mode should mask CloudSensitive in request and
response`() {
        client.post()
            .uri("/secret")
            .contentType(MediaType.APPLICATION_JSON)

            .bodyValue("""{"publicField": "public-value", "secretToken": "super-se
cret-password"}""")
            .retrieve()
            .toBodilessEntity()
            .block()

        val accessLogs = util.captureAccessLogWrite()
        assertThat(accessLogs).hasSize(2)
    }
}

```

```

        val requestLog = accessLogs[0]

assertThat(requestLog.body).contains("""{"publicField":"public-value","secretToken":"*****"}""")

assertThat(requestLog.body).doesNotContain("super-secret-password")

        val responseLog = accessLogs[1]
        assertThat(responseLog.body).contains("""{"message":"OK for public-value","maskedToken":"*****"}""")
        assertThat(responseLog.body).doesNotContain("TOKEN")
    }

@Test
fun `AUTO mode should not parse non-JSON content`() {
    client.post()
        .uri("/echo-raw")
        .contentType(MediaType.TEXT_PLAIN)
        .bodyValue("plain text with secret")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val accessLogs = util.captureAccessLogWrite()
    assertThat(accessLogs).hasSize(2)

    val requestAccessLog = accessLogs[0]

    assertThat(requestAccessLog.body).isEqualTo("[BINARY_CONTENT]")
}

@Test
fun `AUTO mode should log body for even broken JSON for primitive type`() {
    client.post()
        .uri("/echo")
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue("{ invalid json }")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()

    val accessLogs = util.captureAccessLogWrite()
    assertThat(accessLogs).hasSize(2)

    val requestAccessLog = accessLogs[0]
    assertThat(requestAccessLog.body).isEqualTo("{ invalid json }")
}

@Test
fun `AUTO mode should log body for even broken JSON if type is safe`() {
    client.post()
        .uri("/public")

```

```

        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue("{ invalid json }")
        .retrieve()
        .onStatus({ httpStatus -> httpStatus.is4xxClientError
    },
        { _ -> Mono.empty() }
    )
    .toBodilessEntity()
    .block()

    val accessLogs = util.captureAccessLogWrite()
    assertThat(accessLogs).hasSize(2)

    val requestAccessLog = accessLogs[0]
    assertThat(requestAccessLog.body).isEqualTo("{ invalid json
}"))
    }

    @Test
    fun `AUTO mode returns JSON_ERROR marker on JSON parsing
error`() {
        client.post()
            .uri("/secret")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue("{ invalid json }")
            .retrieve()
            .onStatus({ httpStatus -> httpStatus.is4xxClientError
    },
        { _ -> Mono.empty() }
    )
    .toBodilessEntity()
    .block()

    val accessLogs = util.captureAccessLogWrite()
    assertThat(accessLogs).hasSize(2)

    val requestAccessLog = accessLogs[0]
    assertThat(requestAccessLog.body).isEqualTo("[JSON_ERROR]")
    }
}

```

```

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/web/filter/AccessLogBaseTest.kt =====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

import cloud.mws.devp.access.logging.AccessLogWriter
import cloud.mws.devp.access.logging.spring.webflux.TestUtil
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import org.junit.jupiter.api.BeforeEach
import org.mockito.Mockito
import org.springframework.boot.test.web.server.LocalServerPort
import org.springframework.http.HttpStatus

```

```

import
org.springframework.test.context.bean.override.mockito.MockitoBean
import
org.springframework.test.context.bean.override.mockito.MockitoSpyBean
import org.springframework.web.reactive.function.client.WebClient
import org.springframework.web.server.ServerWebExchange
import org.springframework.web.server.WebFilter
import org.springframework.web.server.WebFilterChain
import reactor.core.publisher.Mono

open class AccessLogBaseTest {
    protected lateinit var client: WebClient

    @MockitoBean
    protected lateinit var accessLogWriter: AccessLogWriter

    @MockitoSpyBean
    protected lateinit var sink: LoggerSink
    protected lateinit var util: TestUtil

    @LocalServerPort
    var port: Int = 0

    @BeforeEach
    fun setup() {
        Mockito.`when`(sink.isActive).thenReturn(true)
        client = WebClient.builder()
            .baseUrl("http://localhost:$port")
            .build()
        util = TestUtil(sink, client, accessLogWriter)
    }
}

class RejectingFilter : WebFilter {
    override fun filter(exchange: ServerWebExchange, chain:
WebFilterChain): Mono<Void> {
        return if (exchange.request.path.value() == "/rejected") {
            exchange.response.setStatusCode(HttpStatus.UNAUTHORIZED)

            exchange.response.setComplete()
        } else {
            chain.filter(exchange)
        }
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/web/filter/AccessLogHeadersWhitelistTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

```

```

import cloud.mws.devp.access.logging.AccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import io.kotest.matchers.collections.shouldContain
import io.kotest.matchers.collections.shouldNotBeEmpty
import io.kotest.matchers.collections.shouldNotContain
import io.kotest.matchers.equals.shouldBeEqual
import io.kotest.matchers.nulls.shouldNotBeNull
import io.kotest.matchers.string.shouldContain
import org.junit.jupiter.api.Test
import org.mockito.junit.jupiter.MockitoExtension
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.boot.test.context.TestConfiguration
import org.springframework.context.annotation.Bean
import org.springframework.context.annotation.Import
import org.springframework.core.env.Environment
import
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping

@Import(MockitoExtension::class)
@SpringBootTest(
    classes = [TestApplication::class,
HeadersWhiteListConfiguration::class],
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
)
class AccessLogHeadersWhitelistTest : AccessLogBaseTest() {

    @Test
    fun shouldLogRequestWithBody() {
        client.post()
            .uri("/discard")
            .header("whitelisted-header", "some-test-value-1",
"some-test-value-2")
            .header("not-whitelisted-header", "some-test-value-3",
"some-test-value-4")
            .bodyValue("Hello, world!")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()
        val list = util.captureAccessLogWrite()
        val requestAccessLog = list.first()
        requestAccessLog.method.shouldBeEqual("POST")
    }
}

```



```

        requestAccessLog.body!!.shouldBeEqual("Hello, world!")
        requestAccessLog.uri.shouldContain("/discard")

        val headers = requestAccessLog.headers
        headers.shouldNotBeNull()

        val keys = headers.keys
        keys.shouldNotBeNull()
        headers.keys.shouldNotBeEmpty()
        headers.keys.shouldContain("whitelisted-header")
        headers.keys.shouldNotContain("not-whitelisted-header")
    }
}

@TestConfiguration
open class HeadersWhiteListConfiguration {
    // @MockitoSpyBean требуется бин для замены
    https://docs.spring.io/spring-framework/reference/6.2.3/testing/ann
    otations/integration-spring/annotation-mockitobean.html
    @Bean
    open fun loggerSink(accessLogWriter: AccessLogWriter) =
        LoggerSink(accessLogWriter, listOf("whitelisted-header"))

    @Bean
    open fun requestProcessingStage(sink: Sink):
    RequestProcessingStage {
        return DefaultRequestProcessingStage(
            strategy = WithBodyStrategy(listOf()),
            sink = sink
        )
    }

    @Bean
    open fun filter(
        requestProcessingStage: RequestProcessingStage,
        requestMappingHandlerMapping: RequestMappingHandlerMapping,
        environment: Environment,
    ) = AccessLogWebFilter(requestProcessingStage,
        requestMappingHandlerMapping, environment)

    @Bean
    open fun rejectingFilter() = RejectingFilter()
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/web/filter/AccessLogRawModeTest.kt =====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import org.assertj.core.api.Assertions.assertThat
import org.junit.jupiter.api.Test
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.http.MediaType

```

```

@SpringBootTest(
    classes = [TestApplication::class,
AutoMaskingTestConfig::class],
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = [
        "devp.api.access-log.body-serialization-mode=RAW",
        "devp.api.access-log.strategy=WITH_BODY"
    ]
)
class AccessLogRawModeTest : AccessLogBaseTest() {

    @Test
    fun `RAW mode should not mask CloudSensitive`() {
        client.post()
            .uri("/secret")
            .contentType(MediaType.APPLICATION_JSON)

        .bodyValue("""{"publicField":"public-value","secretToken":"super-se
cret-password"}""")
            .retrieve()
            .toBodilessEntity()
            .block()

        val accessLogs = util.captureAccessLogWrite()

        val requestLog = accessLogs[0]

assertThat(requestLog.body).contains("super-secret-password")
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spri
ng/webflux/web/filter/AccessLogSensitiveKeyMaskingTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import
cloud.mws.devp.access.logging.spring.webflux.dto.KeyBasedSensitiveR
equest
import
cloud.mws.devp.access.logging.spring.webflux.dto.NestedKeyBasedRequ
est
import cloud.mws.devp.access.logging.spring.webflux.dto.UserInfo
import org.assertj.core.api.Assertions.assertThat
import org.junit.jupiter.api.Test
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.http.MediaType

@SpringBootTest(
    classes = [TestApplication::class,
SensitiveKeyMaskingTestConfig::class],

```

```

webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
properties = [
    "devp.api.access-log.body-serialization-mode=AUTO",
    "devp.api.access-log.strategy=WITH_BODY",
    "devp.api.access-log.sensitive-json-keys=password,token,code,apiKey",
    "devp.api.access-log.sensitive-key-detection-strategy=CONTAINS"
]
)
class AccessLogSensitiveKeyMaskingTest : AccessLogBaseTest() {

    @Test
    fun `should mask sensitive keys in request and response`() {
        val request = KeyBasedSensitiveRequest(
            username = "john_doe",
            password = "super-secret-password",
            apiKey = "abc-123-xyz",
            description = "visible description"
        )

        client.post()
            .uri("/echo")
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(request)
            .retrieve()
            .bodyToMono(String::class.java)
            .block()

        val accessLogs = util.captureAccessLogWrite()
        assertThat(accessLogs).hasSize(2)

        val requestLog = accessLogs[0]

        assertThat(requestLog.body).contains("\"password\": \"*****\"")
        assertThat(requestLog.body).contains("\"apiKey\": \"*****\"")

        assertThat(requestLog.body).contains("\"username\": \"john_doe\"")

        assertThat(requestLog.body).contains("\"description\": \"visible description\"")

        assertThat(requestLog.body).doesNotContain("super-secret-password")
        assertThat(requestLog.body).doesNotContain("abc-123-xyz")
    }

    @Test
    fun `should mask nested fields in map`() {
        val request = NestedKeyBasedRequest(
            user = UserInfo(
                login = "john_doe",
                code = "secret123"
            ),
            secrets = mapOf(
                "apiToken" to "xyz789",
            )
        )
    }
}

```

```

        "publicInfo" to "visible"
    )
)

client.post()
    .uri("/echo")
    .contentType(MediaType.APPLICATION_JSON)
    .bodyValue(request)
    .retrieve()
    .bodyToMono(String::class.java)
    .block()

val accessLogs = util.captureAccessLogWrite()
assertThat(accessLogs).hasSize(2)

val requestLog = accessLogs[0]

assertThat(requestLog.body).contains("\login\":"john_doe\")
    assertThat(requestLog.body).contains("\code\":"****\")

assertThat(requestLog.body).contains("\apiToken\":"****\")

assertThat(requestLog.body).contains("\publicInfo\":"visible\")
    assertThat(requestLog.body).doesNotContain("secret123")
    assertThat(requestLog.body).doesNotContain("xyz789")
}

@Test
fun `should mask BOTH keys and CloudSensitive`() {
    client.post()
        .uri("/secret-keys")
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(
            """{"publicField":"public-value","secretToken":"super-secret-passwo
            rd","password":"plain-password"}"""
        )
        .retrieve()
        .toBodilessEntity()
        .block()

    val accessLogs = util.captureAccessLogWrite()
    assertThat(accessLogs).hasSize(2)

    val requestLog = accessLogs[0]
    assertThat(requestLog.body).contains(

        """{"publicField":"public-value","secretToken":"****","password":"*
        ***"}"""
    )

    assertThat(requestLog.body).doesNotContain("super-secret-password")
    assertThat(requestLog.body).doesNotContain("plain-password")

    val responseLog = accessLogs[1]

```

```

        assertThat(responseLog.body).contains(
            ""{"message":"OK for
public-value","maskedToken":"*****","apiKey":"*****"}""
        )
        assertThat(responseLog.body).doesNotContain("TOKEN")

    assertThat(responseLog.body).doesNotContain("default-api-key")
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogWithBodyStrategyTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

import cloud.mws.devp.access.logging.AccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import
cloud.mws.devp.access.logging.spring.webflux.common.PreCorrelation
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
import
cloud.mws.devp.access.logging.spring.webflux.common.http.HttpRequest
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import io.kotest.assertions.throwables.shouldThrowExactly
import io.kotest.matchers.equals.shouldBeEqual
import io.kotest.matchers.string.shouldContain
import org.junit.jupiter.api.Test
import org.mockito.Mockito
import org.mockito.junit.jupiter.MockitoExtension
import org.mockito.kotlin.argumentCaptor
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.boot.test.context.TestConfiguration
import org.springframework.context.annotation.Bean
import org.springframework.context.annotation.Import
import org.springframework.core.env.Environment
import
org.springframework.web.reactive.function.client.WebClientResponseException
import
org.springframework.web.reactive.result.method.annotation.RequestMa

```

```

ppingHandlerMapping

@TestConfiguration
open class WithBodyConfiguration {
    // @MockitoSpyBean требуется бин для замены
    https://docs.spring.io/spring-framework/reference/6.2.3/testing/ann
    otations/integration-spring/annotation-mockitobean.html
    @Bean
    open fun loggerSink(accessLogWriter: AccessLogWriter) =
        LoggerSink(accessLogWriter)

    @Bean
    open fun requestProcessingStage(sink: Sink):
        RequestProcessingStage {
        return DefaultRequestProcessingStage(
            strategy = WithBodyStrategy(listOf("/post",
            "/withoutBodyEndpoint", "/query/.*/test")),
            sink = sink
        )
    }

    @Bean
    open fun filter(
        requestProcessingStage: RequestProcessingStage,
        requestMappingHandlerMapping: RequestMappingHandlerMapping,
        environment: Environment,
    ) = AccessLogWebFilter(requestProcessingStage,
        requestMappingHandlerMapping, environment)

    @Bean
    open fun rejectingFilter() = RejectingFilter()
}

@Import(MockitoExtension::class)
@SpringBootTest(
    classes = [TestApplication::class,
        WithBodyConfiguration::class],
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT,
)
class AccessLogWithBodyStrategyTest : AccessLogBaseTest() {

    @Test
    fun shouldLogRequestWithBody() {
        client.post()
            .uri("/discard")
            .bodyValue("Hello, world!")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()
        val list = util.captureAccessLogWrite()
        val requestAccessLog = list.first()
        requestAccessLog.method.shouldBeEqual("POST")
        requestAccessLog.body!!.shouldBeEqual("Hello, world!")
        requestAccessLog.uri.shouldContain("/discard")
    }
}

```

```

@Test
fun shouldLogRequestWithBodyButInExceptionMode() {
    client.post()
        .uri("/query/prj1/test")
        .bodyValue("Hello, world!")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()
    val list = util.captureAccessLogWrite()
    val requestAccessLog = list.first()
    requestAccessLog.method.shouldBeEqual("POST")
    requestAccessLog.body!!.shouldBeEqual("")
    requestAccessLog.uri.shouldContain("/query/prj1/test")
    list[1].body!!.shouldBeEqual("")
}

@Test
fun shouldLogRequestWithBodyGet() {
    client.get()
        .uri("/gettest")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()
    val (request, response) = util.captureResponse()
    request.method.shouldBeEqual("GET")
    request.requestUri.shouldContain("/gettest")
    response.bodyAsString.shouldBeEqual("Hello")
}

@Test
fun shouldLogRequestWithBodyPost() {
    client.post()
        .uri("/posttest")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()
    val list = util.captureAccessLogWrite()
    val requestAccessLog = list.first()
    requestAccessLog.method.shouldBeEqual("POST")
    requestAccessLog.uri.shouldContain("/posttest")
    list[1].body!!.shouldBeEqual("Hello, Post!")
}

@Test
fun testEndpointWithoutBody() {
    client.post()
        .uri("/withoutBodyEndpoint")
        .bodyValue("Hello withoutBodyEndpoint")
        .retrieve()
        .bodyToMono(String::class.java)
        .block()
    val list = util.captureAccessLogWrite()
    val requestAccessLog = list.first()
    requestAccessLog.method.shouldBeEqual("POST")
    requestAccessLog.body!!.shouldBeEqual("")
    requestAccessLog.uri.shouldContain("/withoutBodyEndpoint")
}

```

```

        list[1].body!!.shouldBeEqual("")
    }

    @Test
    fun shouldLogRequestWithoutBody() {
        util.sendAndReceive()
        val (request, response) = util.captureResponse()

        response.bodyAsString.shouldBeEqual("")
        request.method.shouldBeEqual("POST")

        request.requestUri.shouldBeEqual("http://localhost:$port/echo")
    }

    @Test
    fun shouldNotLogRequestIfInactive() {
        Mockito.`when`(sink.isActive).thenReturn(false)

        util.sendAndReceive()

        Mockito.verify<Sink>(sink, Mockito.never()).write(
            argumentCaptor<PreCorrelation>().capture(),
            argumentCaptor<HttpRequest>().capture(),
            Mockito.isNull()
        )
    }

    @Test
    fun shouldLogRejectedGetRequest() {
        shouldThrowExactly<WebClientResponseException.Unauthorized>
        {
            client.get()
                .uri("/rejected")
                .retrieve()
                .toBodilessEntity()
                .block()
        }

        val (request, response) = util.captureResponse()
        request.method.shouldBeEqual("GET")

        request.requestUri.shouldBeEqual("http://localhost:$port/rejected")

        response.bodyAsString.shouldBeEqual("")
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AccessLogWithoutBodyStrategyTest.kt
=====
package cloud.mws.devp.access.logging.spring.webflux.web.filter

import cloud.mws.devp.access.logging.AccessLogWriter

```



```

import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import cloud.mws.devp.access.logging.spring.webflux.TestApplication
import
cloud.mws.devp.access.logging.spring.webflux.common.PreCorrelation
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
import
cloud.mws.devp.access.logging.spring.webflux.common.http.HttpRequest
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithoutBodyStrategy
import io.kotest.matchers.equals.shouldBeEqual
import io.kotest.matchers.string.shouldContain
import org.junit.jupiter.api.Test
import org.mockito.Mockito
import org.mockito.junit.jupiter.MockitoExtension
import org.mockito.kotlin.argumentCaptor
import org.springframework.boot.test.context.SpringBootTest
import org.springframework.boot.test.context.TestConfiguration
import org.springframework.context.annotation.Bean
import org.springframework.context.annotation.Import
import org.springframework.core.env.Environment
import
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping
import org.springframework.web.server.WebFilter

@TestConfiguration
open class WithoutBodyConfiguration {
    // @MockitoSpyBean требуется бин для замены
https://docs.spring.io/spring-framework/reference/6.2.3/testing/annotations/integration-spring/annotation-mockitobean.html
    @Bean
    open fun loggerSink(accessLogWriter: AccessLogWriter) =
        LoggerSink(accessLogWriter)

    @Bean
    open fun requestProcessingStage(sink: Sink):
        RequestProcessingStage {
        return DefaultRequestProcessingStage(
            strategy = WithoutBodyStrategy(),
            sink = sink
        )
    }

    @Bean
    open fun filter(

```

```

        requestProcessingStage: RequestProcessingStage,
        requestMappingHandlerMapping: RequestMappingHandlerMapping,
        environment: Environment,
    ) = AccessLogWebFilter(requestProcessingStage,
requestMappingHandlerMapping, environment)

@Bean
open fun rejectingFilter(): WebFilter {
    return RejectingFilter()
}
}

@Import(MockitoExtension::class)
@SpringBootTest(
    classes = [TestApplication::class,
WithoutBodyConfiguration::class],
    webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT
)
class AccessLogWithoutBodyStrategyTest : AccessLogBaseTest() {
    @Test
    fun shouldLogRequestWithBody() {
        client.post()
            .uri("/discard")
            .bodyValue("Hello, world!")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()
        val list = util.captureAccessLogWrite()
        val requestAccessLog = list.first()
        requestAccessLog.method.shouldBeEqual("POST")
        requestAccessLog.body!!.shouldBeEqual("")
        requestAccessLog.uri.shouldContain("/discard")
    }

    @Test
    fun shouldLogRequestWithBodyGet() {
        client.get()
            .uri("/gettest")
            .retrieve()
            .bodyToMono(String::class.java)
            .block()
        val list = util.captureAccessLogWrite()
        var request = list[1]
        val response = list[1]
        request.method.shouldBeEqual("GET")
        request.uri.shouldContain("/gettest")
        response.body!!.shouldBeEqual("")
    }

    @Test
    fun shouldLogRequestWithoutBody() {
        util.sendAndReceive()
        val (request, response) = util.captureResponse()

        response.bodyAsString.shouldBeEqual("")
        request.method.shouldBeEqual("POST")
    }
}

```

```
request.requestUri.shouldBeEqual("http://localhost:$port/echo")
}
```

```
@Test
fun shouldNotLogRequestIfInactive() {
    Mockito.`when`(sink.isActive).thenReturn(false)

    util.sendAndReceive()

    Mockito.verify<Sink>(sink, Mockito.never()).write(
        argumentCaptor<PreCorrelation>().capture(),
        argumentCaptor<HttpRequest>().capture(),
        Mockito.isNull()
    )
}
}
```

=====

```
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-logging-spring-webflux/src/test/kotlin/cloud/mws/devp/access/logging/spring/webflux/web/filter/AutoMaskingTestConfig.kt =====
```

```
package cloud.mws.devp.access.logging.spring.webflux.web.filter
```

```
import cloud.mws.devp.access.logging.AccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessingStage
import cloud.mws.devp.access.logging.spring.webflux.common.Sink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySerializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.CompositeSensitiveMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultRequestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySerializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.SensitiveKeyMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodyStrategy
import cloud.mws.devp.cloud.sensitive.CloudSensitive
import cloud.mws.devp.cloud.sensitive.cloudSensitive
import com.fasterxml.jackson.core.JsonGenerator
import com.fasterxml.jackson.core.JsonParser
```

```

import com.fasterxml.jackson.databind.DeserializationContext
import com.fasterxml.jackson.databind.ObjectMapper
import com.fasterxml.jackson.databind.SerializerProvider
import com.fasterxml.jackson.databind.deser.std.StdDeserializer
import com.fasterxml.jackson.databind.module.SimpleModule
import com.fasterxml.jackson.databind.ser.std.StdSerializer
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper
import com.fasterxml.jackson.module.kotlin.registerKotlinModule
import
org.springframework.boot.autoconfigure.condition.ConditionalOnProperty
import org.springframework.boot.test.context.TestConfiguration
import org.springframework.boot.web.codec.CodecCustomizer
import org.springframework.context.annotation.Bean
import org.springframework.core.env.Environment
import org.springframework.http.MediaType
import org.springframework.http.codec.json.Jackson2JsonDecoder
import org.springframework.http.codec.json.Jackson2JsonEncoder
import
org.springframework.web.reactive.result.method.annotation.RequestMappingHandlerMapping

@TestConfiguration
open class AutoMaskingTestConfig {
    @Bean
    @ConditionalOnProperty(name =
["devp.api.access-log.body-serialization-mode"], havingValue =
"AUTO")
    open fun autoLoggerSink(accessLogWriter: AccessLogWriter) =
LoggerSink(
    accessLogWriter = accessLogWriter,
    bodySerializationStrategy = AutoBodySerializationStrategy(
        false,
        CompositeSensitiveMasker(
            jacksonObjectMapper(),
            SensitiveKeyMasker(jacksonObjectMapper())
        )
    )
)

    @Bean
    @ConditionalOnProperty(name =
["devp.api.access-log.body-serialization-mode"], havingValue =
"RAW")
    open fun rawLoggerSink(accessLogWriter: AccessLogWriter) =
LoggerSink(
    accessLogWriter = accessLogWriter,
    bodySerializationStrategy = RawBodySerializationStrategy()
)

    @Bean
    open fun requestProcessingStage(sink: Sink):
RequestProcessingStage {
        return DefaultRequestProcessingStage(
            strategy = WithBodyStrategy(emptyList()),
            sink = sink

```

```

    )
}

@Bean
open fun filter(
    requestProcessingStage: RequestProcessingStage,
    requestMappingHandlerMapping: RequestMappingHandlerMapping,
    environment: Environment
) = AccessLogWebFilter(requestProcessingStage,
requestMappingHandlerMapping, environment)

@Bean
open fun rejectingFilter() = RejectingFilter()

@Bean
open fun objectMapper(): ObjectMapper {
    return jacksonObjectMapper()
        .registerKotlinModule()
        .registerModule(createCloudSensitiveModule())
}

@Bean
open fun jsonCodecCustomizer(objectMapper: ObjectMapper):
CodecCustomizer {
    return CodecCustomizer { configurer ->
        configurer.defaultCodecs().jackson2JsonEncoder(
            Jackson2JsonEncoder(objectMapper,
MediaType.APPLICATION_JSON)
        )
        configurer.defaultCodecs().jackson2JsonDecoder(
            Jackson2JsonDecoder(objectMapper,
MediaType.APPLICATION_JSON)
        )
    }
}

private fun createCloudSensitiveModule(): SimpleModule {
    return SimpleModule().apply {
        addSerializer(object :
StdSerializer<CloudSensitive<*>>(CloudSensitive::class.java) {
            override fun serialize(value: CloudSensitive<*>,
gen: JsonGenerator, serializers: SerializerProvider) {
                gen.writeString(value.toString())
            }
        })

        addDeserializer(CloudSensitive::class.java, object :
StdDeserializer<CloudSensitive<*>>(CloudSensitive::class.java) {
            override fun deserialize(parser: JsonParser,
context: DeserializationContext): CloudSensitive<*> {
                val value = parser.text
                return value.cloudSensitive()
            }
        })
    }
}

```

}

Листинг Г.2 – Интеграционные тесты на сложных структурах

Test Summary

225 tests	0 failures	7 ignored	5.075s duration	100% successful
--------------	---------------	--------------	--------------------	--------------------

Ignored tests	Packages	Classes	Tests	Failures	Ignored	Duration	Success rate
		Class					
		cloud.mws.devdp.access.logging.spring.webflux.BufferingClientHttpRequestUnitTest	4	0	0	0.957s	100%
		cloud.mws.devdp.access.logging.spring.webflux.BufferingServerHttpResponseUnitTest	2	0	0	0.027s	100%
		cloud.mws.devdp.access.logging.spring.webflux.ClientRequestUnitTest	2	0	0	0.012s	100%
		cloud.mws.devdp.access.logging.spring.webflux.ContentTypeTest	41	0	0	0.042s	100%
		cloud.mws.devdp.access.logging.spring.webflux.HttpAttributeTest	3	0	0	0.001s	100%
		cloud.mws.devdp.access.logging.spring.webflux.HttpHeaderTest	3	0	0	0.077s	100%
		cloud.mws.devdp.access.logging.spring.webflux.RequestURITest	11	0	0	0.023s	100%
		cloud.mws.devdp.access.logging.spring.webflux.ServerRequestInitTest	1	0	0	0.027s	100%
		cloud.mws.devdp.access.logging.spring.webflux.StateTest	1	0	0	0.002s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CloudSensitiveMaskerPerformanceTest	7	0	7	0s	-
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CloudSensitiveMaskerResolvableTypeTest	45	0	0	1.109s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CloudSensitiveMaskerTest	42	0	0	0.238s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CompositeSensitiveMaskerCoWTest\$CachingTests	1	0	0	0.040s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CompositeSensitiveMaskerCoWTest\$CloudSensitiveBasedMaskingTests	3	0	0	0.007s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CompositeSensitiveMaskerCoWTest\$HybridMaskingTests	6	0	0	0.299s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CompositeSensitiveMaskerCoWTest\$KeyBasedMaskingTests	3	0	0	0.002s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.CompositeSensitiveMaskerCoWTest\$MinimalOverheadTests	3	0	0	0.002s	100%
		cloud.mws.devdp.access.logging.spring.webflux.common.impl.SensitiveKeyMaskingTest	9	0	0	0.094s	100%
		cloud.mws.devdp.access.logging.spring.webflux.exchange.filter.AccessLogClientAutoMaskingTest	1	0	0	1.476s	100%
		cloud.mws.devdp.access.logging.spring.webflux.exchange.filter.AccessLogClientRawModeTest	1	0	0	0.045s	100%
		cloud.mws.devdp.access.logging.spring.webflux.exchange.filter.AccessLogExchangeFilterFunctionBaseTest	13	0	0	0.249s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogAutoMaskingTest	6	0	0	0.167s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogHeadersWhitelistTest	1	0	0	0.025s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogRawModeTest	1	0	0	0.015s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogSensitiveKeyMaskingTest	3	0	0	0.036s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogWithBodyStrategyTest	8	0	0	0.075s	100%
		cloud.mws.devdp.access.logging.spring.webflux.web.filter.AccessLogWithoutBodyStrategyTest	4	0	0	0.028s	100%

Рисунок Г.3 – Скриншот прохождения тестов

ПРИЛОЖЕНИЕ Д

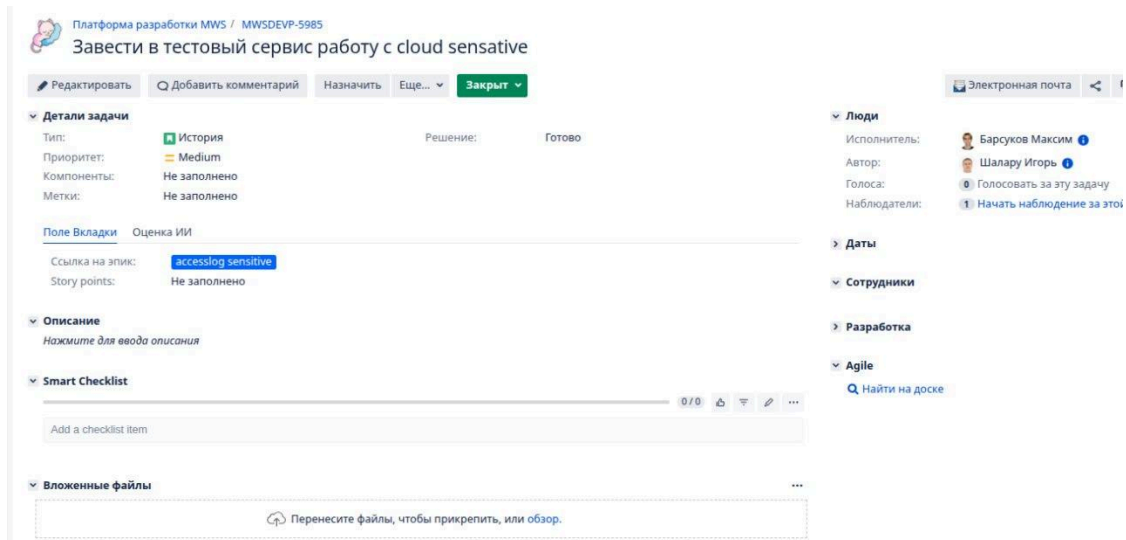


Рисунок Д.1 – Задача MWSDEVP-5985

```
diff --git a/mws/devp/testService/src/openapi/openapi.yaml
b/mws/devp/testService/src/openapi/openapi.yaml
index
f689fe25916b30e32ba458fc16cc069137d721db..816af230e446d15e8207179d1
b0b1bd62871ce85 100644
--- a/mws/devp/testService/src/openapi/openapi.yaml
+++ b/mws/devp/testService/src/openapi/openapi.yaml
@@ -1253,3 +1253,115 @@ paths:
         type: string
         status:
         $ref:
"../../../../common/src/openapi/schemas/ResourceStatus.yaml"
+ /cloudSensitive/list:
+   get:
+     tags:
+       - cloudSensitive
+     summary: Get list of cloud sensitive objects
+     operationId: getCloudSensitiveList
+     x-common-errors: true
+     responses:
+       '200':
+         description: Success
+         content:
+           application/json:
+             schema:
+               type: array
+               items:
+                 $ref: "../../../../resources/CloudSensitiveObject.yaml"
+       '500':
+         $ref:
```

```

"../../../../../../common/src/openapi/schemas/errors/commonErrorsForGetDe
lete.yaml#/500"
+   /cloudSensitive:
+     get:
+       tags:
+         - cloudSensitive
+       summary: Get cloud sensitive object with body in request
+       (GET with body)
+       operationId: getCloudSensitiveWithBody
+       x-common-errors: true
+       requestBody:
+         content:
+           application/json:
+             schema:
+               $ref: "../resources/CloudSensitiveRequestBody.yaml"
+       responses:
+         '200':
+           description: Success
+           content:
+             application/json:
+               schema:
+                 $ref: "../resources/CloudSensitiveObject.yaml"
+         '500':
+           $ref:
"../../../../../../common/src/openapi/schemas/errors/commonErrorsForGetDe
lete.yaml#/500"
+     post:
+       tags:
+         - cloudSensitive
+       summary: POST cloud sensitive object with body in request
+       and response
+       operationId: postCloudSensitive
+       x-common-errors: true
+       requestBody:
+         content:
+           application/json:
+             schema:
+               $ref: "../resources/CloudSensitiveRequestBody.yaml"
+       responses:
+         '200':
+           description: Success
+           content:
+             application/json:
+               schema:
+                 $ref: "../resources/CloudSensitiveRequestBody.yaml"
+         '500':
+           $ref:
"../../../../../../common/src/openapi/schemas/errors/commonErrorsForPost.
yaml#/500"
+   /cloudSensitive/{objectId}:
+     get:
+       tags:
+         - cloudSensitive
+       summary: Get cloud sensitive object (GET with body in
+       response)
+       operationId: getCloudSensitive

```



```

+       x-common-errors: true
+       parameters:
+         - name: objectId
+           in: path
+           schema:
+             type: string
+             required: true
+       responses:
+         '200':
+           description: Success
+           content:
+             application/json:
+               schema:
+                 $ref: "../resources/CloudSensitiveObject.yaml"
+         '500':
+           $ref:
+             "../.../.../.../common/src/openapi/schemas/errors/commonErrorsForGetDelete.yaml#/500"
+       post:
+         tags:
+           - cloudSensitive
+         x-upsert: true
+         x-common-errors: true
+         x-optional-response: true
+         summary: Upsert cloud sensitive object
+         operationId: upsertCloudSensitive
+         parameters:
+           - $ref:
+               "../.../.../.../common/src/openapi/parameters/list.yaml#/createOnly"
+           - $ref:
+               "../.../.../.../common/src/openapi/parameters/list.yaml#/updateOnly"
+           - $ref:
+               "../.../.../.../common/src/openapi/parameters/list.yaml#/validateOnly"
+           - name: objectId
+             in: path
+             schema:
+               type: string
+               required: true
+         requestBody:
+           content:
+             application/json:
+               schema:
+                 $ref: "../resources/CloudSensitiveRequestBody.yaml"
+         responses:
+           '200':
+             description: Success
+             content:
+               application/json:
+                 schema:
+                   $ref: "../resources/CloudSensitiveRequestBody.yaml"
+           '500':
+             $ref:
+               "../.../.../.../common/src/openapi/schemas/errors/commonErrorsForPost.yaml#/500"
diff --git
a/mws/devp/testService/src/openapi/resources/CloudSensitiveObject.y

```



```
+           type: string
+           x-sensitive: true
diff --git
a/mws/devp/testService/src/openapi/resources/CloudSensitiveRequestB
ody.yaml
b/mws/devp/testService/src/openapi/resources/CloudSensitiveRequestB
ody.yaml
new file mode 100644
index
00000000000000000000000000000000..c867626e5e89adfc42805fc3
441e13ff9334c29
--- /dev/null
+++
b/mws/devp/testService/src/openapi/resources/CloudSensitiveRequestB
ody.yaml
@@ -0,0 +1,22 @@
+name: CloudSensitiveRequestBody
+type: object
+required:
+  - spec
+properties:
+  kind:
+    type: string
+    readOnly: true
+    default: "devp/v1/cloudSensitive"
+  metadata:
+    $ref:
+"../../../../../../common/src/openapi/schemas/CommonTypedResourceMetad
ata.yaml"
+  spec:
+    $ref: "../CloudSensitiveObject.yaml"
+  status:
+    type: object
+    readOnly: true
+    allOf:
+      - $ref:
+"../../../../../../common/src/openapi/schemas/ResourceStatus.yaml"
+      - $ref: "../CloudSensitiveObject.yaml"
```

Листинг Д.2 – OpenAPI-спецификация новых endpoint в тестовом сервисе
(без учета сгенерированных файлов)


```

CloudSensitive("nested-secret-$objectId")
+
+         ),
+         sensitiveInArrayOfObjects = listOf(
+
+ CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+         name = "item1",
+         secret = CloudSensitive("item-secret1")
+         ),
+
+ CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+         name = "item2",
+         secret = CloudSensitive("item-secret2")
+         )
+     )
+ )
+ )
+ }
+
+ override suspend fun getCloudSensitiveWithBody(
+     request: CloudSensitiveRequestBodyRequest
+ ): ResponseEntity<CloudSensitiveObjectResponse> {
+     val id = request.metadata?.name ?: "unknown"
+
+     return ResponseEntity.ok(
+         CloudSensitiveObjectResponse(
+             regularField = request.spec.regularField ?:
"regular-from-request-$id",
+             sensitiveField = request.spec.sensitiveField ?:
CloudSensitive("secret-from-request-$id"),
+             requiredSensitiveField =
request.spec.requiredSensitiveField,
+             sensitiveArray = request.spec.sensitiveArray ?:
listOf(
+                 CloudSensitive("default-secret1"),
+                 CloudSensitive("default-secret2")
+             ),
+             sensitiveMap = request.spec.sensitiveMap ?: mapOf(
+                 "default-key" to
CloudSensitive("default-value")
+             ),
+             nestedObject = request.spec.nestedObject?.let {
nested ->
+                 CloudSensitiveObjectNestedObjectResponse(
+                     nestedRegularField =
nested.nestedRegularField ?: "nested-regular-from-request",
+                     nestedSensitiveField =
nested.nestedSensitiveField
+                     ?:
CloudSensitive("nested-secret-from-request")
+                 )
+             } ?: CloudSensitiveObjectNestedObjectResponse(
+                 nestedRegularField = "default-nested-regular",
+                 nestedSensitiveField =
CloudSensitive("default-nested-secret")
+             ),
+             sensitiveInArrayOfObjects =

```

```

request.spec.sensitiveInArrayOfObjects?.map { item ->
+
+ CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+         name = item.name ?: "default-item-name",
+         secret = item.secret ?:
+ CloudSensitive("default-item-secret")
+     )
+     } ?: listOf(
+
+ CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+         name = "default-item1",
+         secret = CloudSensitive("default-secret1")
+     ),
+
+ CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+         name = "default-item2",
+         secret = CloudSensitive("default-secret2")
+     )
+     )
+ )
+ }
+
+ override suspend fun postCloudSensitive(
+     request: CloudSensitiveRequestBodyRequest
+ ): ResponseEntity<CloudSensitiveRequestBodyResponse> {
+     val id = request.metadata?.name ?: "unknown"
+
+     return ResponseEntity.ok(
+         CloudSensitiveRequestBodyResponse(
+             kind = "CloudSensitive",
+             metadata = CommonTypedResourceMetadataResponse(
+                 name = id,
+                 etag = "etag-#{System.currentTimeMillis()}",
+                 displayName = "CloudSensitive Object $id",
+                 labels = mapOf("environment" to "test"),
+                 annotations = mapOf("created-by" to
+ "test-service")
+             ),
+             spec = CloudSensitiveObjectResponse(
+                 regularField = request.spec.regularField ?:
+ "regular-from-post-$id",
+                 sensitiveField = request.spec.sensitiveField
+ ?: CloudSensitive("secret-from-post-$id"),
+                 requiredSensitiveField =
+ request.spec.requiredSensitiveField,
+                 sensitiveArray = request.spec.sensitiveArray
+ ?: listOf(
+                     CloudSensitive("post-secret1"),
+                     CloudSensitive("post-secret2")
+                 ),
+                 sensitiveMap = request.spec.sensitiveMap ?:
+ mapOf(
+                     "post-key" to CloudSensitive("post-value")
+                 ),
+                 nestedObject = request.spec.nestedObject?.let

```

```

{ nested ->
+             CloudSensitiveObjectNestedObjectResponse(
+                 nestedRegularField =
nested.nestedRegularField ?: "nested-regular-post",
+                 nestedSensitiveField =
nested.nestedSensitiveField
+                 ?:
CloudSensitive("nested-secret-post")
+             )
+             },
+             sensitiveInArrayOfObjects =
request.spec.sensitiveInArrayOfObjects?.map { item ->
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+                 name = item.name,
+                 secret = item.secret
+             )
+         }
+     ),
+     status = CloudSensitiveRequestBodyStatusResponse(
+         ready = ResourceStatusReadyResponse(state =
OK),
+         regularField = request.spec.regularField,
+         sensitiveField = request.spec.sensitiveField,
+         requiredSensitiveField =
request.spec.requiredSensitiveField,
+         sensitiveArray = request.spec.sensitiveArray,
+         sensitiveMap = request.spec.sensitiveMap,
+         nestedObject = request.spec.nestedObject?.let
{ nested ->
+             CloudSensitiveObjectNestedObjectResponse(
+                 nestedRegularField =
nested.nestedRegularField,
+                 nestedSensitiveField =
nested.nestedSensitiveField
+             )
+             },
+             sensitiveInArrayOfObjects =
request.spec.sensitiveInArrayOfObjects?.map { item ->
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+                 name = item.name,
+                 secret = item.secret
+             )
+         }
+     )
+ }
+
+     override suspend fun getCloudSensitiveList():
ResponseEntity<List<CloudSensitiveObjectResponse>> {
+         val items = listOf(
+             CloudSensitiveObjectResponse(
+                 regularField = "regular-1",
+                 sensitiveField = CloudSensitive("secret-1"),

```

```

+         requiredSensitiveField =
CloudSensitive("required-secret-1"),
+         sensitiveArray = listOf(CloudSensitive("a1"),
CloudSensitive("b1")),
+         sensitiveMap = mapOf("k1" to
CloudSensitive("v1")),
+         nestedObject =
CloudSensitiveObjectNestedObjectResponse(
+             nestedRegularField = "nested-1",
+             nestedSensitiveField =
CloudSensitive("nested-secret-1")
+         ),
+         sensitiveInArrayOfObjects = listOf(
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+             "item1-1",
+             CloudSensitive("secret1-1")
+         ),
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+             "item1-2",
+             CloudSensitive("secret1-2")
+         )
+     )
+ ),
+     CloudSensitiveObjectResponse(
+         regularField = "regular-2",
+         sensitiveField = CloudSensitive("secret-2"),
+         requiredSensitiveField =
CloudSensitive("required-secret-2"),
+         sensitiveArray = listOf(CloudSensitive("a2"),
CloudSensitive("b2")),
+         sensitiveMap = mapOf("k2" to
CloudSensitive("v2")),
+         nestedObject =
CloudSensitiveObjectNestedObjectResponse(
+             nestedRegularField = "nested-2",
+             nestedSensitiveField =
CloudSensitive("nested-secret-2")
+         ),
+         sensitiveInArrayOfObjects = listOf(
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+             "item2-1",
+             CloudSensitive("secret2-1")
+         ),
+
CloudSensitiveObjectSensitiveInArrayOfObjectsResponse(
+             "item2-2",
+             CloudSensitive("secret2-2")
+         )
+     )
+ )
+ )
+ return ResponseEntity.ok(items)
+ }

```



```

+
+   override suspend fun upsertCloudSensitive(
+       objectId: String,
+       createOnly: Boolean?,
+       updateOnly: Boolean?,
+       validateOnly: Boolean?,
+       request: CloudSensitiveRequestBodyUpsertRequest
+   ): ResponseEntity<CloudSensitiveRequestBodyOptionalResponse> {
+       val spec = request.spec.get() ?: return
+       ResponseEntity.badRequest().build()
+
+       val metadata = request.metadata.get()?.let { meta ->
+           CommonTypedResourceMetadataOptionalResponse(
+               name = meta.name.get().orEmpty().cloudOptional(),
+               etag = meta.etag.get().orEmpty().cloudOptional(),
+               displayName =
+               meta.displayName.get().orEmpty().cloudOptional(),
+               labels =
+               meta.labels.get().orEmpty().cloudOptional(),
+               annotations =
+               meta.annotations.get().orEmpty().cloudOptional()
+           )
+       }
+
+       val specResponse = CloudSensitiveObjectOptionalResponse(
+           regularField =
+           spec.regularField.get().cloudOptional(),
+           sensitiveField =
+           spec.sensitiveField.get().cloudOptional(),
+           requiredSensitiveField = CloudOptional.of(
+               spec.requiredSensitiveField.get() ?:
+               CloudSensitive("default-required-$objectId"),
+               isRequired = true
+           ),
+           sensitiveArray =
+           spec.sensitiveArray.get().cloudOptional(),
+           sensitiveMap =
+           spec.sensitiveMap.get().cloudOptional(),
+           nestedObject = spec.nestedObject.get()?.let { nested
+       ->
+           CloudOptional.of(
+               CloudSensitiveObjectNestedObjectOptionalResponse(
+                   nestedRegularField =
+                   nested.nestedRegularField.get().cloudOptional(),
+                   nestedSensitiveField =
+                   nested.nestedSensitiveField.get().cloudOptional()
+               )
+           )
+       } ?: CloudOptional.empty(),
+           sensitiveInArrayOfObjects =
+           spec.sensitiveInArrayOfObjects.get()?.let { list ->
+               CloudOptional.of(
+                   list.map { item ->
+                       item.let {

```

```

CloudSensitiveObjectSensitiveInArrayOfObjectsOptionalResponse(
+
+           name =
it.name.get().cloudOptional(),
+
+           secret =
it.secret.get().cloudOptional()
+
+       )
+
+   }
+
+   )
+
+   } ?: CloudOptional.empty()
+
+ )
+
+
+
+
+   return ResponseEntity.ok(
+       CloudSensitiveRequestBodyOptionalResponse(
+           kind = "CloudSensitive",
+           metadata = metadata.cloudOptional(),
+           spec = specResponse.cloudOptional(isRequired =
true),
+           status = null
+       )
+   )
+
+ }
+}
diff --git
a/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/HelloReactiveApiController.kt
b/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/HelloReactiveApiController.kt
index
896fcfe87c4a7db4a2b39da8a881e1246a2bb1ee..17691d8eef5b3f7842f6ec218
6044672c7315c99 100644
---
a/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/HelloReactiveApiController.kt
+++
b/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/HelloReactiveApiController.kt
@@ -21,7 +21,7 @@ import
cloud.mws.api.devp.examples.dto.PropertyCheckObjectBodyUpsertReques
t
import cloud.mws.api.devp.examples.dto.PropertyCheckObjectResponse
import cloud.mws.api.devp.examples.dto.RequestBodyRequest
import cloud.mws.api.devp.examples.server.HelloReactiveApi
-import cloud.mws.common.dto.ResourceMetadataResponse
+import cloud.mws.common.dto.CommonTypedResourceMetadataResponse
import cloud.mws.common.dto.ResourceStatusReadyResponse
import cloud.mws.common.dto.type.ResourceStatusState.OK
import
cloud.mws.devp.testservice.controller.converters.toResponseList
@@ -77,7 +77,7 @@ open class HelloReactiveApiController(
): ResponseEntity<PropertyCheckObjectBodyResponse> {
    val response = PropertyCheckObjectBodyResponse(
        kind = "notNullable",
-        metadata = ResourceMetadataResponse(name =
"response"),
+        metadata = CommonTypedResourceMetadataResponse(name =

```

```

"response"),
    spec = PropertyCheckObjectResponse(
        nullNotRequiredNotNullable = "notNullable",
        nullNotRequiredNullable = "nullable",
diff --git
a/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/converters/ModelConverter.kt
b/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/converters/ModelConverter.kt
index
996b8f28a8ald2179296adda3c1977b14d00471b..45102b2028f9d917c912a6f4f
4e7dc6357415be6 100644
---
a/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/converters/ModelConverter.kt
+++
b/api-hello-server/src/main/kotlin/cloud/mws/devp/testservice/contr
oller/converters/ModelConverter.kt
@@ -16,7 +16,7 @@ import
cloud.mws.api.devp.examples.dto.ModelWithReferenceRequest
import cloud.mws.api.devp.examples.dto.ModelWithReferenceResponse
import
cloud.mws.api.devp.examples.dto.ModelWithValueObjectsRequest
import
cloud.mws.api.devp.examples.dto.ModelWithValueObjectsResponse
- import cloud.mws.common.dto.ResourceMetadataResponse
+ import cloud.mws.common.dto.CommonTypedResourceMetadataResponse
import cloud.mws.common.dto.ResourceStatusReadyResponse
import cloud.mws.common.dto.TypedResourceMetadataResponse
import cloud.mws.common.dto.UsageResponse
@@ -49,7 +49,7 @@ fun
ModelWithReferenceBodyUpsertRequest.toResponse():
ModelWithReferenceBodyResp
    ModelWithReferenceBodyResponse(
        kind = "ModelWithReferenceBodyResponse",
        metadata = metadata.get()?.let { requestMeta ->
-            ResourceMetadataResponse(
+            CommonTypedResourceMetadataResponse(
                name = requestMeta.name.get().orEmpty(),
                etag = requestMeta.etag.get().orEmpty(),
                displayName =
requestMeta.displayName.get().orEmpty(),
@@ -63,7 +63,7 @@ fun
ModelWithReferenceBodyUpsertRequest.toResponse():
ModelWithReferenceBodyResp
        )
    },
    )
-    } ?: ResourceMetadataResponse(name = "undefined"),
+    } ?: CommonTypedResourceMetadataResponse(name =
"undefined"),
        spec = spec.get()?.let { requestSpec ->
            ModelWithReferenceResponse(
                exactReference = requestSpec.exactReference.get(),
diff --git
a/api/src/main/kotlin/cloud/mws/devp/testservice/controller/CloudSe

```

```
nsitiveController.kt
b/api/src/main/kotlin/cloud/mws/devp/testservice/controller/CloudSe
nsitiveController.kt
new file mode 100644
index
0000000000000000000000000000000000000000..fdd7632d060768d9495370e19
0b3cd78b64b3b6a
--- /dev/null
+++
b/api/src/main/kotlin/cloud/mws/devp/testservice/controller/CloudSe
nsitiveController.kt
@@ -0,0 +1,77 @@
+package cloud.mws.devp.testservice.controller
+
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectResponse
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveRequestBodyResponse
+import cloud.mws.devp.testservice.services.CloudSensitiveService
+import org.springframework.http.ResponseEntity
+import org.springframework.web.bind.annotation.*
+
+@RestController
+@RequestMapping("${api.base-path}/test/cloud-sensitive")
+class CloudSensitiveController(
+    private val clientService: CloudSensitiveService
+) {
+
+    @GetMapping("/client/get/{objectId}")
+    suspend fun testGetCloudSensitive(
+        @PathVariable objectId: String
+    ): ResponseEntity<CloudSensitiveObjectResponse> {
+        val result = clientService.getCloudSensitive(objectId)
+        return ResponseEntity.ok(result)
+    }
+
+    @GetMapping("/client/get-with-body")
+    suspend fun testGetCloudSensitiveWithBody(
+        @RequestParam id: String,
+        @RequestParam(required = false) regularField: String?,
+        @RequestParam(required = false) sensitiveValue: String?
+    ): ResponseEntity<CloudSensitiveObjectResponse> {
+        val result = clientService.getCloudSensitiveWithBody(id,
regularField, sensitiveValue)
+        return ResponseEntity.ok(result)
+    }
+
+    @PostMapping("/client/post")
+    suspend fun testPostCloudSensitive(
+        @RequestParam id: String,
+        @RequestParam(required = false) regularField: String?,
+        @RequestParam(required = false) sensitiveValue: String?
+    ): ResponseEntity<CloudSensitiveRequestBodyResponse> {
+        val result = clientService.postCloudSensitive(id,
regularField, sensitiveValue)
+        return ResponseEntity.ok(result)
+    }
+}
```

```

+     }
+
+     @GetMapping("/client/list")
+     suspend fun testGetCloudSensitiveList():
ResponseEntity<List<CloudSensitiveObjectResponse>> {
+         val result = clientService.getCloudSensitiveList()
+         return ResponseEntity.ok(result)
+     }
+
+     @GetMapping("/client/full-cycle/{id}")
+     suspend fun testFullCycle(@PathVariable id: String):
ResponseEntity<Map<String, Any>> {
+         val result = clientService.fullCycle(id)
+         return ResponseEntity.ok(result)
+     }
+
+     @GetMapping("/client/test-all")
+     suspend fun testAllCases(): ResponseEntity<Map<String, Any>> {
+         val timestamp = System.currentTimeMillis()
+         val baseId = "test-$timestamp"
+
+         val results = mapOf(
+             "case1_get_by_id" to
clientService.getCloudSensitive("static-id-1"),
+             "case2_get_with_body" to
clientService.getCloudSensitiveWithBody(
+                 id = "$baseId-get-body",
+                 regularField = "test-regular",
+                 sensitiveValue = "test-secret"
+             ),
+             "case3_post" to clientService.postCloudSensitive(
+                 id = "$baseId-post",
+                 regularField = "post-regular",
+                 sensitiveValue = "post-secret"
+             ),
+             "case4_list" to clientService.getCloudSensitiveList()
+         )
+
+         return ResponseEntity.ok(results)
+     }
+}
diff --git a/app/src/main/resources/application.yaml
b/app/src/main/resources/application.yaml
index
82ca9fb4019d0ca1e0c8d42d4fe7861e76101475..4656864594fb7e616028e309d
5ebc6a799c7cdc3 100644
--- a/app/src/main/resources/application.yaml
+++ b/app/src/main/resources/application.yaml
@@ -23,9 +23,6 @@ server:
    port: ${MWS_DEVP_API_TEST_PORT:8888}

    devp:
-    r2dbc:
-    migrations:
-    enabled: true
    api:

```

```
error:  
    handler:  
@@ -34,4 +31,10 @@ devp:  
    inmemory:  
        enabled: true  
validation:  
-     enabled: true  
\ No newline at end of file  
+     enabled: true  
+ access-log:  
+     strategy: WITH_BODY  
+     client-log-enabled: true  
+     server-log-enabled: true  
+     body-serialization-mode: AUTO  
+     suppress-unsupported-types: false  
diff --git  
a/app/src/test/kotlin/cloud/mws/devp/testservice/CloudSensitiveTest  
.kt  
b/app/src/test/kotlin/cloud/mws/devp/testservice/CloudSensitiveTest  
.kt  
new file mode 100644  
index  
00000000000000000000000000000000..4b006988838faf37f5944c17f  
57df6d7f5d7081c  
--- /dev/null  
+++  
b/app/src/test/kotlin/cloud/mws/devp/testservice/CloudSensitiveTest  
.kt  
@@ -0,0 +1,163 @@  
+package cloud.mws.devp.testservice  
+  
+import  
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectResponse  
+import  
cloud.mws.api.devp.examples.dto.CloudSensitiveRequestBodyResponse  
+import cloud.mws.devp.cloud.sensitive.CloudSensitive  
+import org.junit.jupiter.api.Assertions.assertEquals  
+import org.junit.jupiter.api.Assertions.assertNotNull  
+import org.junit.jupiter.api.Assertions.assertTrue  
+import org.junit.jupiter.api.Disabled  
+import org.junit.jupiter.api.DisplayName  
+import org.junit.jupiter.api.Test  
+import org.springframework.beans.factory.annotation.Autowired  
+import  
org.springframework.boot.test.autoconfigure.web.reactive.AutoConfig  
ureWebTestClient  
+import org.springframework.test.context.ActiveProfiles  
+import org.springframework.test.web.reactive.server.WebTestClient  
+import org.springframework.test.web.reactive.server.expectBody  
+import kotlin.test.assertFalse  
+  
+// TODO Enable when INCOMING cloud sensitive is fixed  
+@Disabled  
+@AutoConfigureWebTestClient  
+@ActiveProfiles("test")  
+class CloudSensitiveTest : BaseApiServerMockTest() {
```

```

+
+     @Autowired
+     private lateinit var webTestClient: WebTestClient
+
+     @Test
+     @DisplayName("(сервер) GET + CloudSensitive в body response")
+     fun testGetCloudSensitive() {
+         webTestClient.get()
+             .uri("/cloudSensitive/test-id-1")
+             .exchange()
+             .expectStatus().isOk
+             .expectBody<CloudSensitiveObjectResponse>()
+             .consumeWith { response ->
+                 val body = response.responseBody
+                 assertNotNull(body)
+                 assertEquals("regular-value-test-id-1",
body?.regularField)
+
+                 assertEquals(CloudSensitive("secret-value-test-id-1"),
body?.sensitiveField)
+
+                 assertNotNull(body?.sensitiveArray)
+                 assertEquals(2, body?.sensitiveArray?.size)
+                 assertNotNull(body?.nestedObject)
+                 assertEquals("nested-regular-test-id-1",
body?.nestedObject?.nestedRegularField)
+             }
+     }
+
+     @Test
+     @DisplayName("(клиент) GET + CloudSensitive в body response")
+     fun testClientGetCloudSensitive() {
+         webTestClient.get()
+             .uri("/test/cloud-sensitive/client/get/client-test-id-1")
+             .exchange()
+             .expectStatus().isOk
+             .expectBody<CloudSensitiveObjectResponse>()
+             .consumeWith { response ->
+                 val body = response.responseBody
+                 assertNotNull(body)
+                 assertEquals("regular-value-client-test-id-1",
body?.regularField)
+
+                 assertEquals(CloudSensitive("secret-value-client-test-id-1"),
body?.sensitiveField)
+             }
+     }
+
+     @Test
+     @DisplayName("(клиент): GET + CloudSensitive в body request")
+     fun testGetCloudSensitiveWithBody() {
+         webTestClient.get()
+             .uri { builder ->
+                 builder
+
+             }
+             .path("/test/cloud-sensitive/client/get-with-body")

```

```

+             .queryParams("id", "body-test-id")
+             .queryParams("regularField", "custom-regular")
+             .queryParams("sensitiveValue", "custom-secret")
+             .build()
+         }
+         .exchange()
+         .expectStatus().isOk
+         .expectBody<CloudSensitiveObjectResponse>()
+         .consumeWith { response ->
+             val body = response.responseBody
+             assertNotNull(body)
+             assertEquals("custom-regular", body?.regularField)
+             assertEquals(CloudSensitive("custom-secret"),
body?.sensitiveField)
+         }
+     }
+
+     @Test
+     @DisplayName("(клиент): POST + CloudSensitive в body request и
response")
+     fun testPostCloudSensitive() {
+         webTestClient.post()
+             .uri { builder ->
+                 builder
+                     .path("/test/cloud-sensitive/client/post")
+                     .queryParams("id", "post-test-id")
+                     .queryParams("regularField", "post-regular")
+                     .queryParams("sensitiveValue", "post-secret")
+                     .build()
+             }
+         .exchange()
+         .expectStatus().isOk
+         .expectBody<CloudSensitiveRequestBodyResponse>()
+         .consumeWith { response ->
+             val body = response.responseBody
+             assertNotNull(body)
+             assertEquals("post-test-id", body?.metadata?.name)
+             assertEquals("post-regular",
body?.spec?.regularField)
+             assertEquals(CloudSensitive("post-secret"),
body?.spec?.sensitiveField)
+             assertNotNull(body?.status)
+             assertEquals(CloudSensitive("post-secret"),
body?.status?.sensitiveField)
+         }
+     }
+
+     @Test
+     @DisplayName("(клиент): List<> с CloudSensitive в response")
+     fun testGetCloudSensitiveList() {
+         webTestClient.get()
+             .uri("/test/cloud-sensitive/client/list")
+             .exchange()
+             .expectStatus().isOk
+             .expectBody()
+             .jsonPath("$.length()").isEqualTo(3)

```



```

+         .jsonPath("$.regularField").isEqualTo("regular-1")
+         .jsonPath("$.regularField").isEqualTo("regular-2")
+         .jsonPath("$.regularField").isEqualTo("regular-3")
+         .jsonPath("$.sensitiveField.value").value<String> {
+             assertFalse(it.contains("secret"))
+         }
+     }
+
+     @Test
+     fun testFullCycle() {
+         webTestClient.get()
+
+         .uri("/test/cloud-sensitive/client/full-cycle/cycle-test-id")
+         .exchange()
+         .expectStatus().isOk
+         .expectBody<Map<String, Any>>()
+         .consumeWith { response ->
+             val body = response.responseBody
+             assertNotNull(body)
+             assertTrue(body?.containsKey("created") == true)
+             assertTrue(body?.containsKey("retrieved") == true)
+             assertTrue(body?.containsKey("retrievedWithBody")
+ == true)
+         }
+     }
+
+     @Test
+     fun testAllCases() {
+         webTestClient.get()
+         .uri("/test/cloud-sensitive/client/test-all")
+         .exchange()
+         .expectStatus().isOk
+         .expectBody<Map<String, Any>>()
+         .consumeWith { response ->
+             val body = response.responseBody
+             assertNotNull(body)
+             assertEquals(4, body?.size)
+             assertTrue(body?.containsKey("case1_get_by_id") ==
true)
+             assertTrue(body?.containsKey("case2_get_with_body") == true)
+             assertTrue(body?.containsKey("case3_post") ==
true)
+             assertTrue(body?.containsKey("case4_list") ==
true)
+         }
+     }
+ }
+ }
diff --git
a/app/src/test/kotlin/cloud/mws/devp/testservice/TestApplicationTes
t.kt
b/app/src/test/kotlin/cloud/mws/devp/testservice/TestApplicationTes
t.kt
index
49f28c2ef1abc5841ae5f1e49b038c926e3696ba..07d7bba0049c1bfaff091421f
5da04de2152589a 100644

```

```

---
a/app/src/test/kotlin/cloud/mws/devp/testservice/TestApplicationTes
t.kt
+++
b/app/src/test/kotlin/cloud/mws/devp/testservice/TestApplicationTes
t.kt
@@ -2,22 +2,17 @@ package cloud.mws.devp.testservice

import org.junit.jupiter.api.DisplayName
import org.junit.jupiter.api.Test
- import org.junit.jupiter.api.extension.ExtendWith
import org.springframework.beans.factory.annotation.Autowired
import
org.springframework.boot.test.autoconfigure.web.reactive.AutoConfig
ureWebTestClient
- import org.springframework.boot.test.context.SpringBootTest
import org.springframework.test.context.ActiveProfiles
- import
org.springframework.test.context.junit.jupiter.SpringExtension
import org.springframework.test.web.reactive.server.WebTestClient

/**
 * Тестирование контроллера серверного API.
 */
- @ExtendWith(SpringExtension::class)
- @SpringBootTest
@AutoConfigureWebTestClient
@ActiveProfiles("test")
- class TestApplicationTest {
+ class TestApplicationTest : BaseApiServerMockTest() {

    @Autowired
    private lateinit var webTestClient: WebTestClient
diff --git
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiCli
entMockTest.kt
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiCli
entMockTest.kt
index
de1e671220b1b4d12e5a194f2e3329b0523964f3..87addda373dfccac13d2e12fc
d0240be84519706 100644
---
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiCli
entMockTest.kt
+++
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiCli
entMockTest.kt
@@ -29,7 +29,7 @@ import kotlin.test.assertEquals
class HelloApiClientMockTest : BaseApiServerMockTest() {

    @Autowired
-    @Qualifier("HelloApiClientZone1")
+    @Qualifier("devp.test-service.HelloApiClientZone1")
    private lateinit var helloApiClient: HelloApiClient

    @Test

```

```

diff --git
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientRetryTest.kt
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientRetryTest.kt
index
65c13a81c2682432908449f984306d4a7a43bb91..570c236df160ad2a60577ad41e4b22005a8b3cd0 100644
---
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientRetryTest.kt
+++
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientRetryTest.kt
@@ -25,7 +25,7 @@ import kotlin.test.assertEquals
class HelloApiClientRetryTest : BaseApiServerMockTest() {

    @Autowired
-    @Qualifier("HelloApiClientZone1")
+    @Qualifier("devp.test-service.HelloApiClientZone1")
    private lateinit var helloApiClient: HelloApiClient

    @Test
diff --git
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientZonesTest.kt
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientZonesTest.kt
index
b761462fd6d459856729c972fd7748b621795f17..d8b419a6bbeae515998ec8a13cb7b5b8e0flb04 100644
---
a/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientZonesTest.kt
+++
b/app/src/test/kotlin/cloud/mws/devp/testservice/client/HelloApiClientZonesTest.kt
@@ -1,27 +1,23 @@
package cloud.mws.devp.testservice.client

import cloud.mws.api.devp.examples.client.HelloApiClient
+import cloud.mws.devp.testservice.BaseApiServerMockTest
import org.junit.jupiter.api.DisplayName
import org.junit.jupiter.api.Test
-import org.junit.jupiter.api.extension.ExtendWith
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.beans.factory.annotation.Qualifier
-import org.springframework.boot.test.context.SpringBootTest
import org.springframework.test.context.ActiveProfiles
-import
org.springframework.test.context.junit.jupiter.SpringExtension
import kotlin.test.assertEquals
import kotlin.test.assertTrue

-@ExtendWith(SpringExtension::class)
-@SpringBootTest

```

```

    @ActiveProfiles("test")
- class HelloApiClientZonesTest {
+ class HelloApiClientZonesTest : BaseApiServerMockTest() {

        @Autowired
        private lateinit var helloApiClientList: List<HelloApiClient>

        @Autowired
-        @Qualifier("HelloApiClientMap")
+        @Qualifier("devp.test-service.HelloApiClientMap")
        private lateinit var helloApiClientMap: Map<String,
HelloApiClient>

        @Test
diff --git
a/app/src/test/kotlin/cloud/mws/devp/testservice/controller/HelloRe
activeApiControllerTest.kt
b/app/src/test/kotlin/cloud/mws/devp/testservice/controller/HelloRe
activeApiControllerTest.kt
index
40080fdb7b17b1010c2e1dad58895450a4ea5422..b77da43dd583b761cd1480faf
a3cb9727b3226fd 100644
---
a/app/src/test/kotlin/cloud/mws/devp/testservice/controller/HelloRe
activeApiControllerTest.kt
+++
b/app/src/test/kotlin/cloud/mws/devp/testservice/controller/HelloRe
activeApiControllerTest.kt
@@ -6,6 +6,7 @@ import
cloud.mws.devp.testservice.BaseApiServerMockTest
import jakarta.validation.ConstraintViolationException
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.test.runTest
+import org.apache.commons.lang3.RandomStringUtils
import org.assertj.core.api.Assertions
import org.junit.jupiter.api.DisplayName
import org.junit.jupiter.api.Test
@@ -14,7 +15,6 @@ import org.junit.jupiter.api.assertThrows
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.context.i18n.LocaleContextHolder
import org.springframework.test.context.ActiveProfiles
- import
org.testcontainers.shaded.org.apache.commons.lang3.RandomStringUtil
s
import java.util.Locale
import kotlin.test.assertEquals
import kotlin.test.assertNotNull
diff --git a/app/src/test/resources/application-test.yaml
b/app/src/test/resources/application-test.yaml
index
e8b3475f4cf6236a01dd59aa5cd18e3e6096ecb6..5a3660f03848bee472d262625
4c5b0eb568ac0b9 100644
--- a/app/src/test/resources/application-test.yaml
+++ b/app/src/test/resources/application-test.yaml
@@ -11,9 +11,6 @@ cloud:
        settings:

```

```

retryEnabled: false
devp:
- r2dbc:
- migrations:
- enabled: false
api:
validation:
enabled: true
diff --git a/gradle.properties b/gradle.properties
index
9609f31f98dbd1fcf9b958c5b72a1aa2bf3e6fbfd..b0ea31d7ada072a87a72c071c
7657deb9cb4bb28 100644
--- a/gradle.properties
+++ b/gradle.properties
@@ -1,4 +1,4 @@
version=0.0.4-SNAPSHOT
-platformVersion=0.62.0+3
-cloudApiVersion=1.32.389
+platformVersion=0.74.0+1
+cloudApiVersion=1.34.87
kotlin.code.style=official
diff --git a/query/cloud-sensitive.http
b/query/cloud-sensitive.http
new file mode 100644
index
0000000000000000000000000000000000000000..e4ff185a02930951b323dad0b
51318bb99e3edfe
--- /dev/null
+++ b/query/cloud-sensitive.http
@@ -0,0 +1,273 @@
#### 1. GET + CloudSensitive in body response
##### Server
+GET http://localhost:8888/cloudSensitive/test-object-1
+Accept: application/json
+
+### 1. GET + CloudSensitive in body response
##### Client
+GET
http://localhost:8888/test/cloud-sensitive/client/get/test-object-1
-client
+Accept: application/json
+
+### 2. GET + CloudSensitive in body request
##### Server
+GET http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+ "metadata": {
+   "name": "get-body-test-1"
+ },
+ "spec": {
+   "regularField": "custom-regular-value",
+   "sensitiveField": "custom-secret-value",
+   "requiredSensitiveField": "required-secret-123",

```

```

+   "sensitiveArray": ["array-secret-1", "array-secret-2"],
+   "sensitiveMap": {
+     "map-key-1": "map-value-1",
+     "map-key-2": "map-value-2"
+   },
+   "nestedObject": {
+     "nestedRegularField": "nested-regular-value",
+     "nestedSensitiveField": "nested-secret-value"
+   },
+   "sensitiveInArrayOfObjects": [
+     {
+       "name": "object1",
+       "secret": "object-secret-1"
+     },
+     {
+       "name": "object2",
+       "secret": "object-secret-2"
+     }
+   ]
+ }
+}
+
+### 2. GET + CloudSensitive in body request
+#### Client
+GET
+http://localhost:8888/test/cloud-sensitive/client/get-with-body?id=
+get-body-client-1&regularField=client-regular&sensitiveValue=client
+-secret
+Accept: application/json
+
+### 3. POST + CloudSensitive in body request & response
+#### Server
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "post-test-object-1"
+  },
+  "spec": {
+    "regularField": "post-regular-value",
+    "sensitiveField": "post-secret-value",
+    "requiredSensitiveField": "post-required-secret",
+    "sensitiveArray": ["post-secret-1", "post-secret-2",
+"post-secret-3"],
+    "sensitiveMap": {
+      "env": "production",
+      "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"
+    },
+    "nestedObject": {
+      "nestedRegularField": "post-nested-regular",
+      "nestedSensitiveField": "post-nested-secret"
+    },
+    "sensitiveInArrayOfObjects": [
+      {

```

```

+         "name": "database",
+         "secret": "postgres://user:pass@localhost:5432/db"
+     },
+     {
+         "name": "redis",
+         "secret": "redis://:password@localhost:6379"
+     },
+     {
+         "name": "api-key",
+         "secret": "sk_live_12345abcde"
+     }
+ ]
+ }
+}
+
+### 3. POST + CloudSensitive in body request & response
+#### Client
+POST
+http://localhost:8888/test/cloud-sensitive/client/post?id=post-client-1&regularField=client-post-regular&sensitiveValue=client-post-secret
+Accept: application/json
+
+### 3. POST + CloudSensitive in body request & response
+#### POST with minimal request data
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "post-minimal-test"
+  },
+  "spec": {
+    "requiredSensitiveField": "minimal-required-secret"
+  }
+}
+
+### 4. List<> with CloudSensitive in response
+#### Server
+GET http://localhost:8888/cloudSensitive/list
+Accept: application/json
+
+### 4. List<> with CloudSensitive in response
+#### Client
+GET http://localhost:8888/test/cloud-sensitive/client/list
+Accept: application/json
+
+### Creation
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "full-cycle-test"

```



```

+### Only required
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "required-only"
+  },
+  "spec": {
+    "requiredSensitiveField": "only-required-field"
+  }
+}
+
+### Empty lists
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "empty-arrays"
+  },
+  "spec": {
+    "requiredSensitiveField": "required-value",
+    "sensitiveArray": [],
+    "sensitiveInArrayOfObjects": []
+  }
+}
+
+### With nulls
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{
+  "metadata": {
+    "name": "null-values"
+  },
+  "spec": {
+    "requiredSensitiveField": "required-value",
+    "regularField": null,
+    "sensitiveField": null,
+    "sensitiveArray": null,
+    "sensitiveMap": null,
+    "nestedObject": null,
+    "sensitiveInArrayOfObjects": null
+  }
+}
+
+### Many sensitive fields
+POST http://localhost:8888/cloudSensitive
+Accept: application/json
+Content-Type: application/json
+
+{

```



```
+package cloud.mws.devptestservice.services
+
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectResponse
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveRequestBodyResponse
+
+interface CloudSensitiveService {
+
+    suspend fun getCloudSensitive(objectId: String):
CloudSensitiveObjectResponse
+
+    suspend fun getCloudSensitiveWithBody(
+        id: String,
+        regularField: String? = null,
+        sensitiveValue: String? = null
+    ): CloudSensitiveObjectResponse
+
+    suspend fun postCloudSensitive(
+        id: String,
+        regularField: String? = null,
+        sensitiveValue: String? = null
+    ): CloudSensitiveRequestBodyResponse
+
+    suspend fun getCloudSensitiveList():
List<CloudSensitiveObjectResponse>
+
+    suspend fun fullCycle(id: String): Map<String, Any>
+}
diff --git
a/services/src/main/kotlin/cloud/mws/devptestservice/services/ipc/
IpcCloudSensitiveService.kt
b/services/src/main/kotlin/cloud/mws/devptestservice/services/ipc/
IpcCloudSensitiveService.kt
new file mode 100644
index
00000000000000000000000000000000..a2bc2171799b9922013ca92b3
fela2dedf7f2126
--- /dev/null
+++
b/services/src/main/kotlin/cloud/mws/devptestservice/services/ipc/
IpcCloudSensitiveService.kt
@@ -0,0 +1,111 @@
+package cloud.mws.devptestservice.services.ipc
+
+import cloud.mws.api.devp.examples.client.CloudSensitiveApiClient
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectNestedObjectReq
uest
+import cloud.mws.api.devp.examples.dto.CloudSensitiveObjectRequest
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectResponse
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveObjectSensitiveInArra
yOfObjectsRequest
+import
```

```

cloud.mws.api.devp.examples.dto.CloudSensitiveRequestBodyRequest
+import
cloud.mws.api.devp.examples.dto.CloudSensitiveRequestBodyResponse
+import cloud.mws.common.dto.CommonTypedResourceMetadataRequest
+import cloud.mws.devp.cloud.sensitive.CloudSensitive
+import cloud.mws.devp.testservice.services.CloudSensitiveService
+import org.springframework.beans.factory.annotation.Qualifier
+import org.springframework.stereotype.Component
+
+@Component
+class IpcCloudSensitiveService(
+    @Qualifier("devp.test-service.CloudSensitiveApiClientGlobal")
+    private val cloudSensitiveClient: CloudSensitiveApiClient
+) : CloudSensitiveService {
+
+    override suspend fun getCloudSensitive(objectId: String):
CloudSensitiveObjectResponse {
+        return cloudSensitiveClient.getCloudSensitive(objectId)
+    }
+
+    override suspend fun getCloudSensitiveWithBody(
+        id: String,
+        regularField: String?,
+        sensitiveValue: String?
+    ): CloudSensitiveObjectResponse {
+        val request = CloudSensitiveRequestBodyRequest(
+            metadata = CommonTypedResourceMetadataRequest().apply
+        {
+            name = id
+        },
+        spec = CloudSensitiveObjectRequest(
+            regularField = regularField,
+            sensitiveField = sensitiveValue?.let {
CloudSensitive(it) },
+            requiredSensitiveField =
CloudSensitive("required-$id"),
+            sensitiveArray = listOf(CloudSensitive("array1"),
CloudSensitive("array2")),
+            sensitiveMap = mapOf("mapKey" to
CloudSensitive("mapValue")),
+            nestedObject =
CloudSensitiveObjectNestedObjectRequest(
+                nestedRegularField = "nested-regular-$id",
+                nestedSensitiveField =
CloudSensitive("nested-secret-$id")
+            ),
+            sensitiveInArrayOfObjects = listOf(
+
CloudSensitiveObjectSensitiveInArrayOfObjectsRequest(
+                name = "client-item1",
+                secret = CloudSensitive("client-secret1")
+            ),
+
CloudSensitiveObjectSensitiveInArrayOfObjectsRequest(
+                name = "client-item2",
+                secret = CloudSensitive("client-secret2")

```

```

+         )
+     )
+ )
+     return
+ cloudSensitiveClient.getCloudSensitiveWithBody(request)
+ }
+
+     override suspend fun postCloudSensitive(
+         id: String,
+         regularField: String?,
+         sensitiveValue: String?
+     ): CloudSensitiveRequestBodyResponse {
+         val request = CloudSensitiveRequestBodyRequest(
+             metadata = CommonTypedResourceMetadataRequest().apply {
+                 name = id
+             },
+             spec = CloudSensitiveObjectRequest(
+                 regularField = regularField ?: "post-regular-$id",
+                 sensitiveField = CloudSensitive(sensitiveValue ?:
"post-secret-$id"),
+                 requiredSensitiveField =
CloudSensitive("post-required-$id"),
+                 sensitiveArray =
listOf(CloudSensitive("post-array1"),
CloudSensitive("post-array2")),
+                 sensitiveMap = mapOf("post-key" to
CloudSensitive("post-value")),
+                 nestedObject =
CloudSensitiveObjectNestedObjectRequest(
+                     nestedRegularField =
"post-nested-regular-$id",
+                     nestedSensitiveField =
CloudSensitive("post-nested-secret-$id")
+                 ),
+                 sensitiveInArrayOfObjects = listOf(
+                     CloudSensitiveObjectSensitiveInArrayOfObjectsRequest(
+                         name = "post-item1",
+                         secret = CloudSensitive("post-secret1")
+                     )
+                 )
+             )
+         )
+         return cloudSensitiveClient.postCloudSensitive(request)
+     }
+
+     override suspend fun getCloudSensitiveList():
List<CloudSensitiveObjectResponse> {
+         return cloudSensitiveClient.getCloudSensitiveList()
+     }
+
+     override suspend fun fullCycle(id: String): Map<String, Any> {
+         val created = postCloudSensitive(id, "full-cycle-regular",
"full-cycle-secret")

```

```

+
+         val retrieved = getCloudSensitive(id)
+
+         val retrievedWithBody = getCloudSensitiveWithBody(
+             id = "$id-modified",
+             regularField = "modified-regular",
+             sensitiveValue = "modified-secret"
+         )
+
+         return mapOf(
+             "created" to created,
+             "retrieved" to retrieved,
+             "retrievedWithBody" to retrievedWithBody
+         )
+     }
+}
diff --git
a/services/src/main/kotlin/cloud/mws/devp/testservice/services/ipc/
IpcHelloService.kt
b/services/src/main/kotlin/cloud/mws/devp/testservice/services/ipc/
IpcHelloService.kt
index
dc832a365a3294e3cc4db2580f4f2a389932ec7d..713be2c3944998ec73ec3f748
20c5f62d5f738d6 100644
---
a/services/src/main/kotlin/cloud/mws/devp/testservice/services/ipc/
IpcHelloService.kt
+++
b/services/src/main/kotlin/cloud/mws/devp/testservice/services/ipc/
IpcHelloService.kt
@@ -24,10 +24,10 @@ import org.springframework.stereotype.Component
@Component
@Suppress("UnusedPrivateProperty")
internal open class IpcHelloService(
-    @Qualifier("HelloApiClientZone1") private val
helloApiClientZone1: HelloApiClient,
-    @Qualifier("HelloApiClientZone2") private val
helloApiClientZone2: HelloApiClient,
-    @Qualifier("HelloApiClientGlobal") private val
helloApiClientGlobal: HelloApiClient,
-    @Qualifier("HelloApiClientMap") private val helloApiClientMap:
Map<String, HelloApiClient>,
+    @Qualifier("devp.test-service.HelloApiClientZone1") private
val helloApiClientZone1: HelloApiClient,
+    @Qualifier("devp.test-service.HelloApiClientZone2") private
val helloApiClientZone2: HelloApiClient,
+    @Qualifier("devp.test-service.HelloApiClientGlobal") private
val helloApiClientGlobal: HelloApiClient,
+    @Qualifier("devp.test-service.HelloApiClientMap") private val
helloApiClientMap: Map<String, HelloApiClient>,
    private val translator: BusinessExceptionTranslator
) : HelloService {

```

Листинг Д.3 – Листинг изменений в тестовом сервисе

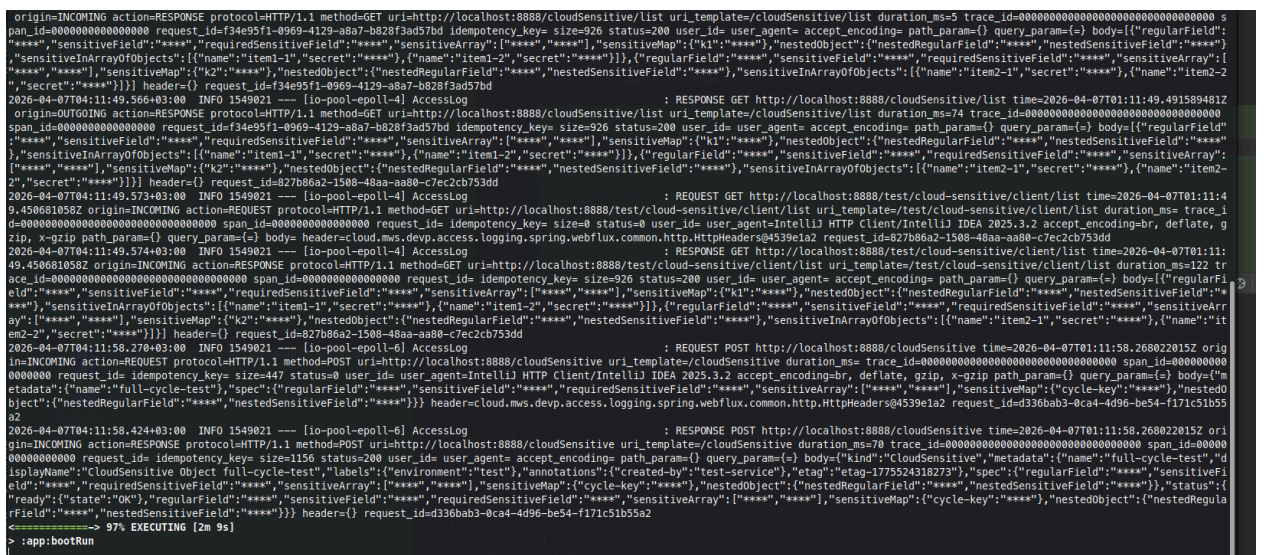
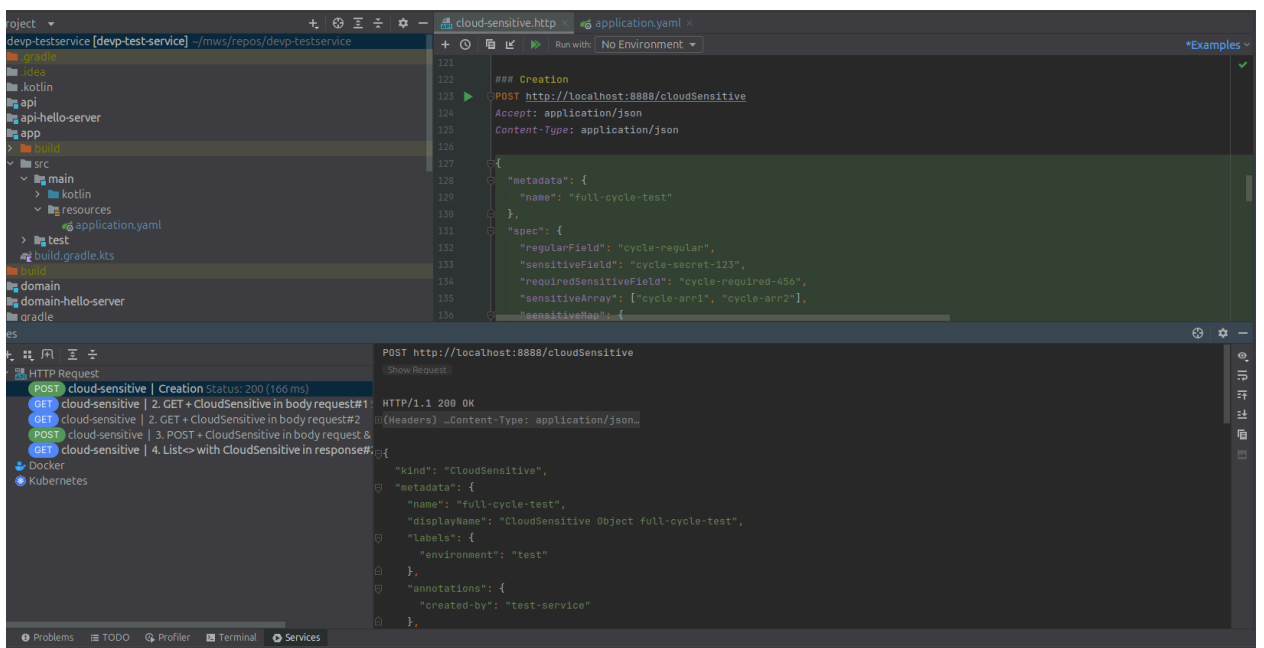


Рисунок Д.4 – Скриншоты логов после внедрения

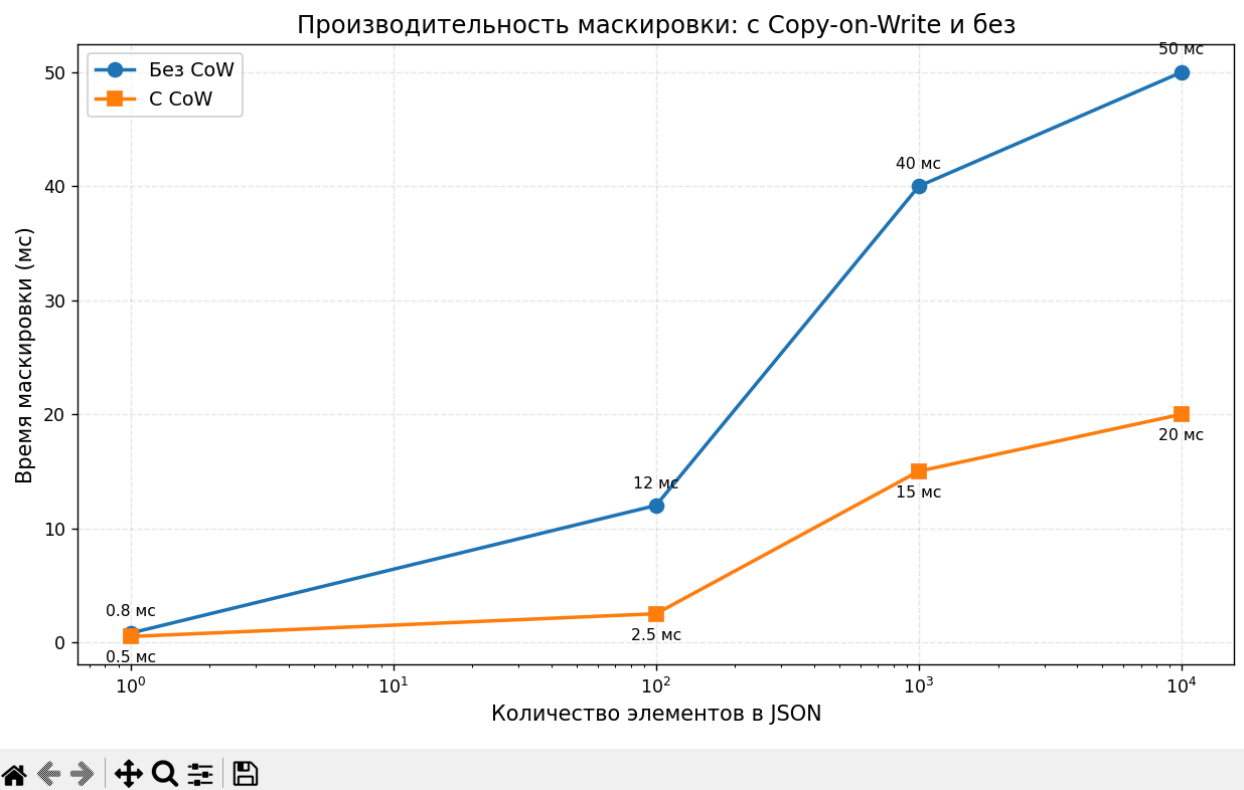


Рисунок Д.5 – График производительности, построенный на основе замеров

ПРИЛОЖЕНИЕ Е

```
devp:
  api:
    access-log:
      strategy: WITH_BODY # WITH_BODY /
WITHOUT_BODY # AUTO / RAW
      body-serialization-mode: AUTO # подавлять ошибки
      suppress-unsupported-types: false # дополнительные ключи
неподдерживаемых типов
      sensitive-json-keys:
для маскировки
      - apiKey
      - creditCard
      sensitive-key-detection-strategy: CONTAINS # EXACT /
CONTAINS
      with-headers: # белый список
заголовков
      - X-Request-Id
      without-body-endpoints: # endpoint'ы, чьи тела
не логируются
      - /metrics
```

Листинг Е.1 – Фрагмент конфигурации (application.yml)

```
origin=INCOMING action=RESPONSE protocol=HTTP/1.1 method=GET uri=http://localhost:8888/cloudSensitive/list uri_template=/cloudSensitive/list duration_ms=5 trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id=f34e95f1-0969-4129-a8a7-b828f3ad57bd idempotency_key= size=926 status=200 user_id= user_agent= accept_encoding= path_param={} query_param={} body={"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k1": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item1-1", "secret": "****"}, {"name": "item1-2", "secret": "****"}]}, {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k2": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item2-1", "secret": "****"}, {"name": "item2-2", "secret": "****"}]}]} headers={} request_id=f34e95f1-0969-4129-a8a7-b828f3ad57bd
2026-04-07T04:11:49.566+03:00 INFO 1549021 --- [io-pool-epoll-4] AccessLog : RESPONSE GET http://localhost:8888/cloudSensitive/list time=2026-04-07T01:11:49.491589481Z
origin=OUTGOING action=RESPONSE protocol=HTTP/1.1 method=GET uri=http://localhost:8888/cloudSensitive/list uri_template=/cloudSensitive/list duration_ms=74 trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id=f34e95f1-0969-4129-a8a7-b828f3ad57bd idempotency_key= size=926 status=200 user_id= user_agent= accept_encoding= path_param={} query_param={} body={"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k1": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item1-1", "secret": "****"}, {"name": "item1-2", "secret": "****"}]}, {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k2": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item2-1", "secret": "****"}, {"name": "item2-2", "secret": "****"}]}]} headers={} request_id=827b86a2-1508-48aa-a808-c7ec2cb753dd
2026-04-07T04:11:49.573+03:00 INFO 1549021 --- [io-pool-epoll-4] AccessLog : REQUEST GET http://localhost:8888/test/cloud-sensitive/client/list time=2026-04-07T01:11:49.450681058Z origin=INCOMING action=REQUEST protocol=HTTP/1.1 method=GET uri=http://localhost:8888/test/cloud-sensitive/client/list uri_template=/test/cloud-sensitive/client/list duration_ms= trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id= idempotency_key= size=0 status=0 user_id= user_agent=IntelliJ HTTP Client/IntelliJ IDEA 2025.3.2 accept_encoding=br, deflate, gzip, x-gzip path_param={} query_param={} body= headers=cloud.mvs.dev.access.logging.spring.webflux.common.http.HttpHeaders@4539e1a2 request_id=827b86a2-1508-48aa-a808-c7ec2cb753dd
2026-04-07T04:11:49.574+03:00 INFO 1549021 --- [io-pool-epoll-4] AccessLog : RESPONSE GET http://localhost:8888/test/cloud-sensitive/client/list time=2026-04-07T01:11:49.450681058Z origin=INCOMING action=RESPONSE protocol=HTTP/1.1 method=GET uri=http://localhost:8888/test/cloud-sensitive/client/list uri_template=/test/cloud-sensitive/client/list duration_ms=122 trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id= idempotency_key= size=926 status=200 user_id= user_agent= accept_encoding= path_param={} query_param={} body={"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k1": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item1-1", "secret": "****"}, {"name": "item1-2", "secret": "****"}]}, {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k2": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item2-1", "secret": "****"}, {"name": "item2-2", "secret": "****"}]}]} headers={} request_id=827b86a2-1508-48aa-a808-c7ec2cb753dd
2026-04-07T04:11:50.270+03:00 INFO 1549021 --- [io-pool-epoll-6] AccessLog : REQUEST POST http://localhost:8888/cloudSensitive time=2026-04-07T01:11:50.268022015Z origin=INCOMING action=REQUEST protocol=HTTP/1.1 method=POST uri=http://localhost:8888/cloudSensitive uri_template=/cloudSensitive duration_ms=70 trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id= idempotency_key= size=1156 status=200 user_id= user_agent=IntelliJ HTTP Client/IntelliJ IDEA 2025.3.2 accept_encoding=br, deflate, gzip, x-gzip path_param={} query_param={} body={"metadata": {"name": "full-cycle-test"}, "spec": {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k1": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item1-1", "secret": "****"}, {"name": "item1-2", "secret": "****"}]}, {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k2": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item2-1", "secret": "****"}, {"name": "item2-2", "secret": "****"}]}]} headers=cloud.mvs.dev.access.logging.spring.webflux.common.http.HttpHeaders@4539e1a2 request_id=d336bab3-0ca4-4d96-be54-f171c51b55a2
2026-04-07T04:11:58.424+03:00 INFO 1549021 --- [io-pool-epoll-6] AccessLog : RESPONSE POST http://localhost:8888/cloudSensitive time=2026-04-07T01:11:58.268022015Z origin=INCOMING action=RESPONSE protocol=HTTP/1.1 method=POST uri=http://localhost:8888/cloudSensitive uri_template=/cloudSensitive duration_ms=70 trace_id=00000000000000000000000000000000 span_id=0000000000000000 request_id= idempotency_key= size=1156 status=200 user_id= user_agent=IntelliJ HTTP Client/IntelliJ IDEA 2025.3.2 accept_encoding=br, deflate, gzip, x-gzip path_param={} query_param={} body={"metadata": {"name": "full-cycle-test"}, "spec": {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k1": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item1-1", "secret": "****"}, {"name": "item1-2", "secret": "****"}]}, {"regularField": "****", "sensitiveField": "****", "requiredSensitiveField": "****", "sensitiveArray": [{"k2": "****"}, {"nestedObject": {"nestedRegularField": "****", "nestedSensitiveField": "****"}, "sensitiveInArrayOfObjects": [{"name": "item2-1", "secret": "****"}, {"name": "item2-2", "secret": "****"}]}]} headers=cloud.mvs.dev.access.logging.spring.webflux.common.http.HttpHeaders@4539e1a2 request_id=d336bab3-0ca4-4d96-be54-f171c51b55a2
<===== 97% EXECUTING [2m 9s]
> :app:bootRun
```

Рисунок Е.2 – Примеры access-логов после внедрения

```

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-client-spring-boot-starter/src/main/kotlin/cloud/mws/devp
/access/logging/spring/webflux/spring/boot/starter/AccessLogClientA
utoConfiguration.kt =====
package
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter

import cloud.mws.devp.access.logging.AccessLogWriter
import cloud.mws.devp.access.logging.LoggerAccessLogWriter
import
cloud.mws.devp.access.logging.spring.webflux.AccessLogExchangeFilt
erFunction
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessi
ngStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.AutoBodySe
rializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySerial
izationMode
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.CompositeS
ensitiveMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.ContainsMa
tchStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.DefaultReq
uestProcessingStage
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.ExactMatch
Strategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.LoggerSink
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.NoOpSensit
iveKeyStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.RawBodySer
ializationStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.SensitiveK
eyDetectionStrategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.SensitiveK
eyMasker
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithBodySt
rategy
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.WithoutBod
yStrategy
import com.fasterxml.jackson.module.kotlin.jacksonObjectMapper

```

```

import org.springframework.boot.autoconfigure.AutoConfiguration
import
org.springframework.boot.autoconfigure.condition.ConditionalOnBean
import
org.springframework.boot.autoconfigure.condition.ConditionalOnProperty
import
org.springframework.boot.context.properties.EnableConfigurationProperties
import
org.springframework.boot.web.reactive.function.client.WebClientCustomizer
import org.springframework.context.annotation.Bean
import org.springframework.core.Ordered
import org.springframework.core.annotation.Order

/**
 * Автоконфигурация для записи access логов HTTP клиентов.
 */
@AutoConfiguration
@EnableConfigurationProperties(AccessLogProperties::class)
open class AccessLogClientAutoConfiguration {

    /**
     * Бин для непосредственной записи данных в лог.
     */
    @Bean("devp.AccessLogWriter")
    open fun accessLogWriter() = LoggerAccessLogWriter()

    /**
     * Бин для построения процесса обработки request'а и записи его
     в лог.
     * По умолчанию access логи записываются без body
     */
    @Bean("devp.RequestProcessingStage")
    open fun requestProcessingStage(
        accessLogProperties: AccessLogProperties,
        accessLogWriter: AccessLogWriter,
        compositeSensitiveMasker: CompositeSensitiveMasker
    ): RequestProcessingStage {
        val strategy = when (accessLogProperties.strategy) {
            "WITHOUT_BODY" -> WithoutBodyStrategy()
            "WITH_BODY" ->
WithBodyStrategy(accessLogProperties.withoutBodyEndpoints)
            else -> WithoutBodyStrategy()
        }

        val bodySerializationStrategy = when
(accessLogProperties.bodySerializationMode) {
            BodySerializationMode.RAW ->
RawBodySerializationStrategy()
            BodySerializationMode.AUTO ->
AutoBodySerializationStrategy(
                accessLogProperties.suppressUnsupportedTypes,
                compositeSensitiveMasker
            )
        }
    }
}

```

```

    }

    val sink = LoggerSink(
        accessLogWriter = accessLogWriter,
        headersWhiteList = accessLogProperties.withHeaders,
        bodySerializationStrategy = bodySerializationStrategy
    )

    return DefaultRequestProcessingStage(strategy = strategy,
sink = sink)
    }

    /**
     * Бин встраивается в процесс формирования запроса от клиента и
добавляет шаги для отправки в лог.
     */
    @Bean("devp.AccessLoggingExchangeFilterFunction")
    @Order(Ordered.HIGHEST_PRECEDENCE)
    @ConditionalOnProperty(
        "devp.api.access-log.client-log-enabled",
        havingValue = "true",
        matchIfMissing = true
    )
    open fun
accessLoggingExchangeFilterFunction(requestProcessingStage:
RequestProcessingStage) =
        AccessLogExchangeFilterFunction(requestProcessingStage)

    /**
     * Кастомизация WebClient'а обработки клиентских запросов.
     */
    @Bean("devp.WebClientAccessLogCustomizer")
    @Order(Ordered.HIGHEST_PRECEDENCE)
    @ConditionalOnBean(AccessLogExchangeFilterFunction::class)
    open fun webClientAccessLogCustomizer(logFilterFunction:
AccessLogExchangeFilterFunction): WebClientCustomizer {
        return WebClientCustomizer { it.filter(logFilterFunction) }
    }

    /**
     * Создание маскировщика.
     */
    @Bean
    @ConditionalOnProperty(
        name = ["devp.api.access-log.body-serialization-mode"],
        havingValue = "AUTO",
        matchIfMissing = true
    )
    open fun compositeSensitiveMasker(
        accessLogProperties: AccessLogProperties,
        sensitiveJsonKeys: List<String>
    ): CompositeSensitiveMasker {
        val objectMapper = jacksonObjectMapper()

        val strategy = if (sensitiveJsonKeys.isEmpty()) {
            NoOpSensitiveKeyStrategy

```

```

        } else {
            when
                (accessLogProperties.sensitiveKeyDetectionStrategy) {
                    SensitiveKeyDetectionStrategy.EXACT ->
                        ExactMatchStrategy(sensitiveJsonKeys.toSet())
                    SensitiveKeyDetectionStrategy.CONTAINS ->
                        ContainsMatchStrategy(sensitiveJsonKeys)
                }
        }

        val sensitiveKeyMasker = SensitiveKeyMasker(objectMapper,
            strategy)
        return CompositeSensitiveMasker(objectMapper,
            sensitiveKeyMasker)
    }

    @Bean
    @ConditionalOnProperty(
        name = ["devp.api.access-log.body-serialization-mode"],
        havingValue = "AUTO",
        matchIfMissing = true
    )
    open fun sensitiveJsonKeys(accessLogProperties:
        AccessLogProperties): List<String> {
        val defaultSensitiveKeys = listOf(
            "password",
            "token",
            "secret",
        )

        return defaultSensitiveKeys +
            accessLogProperties.sensitiveJsonKeys
    }
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-client-spring-boot-starter/src/main/kotlin/cloud/mws/devp
/access/logging/spring/webflux/spring/boot/starter/AccessLogPropert
ies.kt =====
package
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter

import
cloud.mws.devp.access.logging.spring.webflux.common.impl.BodySerial
izationMode
import
cloud.mws.devp.access.logging.spring.webflux.common.impl.SensitiveK
eyDetectionStrategy
import
org.springframework.boot.context.properties.ConfigurationProperties

@ConfigurationProperties(prefix = "devp.api.access-log")
open class AccessLogProperties {
    /**

```

```

    * Выбранная стратегия записи http request и response в access
log.
    * Возможно 2 варианта: WITH_BODY, WITHOUT_BODY (по умолчанию).
    */
    var strategy: String? = null

    /**
    * Логгирование запросов клиентов. По умолчанию true
(включено).
    */
    var clientLogEnabled: Boolean = true

    /**
    * Логирование запросов к серверу. По умолчанию true
(включено).
    * Активируется если в проекте подключен серверный стартер.
    */
    var serverLogEnabled: Boolean = true

    /**
    * Префиксы endpoints для которых боди не логируется.
    */
    var withoutBodyEndpoints: List<String> = listOf("/metrics",
"/customer-metrics")

    /**
    * Список header'ов, которые будут логироваться.
    */
    var withHeaders: List<String> = emptyList()

    /**
    * Режим сериализации тела запроса/ответа.
    * RAW - тело логируется как есть
    * AUTO - маскирует CloudSensitive поля в JSON контенте
    */
    var bodySerializationMode: BodySerializationMode =
BodySerializationMode.AUTO

    /**
    * Если true, то при обнаружении неподдерживаемого типа тело
логируется в исходном виде (как в режиме RAW),
    * и предупреждение не выводится. Применимо только при
bodySerializationMode = AUTO.
    */
    var suppressUnsupportedTypes: Boolean = false

    /**
    * Список ключей JSON, значения которых должны быть
замаскированы.
    * Работает только при bodySerializationMode = AUTO.
    */
    var sensitiveJsonKeys: List<String> = emptyList()

    /**
    * Стратегия обнаружения чувствительных ключей.
    * EXACT - полное совпадение имени ключа

```

```

    * CONTAINS (по умолчанию) - ключ содержит подстроку из списка
    (без учета регистра)
    */
    var sensitiveKeyDetectionStrategy:
SensitiveKeyDetectionStrategy =
SensitiveKeyDetectionStrategy.CONTAINS
}

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-client-spring-boot-starter/src/main/resources/META-INF/ad
ditional-spring-configuration-metadata.json =====
{
  "hints": [
    {
      "name": "devp.api.access-log.strategy",
      "values": [
        {
          "value": "WITH_BODY"
        },
        {
          "value": "WITHOUT_BODY"
        }
      ]
    },
    {
      "name": "devp.api.access-log.body-serialization-mode",
      "values": [
        {
          "value": "RAW",
          "description": "Тело логируется как есть."
        },
        {
          "value": "AUTO",
          "description": "Маскирует CloudSensitive поля в JSON
контенте."
        }
      ]
    },
    {
      "name": "devp.api.access-log.suppress-unsupported-types",
      "values": [
        {
          "value": false,
          "description": "Тело не логируется, записывается
[UNSUPPORTED_TYPE], выводится предупреждение."
        },
        {
          "value": true,
          "description": "Тело логируется в исходном виде (как в
режиме RAW), предупреждение подавляется."
        }
      ]
    }
  ],
  {

```



```

        "name":
"devp.api.access-log.sensitive-key-detection-strategy",
        "values": [
            {
                "value": "EXACT",
                "description": "Полное совпадение имени ключа со
значением из списка."
            },
            {
                "value": "CONTAINS",
                "description": "Имя ключа содержит подстроку из списка
чувствительных ключей (без учета регистра)."

```

```

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-client-spring-boot-starter/src/main/resources/META-INF/sp

```

```

ring/org.springframework.boot.autoconfigure.AutoConfiguration.import
ts =====
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter.Ac
cessLogClientAutoConfiguration

=====
/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-server-spring-boot-starter/src/main/kotlin/cloud/mws/devp
/access/logging/spring/webflux/spring/boot/starter/AccessLogServerA
utoConfiguration.kt =====
package
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter

import
cloud.mws.devp.access.logging.spring.webflux.AccessLogWebFilter
import
cloud.mws.devp.access.logging.spring.webflux.common.RequestProcessi
ngStage
import org.springframework.boot.autoconfigure.AutoConfiguration
import
org.springframework.boot.autoconfigure.condition.ConditionalOnPrope
rty
import
org.springframework.boot.autoconfigure.condition.ConditionalOnWebAp
plication
import org.springframework.context.annotation.Bean
import org.springframework.core.Ordered
import org.springframework.core.annotation.Order
import org.springframework.core.env.Environment
import
org.springframework.web.reactive.result.method.annotation.RequestMa
ppingHandlerMapping

/**
 * Автоконфигурация для записи access логов на сервере.
 */
@AutoConfiguration
@ConditionalOnWebApplication(type =
ConditionalOnWebApplication.Type.REACTIVE)
open class AccessLogServerAutoConfiguration {

    /**
     * Фильтр для перехвата всех входящих запросов и последующей
     обработки, для вывода данных в лог.
     */
    @Bean("devp.AccessLogWebFilter")
    @Order(Ordered.HIGHEST_PRECEDENCE)
    @ConditionalOnProperty(
        "devp.api.access-log.server-log-enabled",
        havingValue = "true",
        matchIfMissing = true
    )
    open fun accessLogWebFilter(
        requestProcessingStage: RequestProcessingStage,
        requestMappingHandlerMapping: RequestMappingHandlerMapping,

```

```
        environment: Environment,  
    ) = AccessLogWebFilter(requestProcessingStage,  
requestMappingHandlerMapping, environment)  
}  
  
=====
```

/home/mbarsukov/mws/repos/platform-jvm/libraries/devp-access-loggin
g-webflux-server-spring-boot-starter/src/main/resources/META-INF/sp
ring/org.springframework.boot.autoconfigure.AutoConfiguration.impor
ts =====

```
cloud.mws.devp.access.logging.spring.webflux.spring.boot.starter.Ac  
cessLogServerAutoConfiguration
```

Листинг Е.3 – Листинг автоконфигурации